

SFAAO 2.4.0

Software Forense para Análise de Assinaturas Offline

Manual Técnico, Científico e Operacional

Edição Consolidada do Release 2.4.0

Diógenes Antonio Marques José

Universidade do Estado de Mato Grosso - UNEMAT

SFAAO 2.4.0 - documentação da distribuição multiplataforma

Registro INPI BR512026005360-9

28 de outubro de 2026

Documento técnico-científico e operacional. Distribuição do software sem datasets de terceiros e sem modelos treinados.

Sumário

Atualização documental do release 2.4.0 **1**

1 Fundamentos, objetivos e escopo **3**

1.1 Visão geral . . . 3

1.1.1 Por que o SFAAO é importante . . . 3

1.2 Filosofia da plataforma . . . 5

1.3 Problema técnico . . . 5

1.4 Objetivos . . . 6

1.5 Escopo e limites . . . 6

1.6 Princípios arquiteturais . . . 6

1.6.1 Responsabilidade única . . . 6

1.6.2 Separação de responsabilidades . . . 6

1.6.3 Baixo acoplamento . . . 6

1.6.4 Alta coesão . . . 6

1.6.5 Arquitetura modular . . . 6

1.6.6 Reprodutibilidade . . . 6

1.6.7 Rastreabilidade . . . 7

1.6.8 Evolução controlada . . . 7

1.7 Conceitos fundamentais . . . 7

2 Arquitetura e organização da plataforma **11**

2.1 Arquitetura geral . . . 11

2.2 Organização do projeto . . . 12

2.3 Interface gráfica . . . 12

2.4 Controllers . . . 13

2.5 Persistência . . . 14

2.6 Storage . . . 14

3 Fundamentos do núcleo de inteligência artificial **15**

3.1 Núcleo de IA . . . 15

3.2 Framework modular de IA . . . 15

3.3 Backbones . . . 16

3.4 Embeddings . . . 17

3.5 Funções de perda . . . 17

3.5.1 Contrastive Loss . . . 17

3.5.2 Cosine Contrastive Loss . . . 17

3.5.3 Triplet Loss . . . 17
3.5.4 ArcFace . . . 17
3.6 Métricas e distâncias . . . 18
3.6.1 Similaridade por cosseno . . . 18
3.6.2 Distância Euclidiana . . . 18
3.7 Preprocessamento . . . 18
3.7.1 Conformidade com a Especificação Oficial da Arquitetura . . . 19

4 Datasets, treinamento e inferência 21
4.1 Datasets e profiles . . . 21
4.2 Pipeline detalhado de cadastro e validação de dataset . . . 21
4.3 PairBuilder . . . 22
4.3.1 Hiperparâmetros, componentes configuráveis e modos de treinamento . . . 22
4.4 Pipeline detalhado de treinamento . . . 28
4.5 ModelConfig . . . 29
4.6 TrainingConfig . . . 29
4.7 Early stopping [6] . . . 30
4.8 Avaliação do modelo . . . 30
4.9 Pipeline detalhado de inferência . . . 30

5 Protocolos forenses, calibração e decisão 31
5.1 Caso pericial . . . 31
5.2 Tipos de assinatura: RC, RO e QUESTIONED . . . 31
5.2.1 Classificação pericial humana das QUESTIONED . . . 32
5.3 Calibração estatística . . . 32
5.4 Dupla calibração adaptativa . . . 33
5.5 Pipeline detalhado FORENSIC_CASE — RC . . . 33
5.6 Pipeline detalhado FORENSIC_CASE — RO . . . 33
5.7 Pipeline detalhado SCIENTIFIC_EXTERNAL [16] . . . 34
5.7.1 Cenário científico RC . . . 35
5.7.2 Cenário científico RO . . . 35
5.7.3 Papel das QUESTIONED no protocolo científico . . . 36
5.8 Proveniência científica [16] . . . 36
5.9 FAR, FRR e EER . . . 37
5.9.1 FAR — False Acceptance Rate [5] . . . 37
5.9.2 FRR — False Rejection Rate [5] . . . 37
5.9.3 EER — Equal Error Rate [5] . . . 37
5.10 Thresholds, FAR-alvo e dataset shift [15], [16] . . . 38
5.10.1 Similaridade por cosseno: maior valor é mais genuíno . . . 38

5.10.2 Distância Euclidiana: menor valor é mais genuíno . . . 39
5.11 Política operacional de três zonas e intervalo entre limiares . . . 39
5.11.1 Domínio estatístico alinhado: A1, A(N-1) e A(N) . . . 40
5.11.2 Thresholds e política operacional de três zonas . . . 40
5.11.3 Consolidação conservadora dos folds . . . 41
5.11.4 Alinhamento entre calibração e inferência . . . 41
5.11.5 Semântica dos protocolos RC, RO e científico . . . 41
5.11.6 Persistência dos folds e decisão auditável . . . 42
5.12 Pipeline detalhado de comparação . . . 42
5.13 Decisão técnica versus classificação pericial [16] . . . 42

6 Rastreabilidade, auditoria e segurança 43
6.1 Rastreabilidade . . . 43
6.2 Hierarquia dos dados . . . 43
6.3 Modelo como artefato científico . . . 43
6.4 Auditoria e integridade . . . 44
6.5 Regras de exclusão . . . 44
6.5.1 Política de acesso a dados . . . 44
6.6 Segurança . . . 45
6.6.1 Autenticação . . . 45
6.6.2 ADMIN . . . 45
6.6.3 EXAMINER . . . 45
6.6.4 Bootstrap de segurança e integridade . . . 46
6.7 Primeiro administrador . . . 47
6.8 Segurança do storage . . . 47

7 Laudos e fluxos operacionais 49
7.1 Pipeline detalhado de geração de laudo . . . 49
7.2 Fluxo operacional completo . . . 49
7.2.1 Pipeline de inicialização da aplicação . . . 52
7.2.2 Pipeline de governança de modelo . . . 53
7.2.3 Pipeline de governança de dataset . . . 53
7.2.4 Licenciamento, perfis e cotas . . . 53
7.2.5 Segurança de inicialização e continuidade . . . 55

8 Requisitos, instalação e utilização 56
8.1 Dependências de software, bibliotecas, serviços e requisitos . . . 56
8.1.1 Plataforma Python e runtime . . . 56
8.1.2 Bibliotecas principais . . . 56
8.1.3 Internet e pesos pretrained . . . 57

8.1.4 Requisitos do sistema operacional . . . 57
8.1.5 Hardware . . . 57
8.1.6 Instalação Windows CPU . . . 57
8.1.7 Instalação Linux 2.4.0 CPU . . . 58
8.1.8 Proteção do código distribuído — PyArmor Pro . . . 58
8.2 Uso: adicionar dataset . . . 59
8.3 Uso: modelos . . . 59
8.4 Uso: treinamento . . . 60
8.5 Uso: calibração . . . 61
8.6 Uso: comparação . . . 63
8.6.1 Comparação Individual (LORO/A4) . . . 63
8.6.2 Inspecionar Score Pairwise . . . 64
8.6.3 Comparar em Lote . . . 64
8.6.4 Atualizar Contexto . . . 64
8.6.5 Regra de decisão e interpretação . . . 64
8.7 Uso: laudo . . . 66

9 Governança, distribuição e homologação 69
9.1 Política de datasets e responsabilidade de aquisição . . . 69
9.1.1 Datasets não fazem parte do produto comercial . . . 69
9.1.2 Responsabilidade do usuário ou instituição . . . 69
9.1.3 Suporte não significa redistribuição . . . 69
9.1.4 Situação das licenças . . . 70
9.1.5 Dataset para treinamento . . . 70
9.1.6 Dataset para SCIENTIFIC_EXTERNAL . . . 70
9.1.7 Bases próprias da instituição . . . 70
9.1.8 Privacidade, cadeia de custódia e direitos . . . 71
9.2 Política de modelos . . . 71
9.3 Distribuição . . . 71
9.3.1 O que o comprador recebe . . . 72
9.3.2 O que o comprador não recebe . . . 72
9.3.3 Fluxo após aquisição . . . 72
9.4 Testes e homologação . . . 72
9.4.1 Linux Mint, DEB e contêiner Podman . . . 72
9.4.2 Windows 11 x64 CPU . . . 73
9.4.3 Red Hat (RPM): instalação e operação homologadas . . . 73
9.4.4 Política de fechamento . . . 73
9.5 Limitações . . . 73

10 Boas práticas e arquitetura técnica detalhada **74**
10.1 Boas práticas . . . 74
10.2 Extensibilidade . . . 74
10.3 Estrutura recomendada do repositório privado . . . 74
10.3.1 Mapa da implementação: módulos, arquivos e classes . . . 75
10.3.2 Modelo de dados e schema SQLite . . . 77
10.3.3 Propriedade intelectual e identificação do software . . . 80
10.3.4 Declaração de uso de Inteligência Artificial Generativa . . . 81

11 Glossário, listas de verificação e encerramento **82**
11.1 Glossário . . . 82
11.1.1 Glossário de componentes de treinamento . . . 83
11.2 Checklist de operação . . . 83
11.3 Checklist de release . . . 84
11.3.1 Estado de homologação do Manual e do Release 2.4.0 . . . 84
11.4 Conclusão . . . 84

12 Referências bibliográficas e citação **86**
12.1 Referências científicas e técnicas . . . 86
12.2 Relação entre os conceitos do manual e as referências . . . 87
12.2.1 Citações recomendadas ao longo do manual . . . 88
12.3 Como citar o SFAAO . . . 88
12.3.1 Referência ao software . . . 88
12.3.2 Referência científica à metodologia . . . 88

Atualização documental do release 2.4.0 — 28/10/2026

Situação do software, da documentação e do artigo científico

Este manual mantém os fundamentos arquiteturais e científicos da edição anterior e atualiza os procedimentos operacionais conforme os artefatos do release 2.4.0 verificados em outubro de 2026. O artigo *Independent External-Impostor Calibration for Mitigating Dataset Shift in Forensic Offline Signature Verification* foi **aceito para publicação no IEEE Access**, segundo confirmação dos autores. A referência bibliográfica definitiva (DOI, volume, páginas e data de publicação) será incorporada somente após sua disponibilização editorial. **Aceite não significa publicação já concluída.**

As atualizações desta revisão incluem os instaladores Linux DEB/RPM e Windows CPU, o manifesto SHA-256 dos três artefatos, a correção da geração DOCX, o inicializador Windows, o armazenamento persistente e a interface de comparação com quatro opções. As 23 capturas que correspondem às telas operacionais documentadas foram incorporadas ao manual. A homologação funcional da versão 2.4.0, incluindo a geração de laudos DOCX, foi confirmada pelo responsável técnico nos ambientes Windows 11, Linux Mint (DEB/Podman) e Red Hat (RPM). As verificações criptográficas dos três instaladores complementam, mas não substituem, esses ensaios funcionais.

Separação de responsabilidades

Os resultados do software fornecem evidência técnica computacional e não substituem a análise grafoscópica. A classificação das assinaturas QUESTIONED e a conclusão pericial final permanecem sob responsabilidade do profissional habilitado.

Controle documental

Campo	Valor
Documento	Manual Técnico, Científico e Operacional do SFAAO 2.4.0
Versão do manual	2.4
Versão-alvo do software	SFAAO 2.4.0
Natureza	Documento técnico-científico e operacional
Estado	Software homologado nas plataformas declaradas; figuras incorporadas nesta edição
Fonte arquitetural normativa	`ARCHITECTURE_v2` — Especificação Oficial da Plataforma
Fonte científico-metodológica	Manuscrito *Independent External-Impostor Calibration for Mitigating Dataset Shift in Forensic Offline Signature Verification*
Política de distribuição	Software sem datasets de terceiros e sem modelos treinados
Público-alvo	Administradores, peritos, pesquisadores, equipes técnicas, instituições policiais e judiciárias

Hierarquia das fontes deste manual

Este manual consolida três níveis de informação:

1. **Especificação arquitetural normativa** — define como a plataforma deve ser organizada e evoluir;
2. **Comportamento funcional homologado** — descreve fluxos efetivamente testados no release 2.4.0;
3. **Fundamentação científica** — apresenta os conceitos e métodos sustentados pela literatura e pelo manuscrito científico do SFAAO.

Quando uma afirmação representar uma regra de arquitetura, ela deve ser entendida como requisito normativo da plataforma. Quando representar um comportamento homologado, refere-se ao release 2.4.0 testado. Quando representar fundamentação científica, as referências correspondentes são indicadas entre colchetes.

> **Propriedade intelectual.** Enquanto houver procedimento de proteção intelectual, pedido de patente, análise por NIT/AGINOV ou orientação institucional de confidencialidade, este manual e o repositório devem permanecer privados. Uma versão pública poderá ser derivada posteriormente.

Capítulo 1

Fundamentos, objetivos e escopo

1.1 Visão geral

O **SFAAO — Software Forense para Análise de Assinaturas Offline** é uma plataforma computacional modular destinada ao apoio da perícia grafoscópica. Para compreender sua proposta desde o início, os principais termos usados nessa definição são apresentados abaixo.

Tecnologias centrais do SFAAO

Deep Learning é uma forma de Inteligência Artificial baseada em redes neurais com várias camadas; serve para aprender características diretamente das imagens e é utilizado porque assinaturas offline possuem variação visual complexa. Uma **rede neural siamesa** utiliza ramos com pesos compartilhados para produzir representações comparáveis; serve para aprender relações de proximidade entre assinaturas. Um **embedding** é o vetor numérico produzido para representar uma assinatura; serve para calcular similaridade ou distância. A **calibração estatística** estima taxas de erro e limiares a partir de populações de comparação; serve para adaptar a decisão ao modelo e ao protocolo. **Rastreabilidade** é a capacidade de reconstruir qual caso, usuário, modelo, calibração e configuração produziram determinado resultado; é utilizada porque resultados periciais precisam ser tecnicamente auditáveis.

O SFAAO não é apenas um classificador. Ele organiza todo o ciclo computacional necessário para cadastrar dados, treinar modelos, calibrar thresholds, executar comparações, preservar evidências computacionais e produzir laudos auditáveis.

O sistema integra:

- gerenciamento de usuários;
- gerenciamento de casos;
- cadastro de peritos;
- cadastro e validação de datasets;
- treinamento de modelos;
- importação, ativação e governança de modelos;
- extração de embeddings;
- comparação biométrica;
- PairBuilder;
- calibração forense;
- calibração científica externa;
- cálculo de FAR, FRR e EER;
- thresholds operacionais;
- rastreabilidade;
- auditoria;
- integridade referencial;
- laudos DOCX/PDF;
- anexos de imagens periciais.

1.1.1 Por que o SFAAO é importante

A análise de assinaturas manuscritas **offline** trabalha com a imagem já produzida da assinatura, como uma digitalização ou fotografia. Diferentemente de uma assinatura capturada em mesa digitalizadora,

não estão disponíveis diretamente informações dinâmicas como velocidade do traço, pressão da caneta, aceleração ou sequência temporal. Por isso, a verificação offline é um problema complexo de visão computacional e biometria.

O SFAAO foi desenvolvido para acrescentar ao exame grafoscópico uma camada computacional quantitativa, rastreável e reproduzível. A plataforma **não substitui o perito grafoscopista**. O profissional continua responsável pela análise pericial, pela classificação humana das assinaturas questionadas quando o protocolo assim exigir e pela conclusão do exame. O SFAAO fornece medidas, comparações, estatística, rastreabilidade e documentação técnica que auxiliam essa atividade.

1.1.1.1 Perícia humana e análise computacional são complementares

Critério	Perícia grafoscópica humana	SFAAO / Deep Learning
Natureza	Exame técnico realizado por profissional habilitado, considerando características gráficas, contexto documental e material disponível.	Processamento matemático das imagens por redes neurais, embeddings, métricas e calibração estatística.
Informação observada	Aspectos gráficos e documentais interpretados no contexto do exame.	Características visuais aprendidas a partir dos pixels da imagem.
Escalabilidade	Limitada pelo tempo necessário à análise especializada.	Permite automatizar grande número de comparações após configuração adequada.
Consistência operacional	Depende da aplicação sistemática do método pericial e da documentação do exame.	A mesma configuração matemática é aplicada de forma uniforme às amostras processadas.
Reprodutibilidade	Depende da documentação e das condições do exame.	O mesmo modelo, imagens, parâmetros, métrica e calibração permitem reconstruir o cálculo computacional.
Interpretação	Integra elementos técnicos e contexto pericial amplo.	Produz scores, distâncias, thresholds e taxas estatísticas que precisam ser interpretados no contexto do caso.
Decisão final	Compete ao profissional responsável pelo exame.	Não emite conclusão grafoscópica autônoma; fornece evidência computacional complementar.

1.1.1.2 Por que utilizar Deep Learning e CNNs

Deep Learning é uma área da Inteligência Artificial que utiliza redes neurais com várias camadas para aprender representações a partir de dados. Uma **CNN — Convolutional Neural Network** é uma rede neural especialmente apropriada para imagens porque aprende padrões locais e hierárquicos, como bordas, formas, texturas e combinações mais complexas dessas estruturas.

No SFAAO, CNNs e arquiteturas relacionadas são utilizadas como extratores de características. Em vez de depender apenas de medidas desenhadas manualmente, a rede aprende representações numéricas das assinaturas. Essas representações podem ser comparadas por similaridade ou distância.

Esse recurso é importante porque assinaturas apresentam variabilidade natural entre produções genuínas, diferenças de escala e aquisição, falsificações com diferentes níveis de habilidade e possível mudança de distribuição entre o dataset usado no treinamento e o caso real.

1.1.1.3 Onde o SFAAO agrega valor

A plataforma é especialmente útil quando é necessário combinar análise pericial com processamento computacional repetível, por exemplo em instituições policiais e judiciárias, laboratórios de documentos-

copia, auditorias, prevenção a fraudes, instituições financeiras, gestão documental e pesquisa científica em biometria.

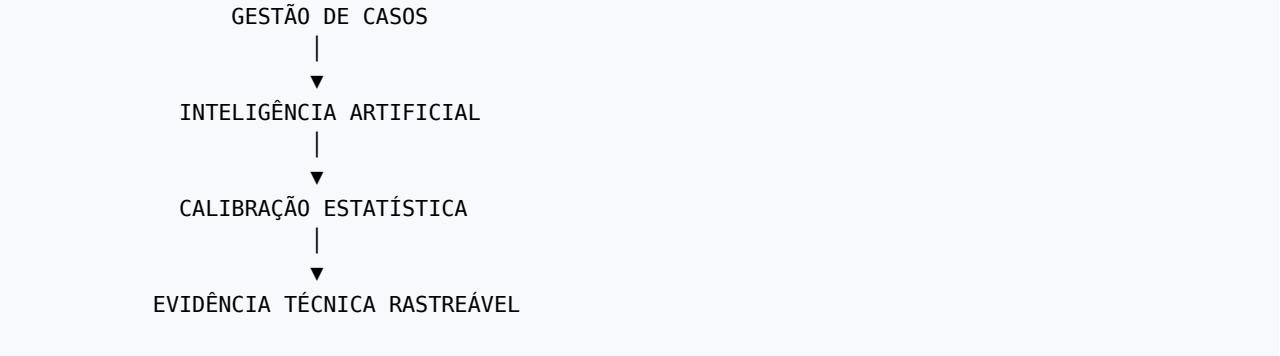
O diferencial do SFAAO não está apenas em usar uma rede neural. Ele integra, em um mesmo fluxo, gestão de casos, **RC** (assinaturas colhidas diretamente do subscritor reproduzindo o padrão que ele usa em contratos), **RO** (assinaturas também colhidas diretamente do subscritor, mas reproduzindo o padrão que ele usa em documentos oficiais) e **QUESTIONED** (todas as assinaturas existentes no contrato sob exame), além de treinamento configurável, calibração forense, calibração científica externa, comparação individual e em lote, rastreabilidade, integridade, licenciamento e geração de laudos DOCX/PDF.

Princípio de complementaridade

O SFAAO deve ser entendido como ferramenta de apoio técnico-científico. O perito interpreta o material e assume a responsabilidade profissional; o software executa o processamento computacional de forma controlada, registrada e reproduzível.

1.2 Filosofia da plataforma

O SFAAO deve ser entendido como um ecossistema pericial computacional:



A IA produz representações e scores. A calibração interpreta estatisticamente esses scores. O caso organiza os elementos periciais. A rastreabilidade permite reconstruir o caminho que produziu cada resultado.

1.3 Problema técnico

Soluções biométricas tradicionais podem apresentar:

- arquitetura fixa;
- backbone fixo;
- dataset acoplado ao código;
- threshold global;
- threshold obtido somente no treinamento;
- ausência de adaptação ao caso;
- ausência de calibração externa independente;
- baixa rastreabilidade;
- baixa reprodutibilidade;
- ausência de documentação pericial automatizada.

O SFAAO aborda essas limitações por meio de arquitetura modular, dupla calibração, gerenciamento de casos e persistência integral dos parâmetros técnicos.

1.4 Objetivos

- apoiar a atividade do perito grafoscopista;
- permitir treinamento de modelos especializados;
- permitir múltiplos backbones;
- permitir múltiplas funções de perda;
- permitir múltiplas métricas;
- desacoplar datasets por perfis;
- permitir calibração caso-específica;
- permitir calibração científica independente;
- produzir FAR, FRR, EER e thresholds;
- gerar resultados auditáveis;
- preservar histórico;
- produzir documentação técnica;
- viabilizar pesquisa científica reprodutível;
- permitir evolução arquitetural controlada.

1.5 Escopo e limites

O SFAAO contempla autenticação, usuários, casos, assinaturas, datasets, modelos, treinamento, avaliação, calibração, inferência, comparação, relatórios, auditoria e armazenamento.

O sistema não substitui o perito. O resultado computacional é apoio quantitativo e deve ser interpretado no contexto integral do exame.

1.6 Princípios arquiteturais

1.6.1 Responsabilidade única

Cada classe e módulo deve possuir responsabilidade claramente definida.

1.6.2 Separação de responsabilidades

Treinamento, inferência, calibração, interface, persistência, segurança e relatórios são componentes independentes.

1.6.3 Baixo acoplamento

Módulos se comunicam por interfaces públicas e estruturas bem definidas.

1.6.4 Alta coesão

Cada módulo agrupa apenas funcionalidades do mesmo domínio.

1.6.5 Arquitetura modular

Novos recursos devem preferencialmente ser incorporados por componentes com responsabilidades claras. Uma **Factory** é um construtor que cria o objeto correto a partir da configuração; um **Repository** concentra o acesso e a persistência de dados; um **Pipeline** é uma sequência organizada de etapas de processamento; e um **Profile** descreve regras estruturais de um recurso, como o formato de um dataset. Essa separação reduz acoplamento e facilita evolução, teste e auditoria.

1.6.6 Reprodutibilidade

Devem ser preservados configurações, seeds, métricas, thresholds, calibrações, artefatos e versão da plataforma.

1.6.7 Rastreabilidade

Cada decisão deve poder ser vinculada a usuário, caso, modelo, calibração, execução, parâmetros e versão.

1.6.8 Evolução controlada

Mudanças estruturais só devem ocorrer quando necessárias.

1.7 Conceitos fundamentais

Esta seção introduz os termos essenciais assumindo que o leitor não possui formação prévia em Inteligência Artificial, biometria, estatística ou desenvolvimento de software. Quando um conceito reaparecer nas seções seguintes, será aprofundado apenas no aspecto necessário ao funcionamento do SFAAO, evitando repetir integralmente a mesma definição.

Inteligência Artificial — IA

O que é. Área da Computação que desenvolve métodos capazes de executar tarefas que exigem tratamento inteligente de informação. **Para que serve no SFAAO.** Permitir que modelos aprendam representações visuais das assinaturas e produzam medidas quantitativas para comparação. **Por que é utilizada.** Porque a assinatura offline é uma imagem complexa e variável, difícil de representar por uma única regra manual.

Deep Learning

O que é. Subárea da IA baseada em redes neurais com várias camadas. **Para que serve.** Aprender automaticamente características relevantes a partir de imagens. **Por que é utilizado no SFAAO.** Permite transformar pixels de uma assinatura em representações numéricas adequadas à comparação biométrica.

CNN — Convolutional Neural Network

O que é. Tipo de rede neural projetado para processar imagens por operações convolucionais. **Para que serve.** Identificar padrões visuais em diferentes níveis de complexidade. **Por que é utilizada.** Backbones como ResNet, EfficientNet, ConvNeXt e MobileNet pertencem à família de arquiteturas visuais usadas pelo SFAAO para extração de características.

Assinatura offline

O que é. Imagem de uma assinatura obtida depois que o ato de escrever terminou, como uma digitalização ou fotografia. **Limitação importante.** A imagem não contém diretamente velocidade, pressão, aceleração ou sequência temporal da escrita. **No SFAAO.** É o tipo de dado biométrico analisado pela plataforma.

Biometria comportamental

O que é. Identificação ou verificação baseada em características resultantes do comportamento humano. **No caso da assinatura.** O traçado é influenciado por hábitos motores e gráficos do escritor, embora a versão offline preserve apenas o resultado visual desse comportamento.

Modelo de Inteligência Artificial

O que é. Estrutura matemática com parâmetros aprendidos a partir de dados. **Para que serve.** Receber uma imagem e produzir uma representação ou uma saída útil à tarefa. **No SFAAO.** O modelo é um artefato versionado, associado a configuração, pesos, métricas e proveniência.

Treinamento e inferência

Treinamento é a etapa em que o modelo ajusta seus parâmetros usando exemplos. **Inferência** é a etapa em que um modelo já treinado processa novas imagens sem alterar seus pesos. **No SFAAO.** Treinamento cria ou ajusta modelos; inferência produz embeddings e scores utilizados em calibração e comparação.

Dataset, amostra, classe e label

Dataset é um conjunto organizado de dados. Uma **amostra** é um item desse conjunto, por exemplo uma imagem de assinatura. **Classe** representa uma categoria ou identidade. **Label** é o rótulo associado à amostra ou ao par, indicando a informação supervisionada usada no treinamento. **No SFAAO.** Datasets são acessados por profiles e não são redistribuídos com a plataforma.

Writer e subscritor

Em datasets de pesquisa, **writer** é o identificador do escritor associado às imagens. Em um caso pericial, **subscritor** é a pessoa a quem as assinaturas do documento são atribuídas e de quem são colhidos os padrões de referência RC e RO. Os termos pertencem a contextos diferentes e não devem ser confundidos.

Parâmetro e hiperparâmetro

Parâmetro é um valor aprendido pelo modelo durante o treinamento, como os pesos da rede neural. **Hiperparâmetro** é uma configuração escolhida antes ou durante a execução do treinamento, como learning rate, batch size, número de épocas, dropout ou scheduler. **Para que servem.** Os hiperparâmetros controlam como o treinamento ocorre e influenciam custo, estabilidade, generalização e reprodutibilidade. **No SFAAO.** Devem ser persistidos junto ao modelo e aos artefatos de treinamento.

Overfitting, underfitting e regularização

Overfitting ocorre quando o modelo se adapta excessivamente aos dados de treino e perde capacidade de generalizar. **Underfitting** ou subajuste ocorre quando o modelo não aprende suficientemente o padrão. **Regularização** reúne técnicas usadas para reduzir overfitting, como dropout e weight decay. Esses conceitos ajudam a interpretar escolhas de hiperparâmetros e resultados de validação.

Checkpoint

O que é. Arquivo que guarda pesos de um modelo em determinado estado. **Para que serve.** Permitir carregar pesos pretrained, retomar ou preservar estados de treinamento e empacotar o melhor modelo. No SFAAO, checkpoints e pesos não devem ser confundidos com a configuração que descreve como o modelo foi construído.

Metric learning — aprendizado métrico

O que é. Estratégia em que o modelo aprende um espaço de representação no qual a relação entre amostras pode ser medida por distância ou similaridade. **Para que serve.** Tornar comparáveis assinaturas de pessoas que não precisam ter sido todas memorizadas como classes no treinamento. PAIR e TRIPLET são modos relacionados a essa abordagem.

Backbone

O que é. Rede que funciona como extrator principal de características. **Para que serve.** Converter a imagem em uma representação intermediária rica. **Por que é utilizado.** Permite trocar a arquitetura visual sem reescrever todo o pipeline. O SFAAO contempla ResNet18, ResNet34, ResNet50, EfficientNet-B0, ConvNeXt-Tiny e MobileNetV3.

Embedding

O que é. Vetor numérico que representa a assinatura em um espaço matemático. **Para que serve.** Permitir que duas assinaturas sejam comparadas por uma métrica. **Por que é utilizado.** Reduz uma imagem complexa a uma representação adequada para calcular proximidade ou similaridade.

Rede siamesa

O que é. Arquitetura com ramos de pesos compartilhados que processam entradas diferentes usando o mesmo extrator [2]. **Para que serve.** Aprender um espaço no qual exemplos compatíveis tendem a ficar mais próximos e exemplos incompatíveis mais separados. **No SFAAO.** É uma das bases de metric learning utilizadas nos modos de comparação.

Loss — função de perda

O que é. Função matemática que mede o erro que o treinamento tenta reduzir. **Para que serve.** Orientar o optimizer sobre como ajustar os parâmetros da rede. **No SFAAO.** Contrastive, Cosine Contrastive, Triplet e ArcFace são opções compatíveis com modos específicos de treinamento.

Optimizer e scheduler

Optimizer é o algoritmo que usa os gradientes calculados pela loss para atualizar os parâmetros treináveis. **Scheduler** altera o learning rate ao longo do treinamento segundo uma regra. **No SFAAO.** Adam e AdamW são optimizers; none, StepLR, cosine e plateau são opções de scheduler.

Score, similaridade e distância

Score é o valor numérico obtido ao comparar duas representações. Uma **similaridade** cresce quando as representações se tornam mais parecidas; uma **distância** tende a diminuir quando se tornam mais próximas. **No SFAAO.** Cosine Similarity usa a regra “maior é mais compatível”; Euclidean Distance usa “menor é mais compatível”.

Threshold — limiar

O que é. Valor usado como fronteira de decisão. **Para que serve.** Converter um score contínuo em regiões de decisão. **Por que é importante.** O mesmo score pode ter interpretação diferente conforme modelo, métrica e população de calibração; por isso o threshold deve ser estimado e rastreado.

Match, non-match e impostor

Match é uma comparação entre amostras tratadas pelo protocolo como pertencentes à mesma identidade. **Non-match** é uma comparação entre identidades distintas ou população impostora. **Impostor** é uma amostra usada para representar não correspondência. Um **impostor externo** vem de uma identidade externa ao subscritor e, no protocolo científico, de dataset externo legitimamente obtido.

Calibração

O que é. Processo estatístico que usa distribuições match e non-match para estimar taxas de erro e thresholds. **Para que serve.** Adaptar a decisão ao modelo e ao contexto definidos. **No SFAAO.** Existem protocolos FORENSIC_CASE e SCIENTIFIC_EXTERNAL.

FAR, FRR e EER

FAR — False Acceptance Rate é a proporção de non-matches aceitos incorretamente. **FRR — False Rejection Rate** é a proporção de matches rejeitados incorretamente. **EER — Equal Error Rate** é a taxa associada ao ponto em que FAR e FRR se aproximam. **Para que servem.** Quantificar

o comportamento do sistema sob um protocolo e um threshold definidos.

RC, RO e QUESTIONED

Esses três conjuntos foram definidos na primeira apresentação da plataforma. A Seção 31 formaliza sua origem, uso pericial, classificação humana das QUESTIONED e participação nos protocolos de calibração.

Rastreabilidade e reprodutibilidade

Rastreabilidade é a capacidade de reconstruir qual usuário, caso, modelo, calibração, configuração e execução produziram um resultado. **Reprodutibilidade** é a capacidade de repetir o processamento sob condições equivalentes. O SFAAO preserva configurações, seeds, métricas, thresholds, UIDs, hashes e artefatos para apoiar essas propriedades.

Hash SHA-256, manifesto de integridade e ACL

Hash SHA-256 é um resumo criptográfico de um arquivo; uma alteração no conteúdo muda o hash. Um **manifesto de integridade** é uma lista de arquivos e hashes esperados. **ACL — Access Control List** é a lista de permissões que determina quais contas podem acessar determinado arquivo ou diretório. **No SFAAO.** Esses mecanismos ajudam a detectar modificação de código e restringir acesso ao storage persistente.

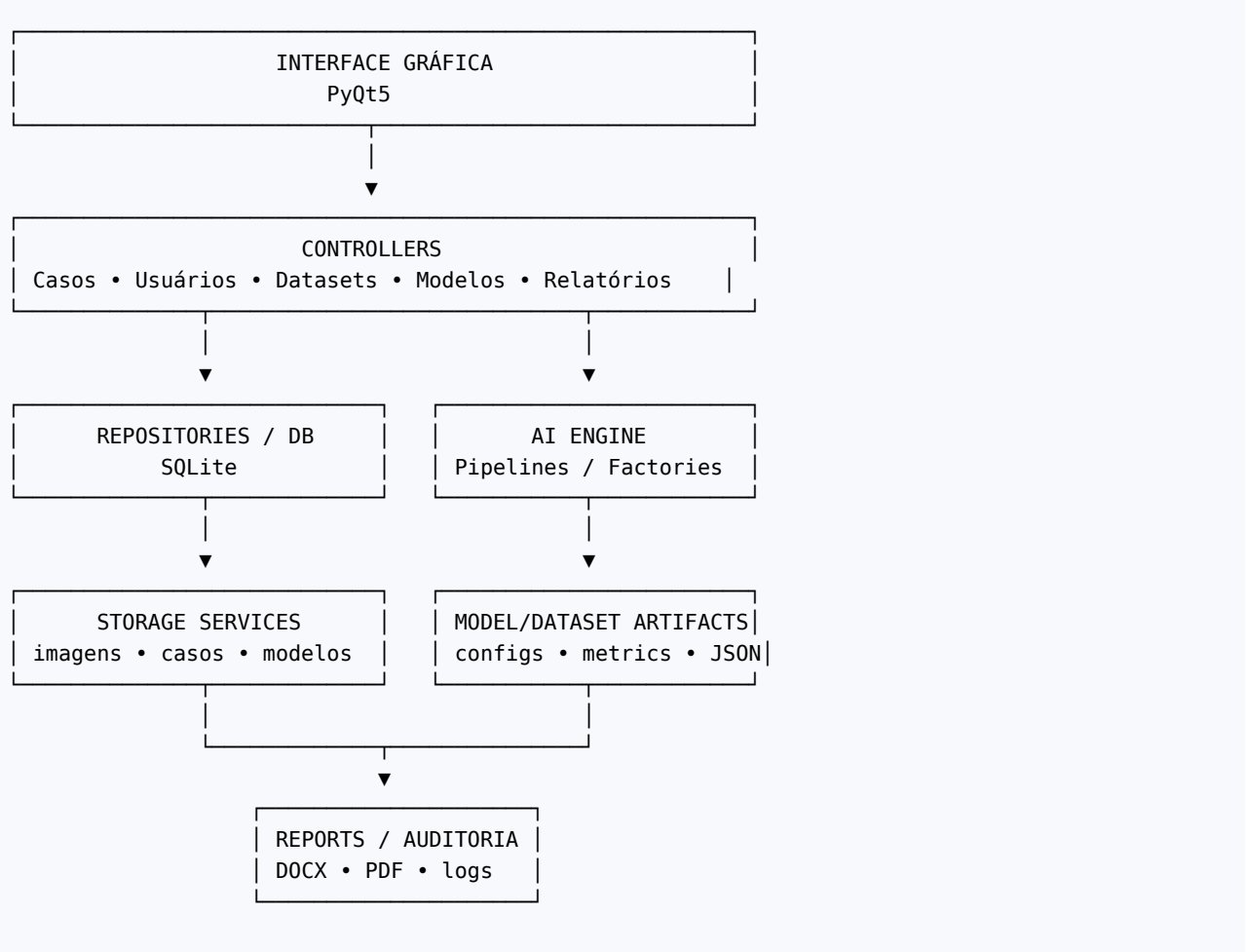
UID, fingerprint e schema

UID — Unique Identifier é um identificador criado para distinguir de forma estável uma entidade ou execução, como caso, calibração ou relatório. **Fingerprint** é uma impressão digital computacional derivada de dados ou contexto para identificar uma combinação específica. **Schema** é a estrutura que define tabelas, campos e relacionamentos do banco de dados. Esses elementos servem para rastreabilidade, consistência e versionamento.

Capítulo 2

Arquitetura e organização da plataforma

2.1 Arquitetura geral



Como ler a arquitetura

A **GUI** é a interface gráfica usada pela pessoa. Um **Controller** coordena a ação solicitada pela GUI e chama serviços apropriados. O **SQLite** é um banco de dados relacional embutido, armazenado localmente em arquivos e usado para metadados e histórico. O **filesystem** é a estrutura de arquivos e diretórios do sistema operacional, utilizada para imagens, modelos e artefatos volumosos. **Reports** são os componentes que transformam resultados persistidos em documentos como DOCX e PDF.

Arquitetura consolidada do release 2.4.0

A versão 2.4.0 alinha explicitamente treinamento, calibração, inferência, persistência, relatórios, licenciamento e integridade. O domínio decisório LORO é `A(N-1) / LORO_MEAN`; A1 permanece como componente de proveniência e A(N) como estatística descritiva. O schema SQLite v6 registra folds, alinhamento e estabilidade, enquanto os laudos utilizam fingerprint canônico de proveniência schema v3.

```
GUI / Controllers
```

```
  -> Training / Evaluation
  -> Calibration (AlignedScoreAggregator / LORO)
  -> Comparison (folds A(N-1) + consolidacao conservadora)
  -> SQLite schema v6 / questioned_decision_folds
  -> Reports / provenance fingerprint schema v3
  -> SecurityBootstrap / Licensing / Integrity
```

Novas execuções devem ser `ALIGNED`; `MISMATCH` bloqueia a execução e `LEGACY_MISMATCH` identifica apenas registros históricos.

2.2 Organização do projeto

```
app.py
ai/
controllers/
database/
reports/
security/
storage_services/
ui/
assets/
scripts/
requirements.txt
```

- `ai/`: algoritmos científicos;
- `controllers/`: orquestração;
- `database/`: schema, conexão e repositories;
- `reports/`: consolidação e geração de laudos;
- `security/`: autenticação e proteção do storage;
- `storage_services/`: armazenamento físico;
- `ui/`: interface;
- `scripts/`: execução de pipelines auxiliares.

2.3 Interface gráfica

A GUI é responsável por interação, visualização e captura de parâmetros.

Ela **não deve conter lógica científica**. A GUI pode validar entrada de usuário, mostrar progresso e chamar controllers, mas cálculos de treinamento, calibração e decisão pertencem ao núcleo científico.

Capturas da interface SFAAO 2.4.0

As telas reproduzidas neste manual foram fornecidas para a documentacao da versao 2.4.0. Os exemplos apresentados servem para orientar a operacao do sistema; dados de demonstracao e graficos ilustrativos nao devem ser confundidos com resultados periciais ou cientificos independentes.

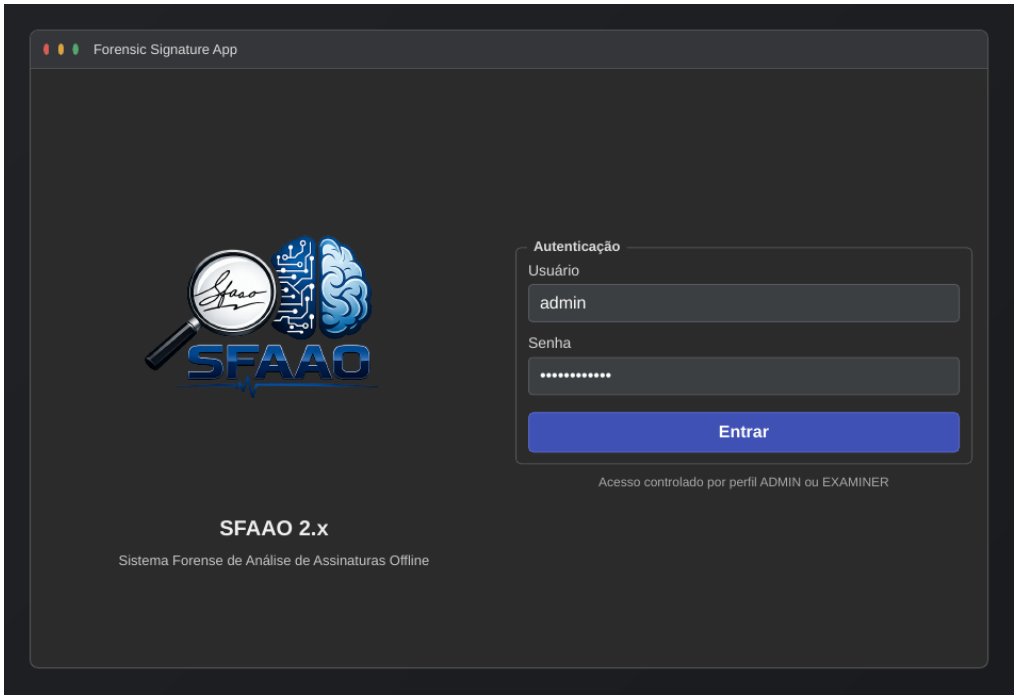


Figura 2.1: Tela de autenticação do SFAAO 2.4.0, com identificação do sistema e campos de acesso.

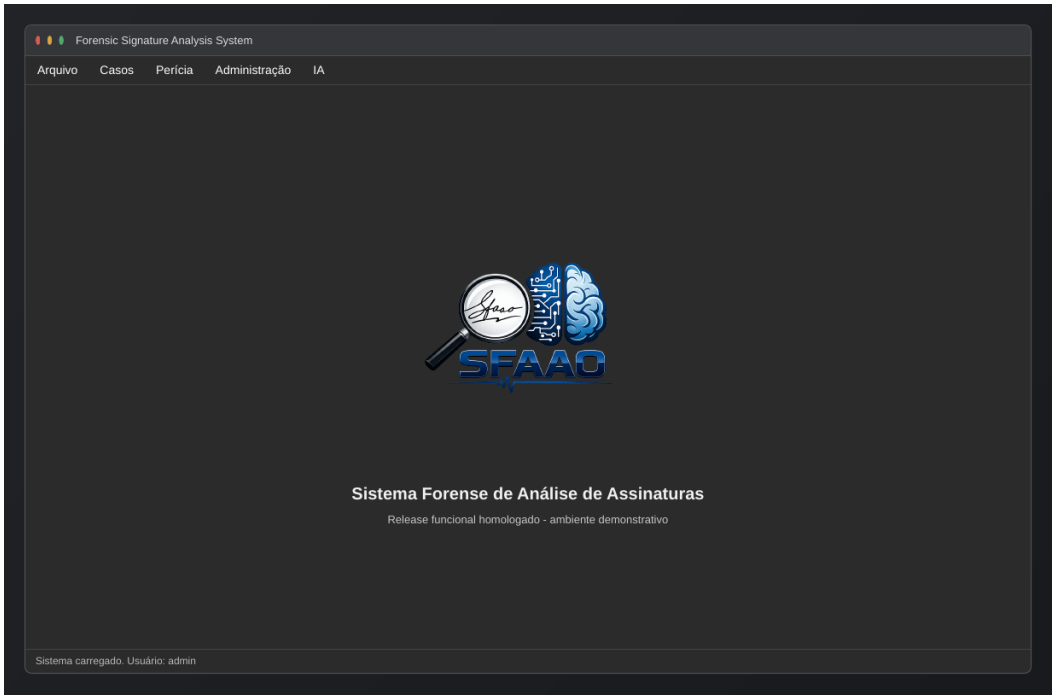


Figura 2.2: Janela principal do SFAAO no perfil administrador, com os menus superiores da plataforma.

2.4 Controllers

Fluxo típico:

```
GUI
 ↓
Controller
 ↓
```

```
Repository / Service / AI Pipeline
 ↓
Resultado
 ↓
GUI
```

Controllers coordenam, mas não duplicam matemática.

2.5 Persistência

O SQLite mantém metadados estruturados como:

- usuários;
- casos;
- assinaturas;
- modelos;
- calibrações;
- execuções de comparação;
- resultados;
- relatórios;
- auditorias.

Pesos neurais, imagens e artefatos volumosos permanecem no filesystem.

2.6 Storage

O storage contém:

- imagens RC;
- imagens RO;
- questionadas;
- modelos;
- calibrações;
- gráficos;
- resultados;
- relatórios.

O armazenamento é organizado por caso e protegido por permissões de filesystem.

Capítulo 3

Fundamentos do núcleo de inteligência artificial

3.1 Núcleo de IA

O diretório `ai/` contém a lógica científica e não deve conhecer:

- usuário autenticado;
- role;
- permissões;
- widgets;
- detalhes da GUI.

Recebe apenas parâmetros e caminhos previamente autorizados pela aplicação.

3.2 Framework modular de IA

O framework de IA do SFAAO é modular: o dataset é descrito por um perfil, o usuário escolhe um **modo de treinamento** e, a partir dessa escolha, o sistema constrói a unidade de amostragem adequada. **PAIR** aprende com pares de imagens rotulados como compatíveis ou incompatíveis; serve para aprendizado métrico com losses de pares. **TRIPLET** aprende com uma referência (anchor), uma amostra da mesma identidade (positive) e uma de identidade diferente (negative); serve para impor relações relativas de proximidade. **CLASSIFICATION** aprende a distinguir classes/identidades conhecidas no dataset; serve para treinamento supervisionado convencional ou angular compatível. O pipeline geral contempla explicitamente os três modos.

```
DATASET
  ↓
DATASET PROFILE
  ↓
MODO DE TREINAMENTO
  ├─ PAIR           → pares positivo/negativo
  ├─ TRIPLET        → anchor/positive/negative
  └─ CLASSIFICATION → imagem/classe
  ↓
PREPROCESSAMENTO
  ↓
BACKBONE
  ↓
EMBEDDING / REPRESENTAÇÃO
  ↓
LOSS COMPATÍVEL COM O MODO
  ↓
OPTIMIZER / SCHEDULER
  ↓
TRAINING
  ↓
VALIDATION
  ↓
EER / FAR / FRR / THRESHOLD
  ↓
MODEL PACKAGE
```

Regra de compatibilidade

A unidade de amostragem e a função de perda precisam ser compatíveis. No SFAAO, `PAIR` é destinado a perdas de pares, `TRIPLET` à Triplet Loss e `CLASSIFICATION` a perdas de classificação/angulares compatíveis. A GUI não deve combinar livremente modo e loss sem a validação do núcleo de IA.

Factories desacoplam configuração e implementação:

```
BackboneFactory
ModelFactory
LossFactory
DistanceFactory
OptimizerFactory
SchedulerFactory
```

3.3 Backbones

Definição formal. O backbone é o extrator de características da rede: recebe a imagem pré-processada e produz uma representação intermediária que será projetada no espaço de embeddings ou enviada a uma cabeça de classificação. Em aprendizagem profunda, sua escolha controla capacidade representacional, custo computacional, memória e velocidade de treinamento/inferência.

A família ResNet deriva da arquitetura residual de He et al. [4]. O SFAAO 2.4.0 contempla:

- **ResNet18** — residual compacta; boa opção de baseline, testes rápidos e cenários com menor capacidade computacional;
- **ResNet34** — residual intermediária, com maior profundidade que a ResNet18;
- **ResNet50** — residual mais profunda, baseada em blocos bottleneck e com maior custo computacional;
- **EfficientNet-B0** — arquitetura eficiente que equilibra profundidade, largura e resolução;
- **ConvNeXt-Tiny** — CNN moderna com desenho inspirado em avanços arquiteturais recentes;
- **MobileNetV3** — arquitetura leve, indicada quando eficiência e restrição de recursos são importantes.

Aplicação no SFAAO. O backbone é selecionado no `ModelConfig` e construído pela `BackboneFactory`. Sua saída alimenta a camada de embedding/cabeça correspondente ao modo de treinamento. A escolha deve ser registrada como parte da identidade do modelo e mantida consistente em treinamento, calibração e inferência.

Backbones homologados no release 2.4.0

Foram executados treinamentos reais de integração com **ResNet18, ResNet34, ResNet50, EfficientNet-B0, ConvNeXt-Tiny e MobileNetV3**. Esses ensaios homologam a integração do pipeline; não significam equivalência de desempenho biométrico entre arquiteturas.

Quando `pretrained=True`, a primeira execução pode exigir Internet para obtenção do checkpoint caso ele ainda não esteja no cache do PyTorch. Depois de armazenado localmente, o uso pode ocorrer sem novo download, respeitadas as dependências da instalação.

3.4 Embeddings

Um **embedding** é um vetor numérico que resume características da assinatura aprendidas pelo modelo. Ele serve como representação intermediária para que duas imagens possam ser comparadas matematicamente. O SFAAO utiliza embeddings porque comparar diretamente todos os pixels seria sensível a variações de aquisição e não aproveitaria a representação aprendida pelo backbone.

A **dimensão do embedding** indica quantos números compõem esse vetor e é um hiperparâmetro. Dimensões maiores aumentam a capacidade representacional e o custo; dimensões menores tornam a representação mais compacta. Exemplos configuráveis incluem:

```
64
128
256
512
1024
```

A dimensão é parte da identidade técnica do modelo e precisa permanecer registrada.

3.5 Funções de perda

A **função de perda — loss** mede numericamente o erro que o treinamento tenta reduzir. Ela não é o threshold de decisão e não representa, sozinha, o desempenho pericial; sua função é orientar o aprendizado dos pesos.

3.5.1 Contrastive Loss

O que é. Loss de pares que aproxima embeddings de pares positivos e impõe separação a pares negativos [3]. **Para que serve.** Aprender uma geometria em que amostras compatíveis fiquem próximas. **Por que usar.** É uma formulação clássica de metric learning e se integra naturalmente ao modo PAIR.

3.5.2 Cosine Contrastive Loss

O que é. Loss de pares construída sobre similaridade angular. **Para que serve.** Penalizar positivos com similaridade insuficiente e negativos excessivamente semelhantes. **Por que usar.** É apropriada quando a orientação relativa dos embeddings é o sinal principal de comparação por cosseno.

3.5.3 Triplet Loss

O que é. Loss que recebe anchor, positive e negative. **Para que serve.** Exigir que o positive permaneça mais próximo do anchor que o negative por uma margem configurada. **Por que usar.** Permite aprender relações relativas entre três exemplos e é compatível com o modo TRIPLET.

```
d(anchor, positive) + margin < d(anchor, negative)
```

3.5.4 ArcFace

O que é. Loss/cabeça angular que acrescenta margem à classe correta no espaço angular. **Para que serve.** Aumentar separação entre identidades e compactação dentro da mesma classe. **Por que usar.** Pode produzir representações discriminativas em treinamento CLASSIFICATION, desde que número de classes, cabeça e configuração sejam compatíveis.

Regra operacional

Loss e modo de treinamento precisam ser compatíveis. O SFAAO valida essa relação antes de executar o pipeline. Margin, scale e angular margin são hiperparâmetros explicados na Seção 23.1.

3.6 Métricas e distâncias

Uma **métrica de comparação** transforma dois embeddings em um valor numérico. No SFAAO, a regra de decisão depende da direção da métrica; portanto, "maior" e "menor" não podem ser tratados da mesma forma.

3.6.1 Similaridade por cosseno

O que é. Medida do alinhamento angular entre dois vetores.

$$\cos(a, b) = \frac{a^\top b}{\|a\|_2 \|b\|_2}.$$

Para que serve. Comparar a direção relativa dos embeddings, reduzindo a influência de sua magnitude. **Por que usar.** É comum em espaços de representação aprendidos e foi utilizada no estudo científico associado ao SFAAO [16]. **Regra no SFAAO.** Maior similaridade indica maior compatibilidade; a orientação é "maior é mais compatível"; a decisão operacional completa utiliza τ_{EER}, $\tau_{forensic}$ e $\tau_{operacional}$ conforme a política de três zonas.

3.6.2 Distância Euclidiana

O que é. Distância geométrica entre dois vetores.

$$d_E(a, b) = \sqrt{\sum_i (a_i - b_i)^2}.$$

Para que serve. Medir quanto os embeddings estão separados no espaço. **Por que usar.** É uma métrica direta de proximidade e pode ser adequada quando a geometria aprendida preserva diferenças de magnitude. **Regra no SFAAO.** Menor distância indica maior compatibilidade; a classificação final respeita a orientação da métrica e a política operacional de três zonas definida pelos limiares calibrados.

Não inverter mecanicamente a métrica

Cosine Similarity e Euclidean Distance são métricas diferentes. Euclidiana não é simplesmente "cosseno ao contrário". Thresholds, FAR, FRR e regras de comparação devem respeitar a orientação registrada no modelo e na calibração.

3.7 Preprocessamento

Preprocessamento é o conjunto de transformações aplicadas à imagem antes de enviá-la ao modelo. Ele deve ser equivalente entre treinamento e inferência; caso contrário, o modelo recebe dados em formato diferente daquele usado para aprender.

Operações típicas incluem:

- **grayscale**: converte a imagem para tons de cinza, reduzindo dependência de cor quando ela não é necessária;
- **binarização**: separa pixels em duas regiões, geralmente traço e fundo, quando o protocolo a utiliza;
- **preservação de proporção**: evita distorcer a relação entre largura e altura da assinatura;

- **padding**: acrescenta margens para ajustar a imagem ao tamanho esperado sem deformá-la;
- **resize**: redimensiona a imagem para a resolução de entrada do backbone;
- **normalização**: transforma a escala estatística dos pixels para o padrão esperado pelo modelo;
- **tensor**: estrutura numérica multidimensional utilizada pelo PyTorch para executar as operações da rede neural.

Uma configuração consolidada do projeto utiliza entrada `224×224`.

3.7.1 Conformidade com a Especificação Oficial da Arquitetura

A arquitetura do SFAAO 2.4.0 é regida pela especificação oficial `ARCHITECTURE_v2`. Os seguintes princípios são normativos:

```
GUI
 ↓
Gerenciamento de Casos / Controllers
 ↓
Pipelines de IA
 ├─ TrainingPipeline
 ├─ InferencePipeline
 └─ CalibrationPipeline
 ↓
Factories / Repositories / Configurações
 ↓
Redes / Datasets / Distâncias
 ↓
Persistência
```

As responsabilidades fundamentais são:

Camada	Responsabilidade
GUI	interação, autenticação, configuração e visualização
Controllers	coordenação das operações da aplicação
Banco de Dados	metadados, histórico, auditoria e relacionamentos
Storage	imagens, modelos, calibrações, gráficos, relatórios e artefatos
IA	algoritmos científicos sem conhecimento de usuários, permissões ou widgets
Reports	consolidação e materialização documental dos resultados

3.7.1.1 Regra de isolamento científico

O módulo `ai/` não deve tomar decisões de autenticação, autorização ou governança. Ele recebe parâmetros e caminhos já autorizados pela aplicação.

3.7.1.2 Regra caso-modelo-calibração

A calibração pertence simultaneamente ao **caso** e ao **modelo**. Ela nunca deve ser tratada como parâmetro global reutilizável indiscriminadamente.

3.7.1.3 Regra de relatório

Todo relatório deve preservar, quando aplicável:

```
caso
modelo
calibração
threshold operacional
versão da plataforma
data de emissão
execuções que originaram os resultados
```

Capítulo 4

Datasets, treinamento e inferência

4.1 Datasets e profiles

O SFAAO separa o **perfil** do dataset do **conteúdo físico**. A pesquisa associada ao sistema utiliza e discute bases de verificação de assinaturas como GPDS, CEDAR e BHSig260 [8]–[11], [14].

Perfis podem ser distribuídos. Os dados não.

O profile abstrai:

- estrutura de diretórios;
- nomes de arquivos;
- writer ID;
- genuína;
- falsificação;
- splits;
- metadados.

Isso permite integrar novas bases sem alterar o núcleo do sistema.

4.2 Pipeline detalhado de cadastro e validação de dataset

```
ADMIN
 ↓
Adicionar Dataset
 ↓
nome + profile + versão + diretório
 ↓
DatasetReference
 ↓
DatasetProvider
 ↓
validate_profile_classification()
 ↓
varredura das imagens
 ├─ imagem suportada?
 ├─ genuine?
 ├─ forgery?
 ├─ writer_id?
 ├─ conflito genuine/forgery?
 └─ não classificada?
 ↓
estatísticas
 ↓
validação final
 ↓
persistência no catálogo SQLite
 ↓
dataset disponível ao sistema
```

A varredura possui progresso real de 0–100%.

4.3 PairBuilder

O `PairBuilder` centraliza construção de populações para treino, validação e calibração.

Tipos possíveis:

```
genuíno × genuíno
genuíno × falsificação
genuíno × outro escritor
falsificação × falsificação
referência × impostor externo
```

Em Triplet:

```
anchor
positive
negative
```

Seeds devem ser preservadas sempre que houver amostragem aleatória.

4.3.1 Hiperparâmetros, componentes configuráveis e modos de treinamento

Esta seção é a referência operacional dos valores que o usuário pode configurar. Os conceitos básicos de backbone, embedding, loss, optimizer, scheduler, score e threshold já foram introduzidos na Seção 7 e aprofundados nas Seções 16–20; por isso não são redefinidos integralmente aqui. O foco passa a ser **o que cada configuração controla, para que serve, por que é utilizada e qual é seu efeito no SFAAO**.

4.3.1.1 Modos de treinamento

PAIR

O que é. Modo em que cada unidade de treinamento contém duas imagens e uma label indicando relação positiva ou negativa. **Para que serve.** Ensinar o modelo a aproximar pares compatíveis e separar pares incompatíveis. **Por que usar.** É adequado a metric learning com losses de pares, como Contrastive e Cosine Contrastive.

```
Dataset
 ↓
PairBuilder
 ↓
pares positivos/negativos
 ↓
imagem_1 + imagem_2
 ↓
backbone compartilhado
 ↓
embedding_1 + embedding_2
 ↓
loss de pares
 ↓
optimizer
```

TRIPLET

O que é. Modo em que cada unidade contém `anchor`, `positive` e `negative`. O positive pertence à mesma identidade do anchor; o negative pertence a identidade diferente. **Para que serve.** Fazer o positive ficar mais próximo do anchor que o negative por uma margem. **Por que usar.** Permite aprender diretamente relações relativas de proximidade no espaço de embeddings.

```
Dataset
 ↓
Triplet Builder
 ↓
anchor + positive + negative
 ↓
backbone compartilhado
 ↓
três embeddings
 ↓
Triplet Loss
 ↓
optimizer
```

CLASSIFICATION

O que é. Modo supervisionado em que cada imagem possui uma classe/identidade e o modelo aprende a separar essas classes. **Para que serve.** Aprender representações discriminativas por identidade. **Por que usar.** É adequado quando o dataset fornece classes confiáveis e pode ser combinado com cabeças de classificação ou perdas angulares, como ArcFace, quando compatíveis.

```
imagem + classe
 ↓
backbone
 ↓
representação
 ↓
cabeça classificadora / angular
 ↓
loss compatível
 ↓
optimizer
```

Compatibilidade entre modo e loss

PAIR é utilizado com losses de pares; TRIPLET com Triplet Loss; CLASSIFICATION com loss/cabeça de classificação compatível, incluindo ArcFace quando configurado corretamente. A plataforma valida combinações para evitar iniciar um treinamento matematicamente incompatível.

4.3.1.2 Hiperparâmetros de arquitetura e representação

backbone_name

O que é. Nome da arquitetura visual usada como extrator de características. **Para que serve.** Selecionar capacidade, custo computacional e tipo de representação. **Por que é configurável.** Diferentes cenários podem exigir modelos mais leves ou mais profundos. **No SFAAO.** ResNet18, ResNet34, ResNet50, EfficientNet-B0, ConvNeXt-Tiny e MobileNetV3 são opções homologadas de integração.

embedding_dim

O que é. Número de componentes do vetor de embedding. **Para que serve.** Define o tamanho da representação matemática produzida pelo modelo. **Efeito.** Dimensões maiores podem oferecer maior capacidade representacional, mas aumentam memória e custo; dimensões menores são mais compactas, mas podem limitar informação. **No SFAAO.** O valor é parte da identidade técnica do modelo e deve ser o mesmo em carregamento, inferência, calibração e comparação.

dropout

O que é. Técnica de regularização que desativa aleatoriamente parte das ativações durante o treinamento. **Para que serve.** Reduzir dependência excessiva de unidades específicas e ajudar a limitar overfitting. **Efeito.** Valor muito alto pode dificultar o aprendizado; valor muito baixo oferece pouca regularização. **No SFAAO.** É persistido no ModelConfig.

pretrained

O que é. Indica se o backbone começa com pesos aprendidos anteriormente em outro conjunto de dados. **Para que serve.** Transferir conhecimento visual genérico e acelerar a convergência. **Por que usar.** É útil em datasets pequenos ou moderados e em ambientes com recursos limitados. **Efeito.** `true` pode exigir download inicial do checkpoint quando ele ainda não estiver no cache local; depois, o processamento é local.

freeze_backbone

O que é. Indica se os pesos do backbone permanecem congelados durante o treinamento. **Para que serve.** Reduzir custo, memória e risco de overfitting quando se deseja treinar principalmente a cabeça/embedding. **Efeito.** `false` permite fine-tuning do extrator; `true` preserva seus pesos. **No SFAAO.** Deve ser interpretado em conjunto com pretrained e learning rate.

pretrained	freeze	Estratégia	Interpretação
true	true	Transfer learning congelado	Mais rápido e econômico; treina principalmente embedding/cabeça.
true	false	Fine-tuning	Adapta também o backbone ao domínio de assinaturas.
false	false	Treino do zero	Exige mais dados, épocas e capacidade computacional.
false	true	Pesos iniciais congelados	Em geral não recomendado se os pesos forem aleatórios.

4.3.1.3 Hiperparâmetros das funções de perda

margin

O que é. Margem de separação usada por losses contrastivas ou Triplet. **Para que serve.** Definir quanto exemplos incompatíveis devem ser separados. **Efeito.** Margem muito pequena pode permitir pouca separação; muito grande pode tornar a restrição difícil de satisfazer. **No SFAAO.** O significado exato depende da loss selecionada e deve ser persistido.

ArcFace: scale e angular margin

scale (s) multiplica os logits angulares para controlar sua escala numérica. **Angular margin (m)** aumenta a separação exigida entre classes no espaço angular. **Para que servem.** Tornar as identidades mais separáveis em treinamento CLASSIFICATION compatível. **Efeito.** Valores inadequados podem causar instabilidade ou separação insuficiente. Ambos fazem parte da configuração repro-

dutível do modelo.

4.3.1.4 Hiperparâmetros de otimização

optimizer_name

O que é. Escolha do algoritmo que atualiza os pesos. **Para que serve.** Controlar a dinâmica de convergência. **No SFAAO.** `adam` e `adamw` são suportados. AdamW aplica weight decay de forma desacoplada e pode ser preferível quando essa regularização é parte do experimento.

learning rate — lr

O que é. Tamanho básico das atualizações dos pesos. **Para que serve.** Equilibrar velocidade e estabilidade do treinamento. **Efeito.** Muito alto pode causar instabilidade ou impedir convergência; muito baixo pode tornar o treinamento excessivamente lento. **No SFAAO.** Deve ser analisado junto a optimizer, scheduler, batch size e freeze do backbone.

weight decay

O que é. Regularização aplicada aos pesos durante otimização. **Para que serve.** Ajudar a limitar crescimento excessivo dos parâmetros e overfitting. **Efeito.** Valor excessivo pode reduzir demais a capacidade do modelo; valor nulo desativa essa forma de regularização.

batch_size

O que é. Quantidade de unidades de treinamento processadas antes de uma atualização do optimizer. **Para que serve.** Controlar memória, throughput e variabilidade do gradiente. **No SFAAO.** Uma unidade é um par em PAIR, uma tripla em TRIPLET e uma imagem/classe em CLASSIFICATION. Batch maior consome mais memória.

epochs

O que é. Número máximo de passagens de treinamento planejadas sobre a amostragem definida. Uma **época** representa um ciclo de treinamento segundo o pipeline. **Para que serve.** Controlar por quanto tempo o modelo pode aprender. **Efeito.** Poucas épocas podem causar subajuste; muitas podem aumentar custo e overfitting. Early stopping pode encerrar antes do máximo.

4.3.1.5 Schedulers de learning rate

scheduler_name

O que é. Regra que modifica o learning rate ao longo das épocas. **Para que serve.** Permitir atualizações maiores no início e/ou menores quando o treinamento amadurece. **No SFAAO.** `none` mantém LR sem agenda; `StepLR` reduz em intervalos; `cosine` segue agenda cossenoidal; `plateau` reduz quando a métrica deixa de melhorar. Parâmetros próprios do scheduler também devem ser registrados.

4.3.1.6 Hiperparâmetros de amostragem

train_pairs e val_pairs

O que são. Quantidades planejadas de amostras de treinamento e validação no modo de pares, ou parâmetros equivalentes de tamanho nos demais modos. **Para que servem.** Controlar volume computacional e quantidade de evidência usada para aprender e avaliar. **Por que separar.** A validação não deve ser usada para cada atualização dos pesos; ela mede comportamento durante o desenvolvimento e auxilia na seleção do melhor estado.

positive_ratio

O que é. Proporção planejada de pares positivos. **Para que serve.** Controlar o equilíbrio entre exemplos da mesma identidade e exemplos de identidades diferentes. **Efeito.** Um conjunto muito desequilibrado pode enviesar o aprendizado. **No SFAAO.** A distribuição solicitada e a produzida devem ser registradas.

forgery_negative_ratio

O que é. Fração planejada dos negativos formada por falsificações do mesmo writer/perfil, quando o dataset permite. **Para que serve.** Controlar a proporção de negativos potencialmente mais difíceis em relação a negativos de outros escritores. **Por que é utilizado.** Permite estudar a capacidade do modelo de separar falsificações e identidades diferentes de forma controlada.

seed e determinismo

Seed é o valor inicial de geradores pseudoaleatórios. **Para que serve.** Tornar amostragens de pares, triplets, subconjuntos e impostores mais repetíveis. **Determinismo** indica o esforço para reduzir variações entre execuções. A mesma seed melhora reprodutibilidade, mas não garante resultados bit a bit idênticos entre hardware, kernels e versões de bibliotecas diferentes.

4.3.1.7 Early stopping

min_epochs

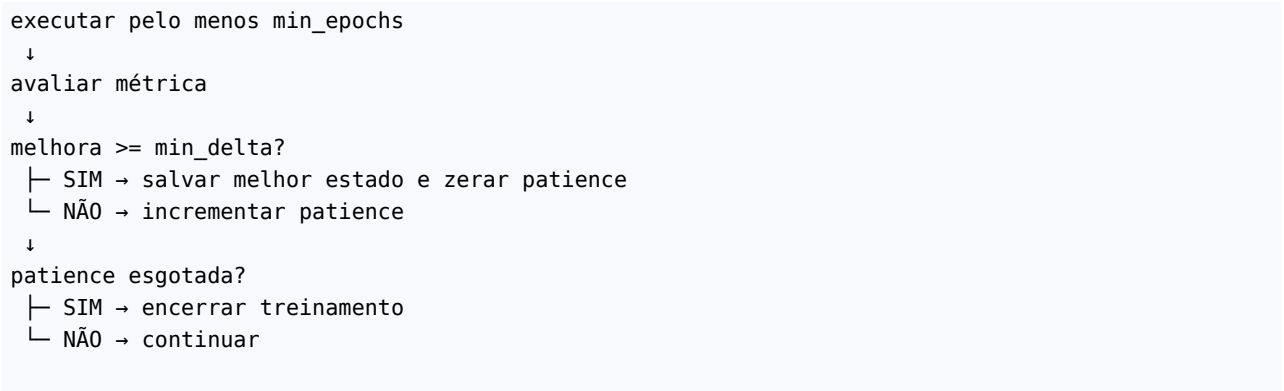
O que é. Número mínimo de épocas que devem ser executadas antes de permitir parada antecipada. **Para que serve.** Evitar encerrar o treinamento cedo demais por oscilações iniciais.

patience

O que é. Quantidade de avaliações consecutivas toleradas sem melhora suficiente. **Para que serve.** Dar ao treinamento oportunidade de recuperar melhora antes de parar. Valor maior torna o early stopping mais tolerante e pode prolongar o treino.

min_delta

O que é. Melhora mínima necessária para considerar que a métrica realmente melhorou. **Para que serve.** Evitar que pequenas oscilações numéricas reiniciem indefinidamente a paciência. **No SFAAO.** Early stopping pode monitorar EER ou outra métrica definida pelo pipeline.



4.3.1.8 Execução e hardware

device

O que é. Dispositivo no qual o PyTorch executará os tensores e o modelo. **Valores típicos.** CPU, GPU/CUDA ou seleção automática. **Para que serve.** Controlar onde o cálculo ocorre. **Na distribuição CPU 2.4.0.** A imagem Linux e o instalador Windows são voltados à execução em CPU. O ciclo de testes Windows 11 incluiu treinos funcionais; o dispositivo de execução empregado deve ser registrado em cada experimento.

num_workers

O que é. Quantidade de processos auxiliares usados pelo carregador de dados. **Para que serve.** Paralelizar leitura/preprocessamento quando o ambiente permite. **Efeito.** Mais workers podem melhorar throughput, mas aumentam uso de recursos e podem causar incompatibilidades em certos ambientes. Para o experimento de homologação Linux 2.4.0, o valor deve ser registrado junto à configuração efetivamente executada; nenhuma evidência Windows histórica é promovida automaticamente para esta versão.

4.3.1.9 Estado ótimo, métricas e artefatos

Best state dict

O que é. Cópia dos pesos do modelo no melhor estado encontrado pelo critério de validação. **Para que serve.** Evitar que a última época substitua um estado anterior superior. **No SFAAO.** O melhor estado é restaurado ao final e a época correspondente é registrada.

Durante e após o treinamento podem ser persistidos, entre outros:

```
epoch
train_loss
val_loss
EER
threshold
learning_rate
best_epoch
best_validation_loss
epochs_requested
epochs_executed
final_eer
final_threshold
history
```

A avaliação pode gerar curvas FAR×FRR e distribuição de scores. Esses artefatos não são meras ilustrações: documentam como as populações se comportaram e ajudam a auditar o threshold estimado.

4.3.1.10 Matriz resumida

Elemento	Opções / exemplos no SFAAO
Modo de treino	PAIR, TRIPLET, CLASSIFICATION
Backbone	ResNet18, ResNet34, ResNet50, EfficientNet-B0, ConvNeXt-Tiny, MobileNetV3
Loss	Contrastive, Cosine Contrastive, Triplet, ArcFace
Comparação	Cosine Similarity, Euclidean Distance
Embedding	dimensão configurável

Elemento	Opções / exemplos no SFAAO
Regularização	dropout, weight decay
Inicialização	pretrained true/false
Fine-tuning	freeze_backbone true/false
Optimizer	Adam, AdamW
Scheduler	none, StepLR, cosine, plateau
Amostragem	train/validation, positive ratio, forgery negative ratio
Reprodutibilidade	seed, determinismo, histórico e configurações persistidas
Early stopping	min_epochs, patience, min_delta
Execução	device, num_workers

4.4 Pipeline detalhado de treinamento

```
01. usuário seleciona dataset
02. seleciona backbone
03. seleciona loss
04. seleciona métrica/distância
05. define embedding_dim
06. define dropout
07. define pretrained
08. define freeze_backbone
09. define epochs
10. define batch_size
11. define learning rate
12. define weight_decay
13. define seed
14. define quantidade de amostras
15. define positive_ratio
16. define forgery_negative_ratio
17. define optimizer
18. define scheduler
19. define early stopping
20. ModelConfig é persistido
21. TrainingConfig é persistido
22. DatasetProvider carrega dataset
23. builder gera pairs/triplets
24. treino e validação são separados
25. DataLoaders são construídos
26. imagem é preprocessada
27. backbone extrai características
28. head produz embedding
29. loss é calculada
30. backward()
31. optimizer.step()
32. scheduler.step() quando aplicável
33. epoch de validação
34. geração de scores match/non-match
35. sweep de thresholds
36. cálculo FAR
37. cálculo FRR
38. cálculo EER
39. cálculo τ_EER
40. atualização do melhor estado
41. avaliação de early stopping
42. restauração do melhor estado
```

```
43. avaliação final
44. salvamento de pesos
45. salvamento de configurações
46. salvamento de métricas
47. geração FAR×FRR
48. geração de distribuição de scores
49. criação do pacote do modelo
50. cadastro automático no Model Manager
```

4.5 ModelConfig

O `ModelConfig` registra a identidade arquitetural do modelo. Os conceitos abaixo já foram definidos nas Seções 7, 16–19 e 23.1; aqui o objetivo é apenas mostrar quais valores precisam permanecer associados ao artefato para que ele seja carregado e reproduzido corretamente.

```
backbone_name
pretrained
freeze_backbone
embedding_dim
dropout
loss_name
distance_name
parâmetros específicos da loss, quando aplicáveis
```

A configuração deve acompanhar o modelo. Alterar silenciosamente qualquer desses valores pode tornar incompatíveis treinamento, inferência, calibração e comparação.

4.6 TrainingConfig

O `TrainingConfig` reúne os hiperparâmetros que controlam a execução do treinamento. As definições e efeitos estão consolidados na Seção 23.1.

```
epochs
batch_size
learning_rate
weight_decay
seed
train_pairs / tamanho de treino
val_pairs / tamanho de validação
positive_ratio
forgery_negative_ratio
min_epochs
patience
min_delta
device
num_workers
optimizer_name
scheduler_name
deterministic
```

O arquivo persistido deve registrar os valores efetivamente usados, e não apenas os defaults da interface.

4.7 Early stopping [6]

O mecanismo de early stopping, bem como `min_epochs`, `patience` e `min_delta`, foi definido na Seção 23.1. Nesta etapa do pipeline, o SFAAO monitora a métrica configurada, preserva o melhor estado e interrompe o treinamento quando o critério de parada é satisfeito.

O objetivo é evitar gasto computacional desnecessário e reduzir o risco de continuar treinando depois que a validação deixa de melhorar. A decisão de parada, o número de épocas executadas e o melhor epoch devem permanecer registrados nos artefatos.

4.8 Avaliação do modelo

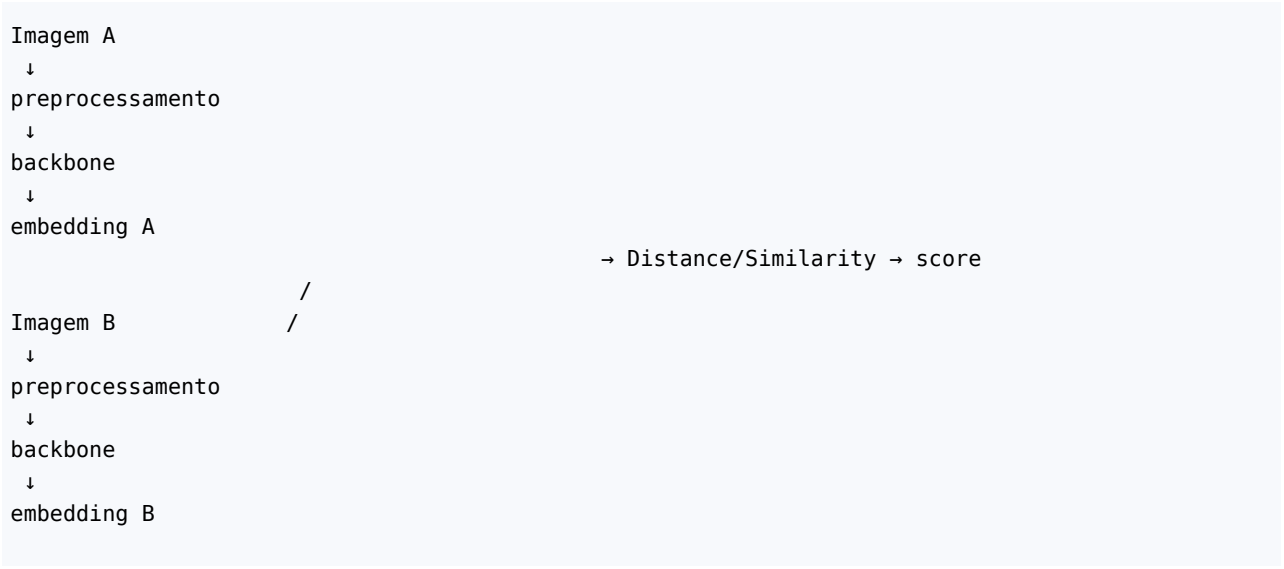
A avaliação transforma scores de validação em métricas que ajudam a caracterizar o modelo. FAR, FRR, EER, threshold e orientação da métrica já foram definidos nas Seções 7, 19 e 38–40.

No SFAAO, a avaliação pode produzir:

- EER e threshold correspondente;
- contagem de matches e non-matches;
- curva FAR×FRR;
- distribuição de scores;
- histórico por época;
- identificação do melhor estado;
- caminhos e hashes dos artefatos gerados.

Esses resultados descrevem o modelo sob a população avaliada. Eles não eliminam a calibração posterior vinculada ao caso ou ao protocolo científico.

4.9 Pipeline detalhado de inferência



O score não deve ser interpretado isoladamente; sua decisão depende da calibração correspondente.

Capítulo 5

Protocolos forenses, calibração e decisão

5.1 Caso pericial

O caso é a unidade central da plataforma.

Um caso contém:

- identificador;
- dados da demanda;
- perito;
- RC;
- RO;
- questionadas;
- calibrações;
- execuções;
- resultados;
- laudos.

5.2 Tipos de assinatura: RC, RO e QUESTIONED

O SFAAO separa três conjuntos porque cada um possui origem e função pericial diferentes. Esta distinção deve ser compreendida antes dos pipelines de calibração e comparação.

RC — referências colhidas no padrão contratual

O que são. Assinaturas colhidas diretamente do cliente/subscritor durante o procedimento pericial. **Como são produzidas.** O subscritor assina reproduzindo o padrão que utiliza habitualmente em contratos. **Para que servem.** Fornecer padrões conhecidos para representar a variabilidade natural do padrão contratual, formar pares genuínos, calibrar e comparar com as assinaturas QUESTIONED. **Regra crítica.** RC não é assinatura retirada do contrato sob exame.

RO — referências oficiais

O que são. Assinaturas também colhidas diretamente do cliente/subscritor. **Como são produzidas.** O subscritor assina reproduzindo o padrão que utiliza em documentos oficiais, como RG, CNH ou outros documentos de identificação em que empregue esse padrão. **Para que servem.** Fornecer um segundo conjunto conhecido de referência, com padrão gráfico oficial, para calibração própria e comparação com as QUESTIONED. **Regra crítica.** RO não significa simplesmente copiar a imagem de um RG ou CNH; a RO usada no procedimento é colhida do subscritor reproduzindo esse padrão.

QUESTIONED — assinaturas questionadas

O que são. Todas as assinaturas existentes no contrato que está sendo submetido ao exame. **Para que servem.** Constituem o material cuja autoria/compatibilidade está sob avaliação. **Regra crítica.** Nenhuma assinatura do contrato é inicialmente RC ou RO; todas permanecem QUESTIONED durante a rastreabilidade do caso, mesmo depois de receber classificação pericial.

```
SUBSCRITOR
   ├─ coleta reproduzindo padrão de contrato → RC
```

```
   └─ coleta reproduzindo padrão oficial     → RO

CONTRATO SOB EXAME
   └─ todas as assinaturas → QUESTIONED
```

5.2.1 Classificação pericial humana das QUESTIONED

No protocolo forense, o perito grafoscopista examina as assinaturas QUESTIONED e atribui o papel pericial necessário ao procedimento. Essa classificação **não é inferida automaticamente pelo modelo**.

QUESTIONED_COMPATIBLE

QUESTIONED classificada pelo perito grafoscopista como compatível com o padrão do subscritor. Ela continua pertencendo ao conjunto QUESTIONED e não se transforma em RC ou RO.

QUESTIONED_IMPOSTOR

QUESTIONED classificada pelo perito como divergente/incompatível com o padrão do subscritor e utilizada, quando o protocolo FORENSIC_CASE assim define, como non-match para calibração. Ela continua sendo uma assinatura questionada do contrato.

Exemplo didático: um contrato possui 14 assinaturas. Todas entram no caso como QUESTIONED. Após o exame grafoscópico, o perito pode classificar 7 como compatíveis e 7 como impostoras. Essa classificação humana antecede a calibração forense.

```
14 QUESTIONED do contrato
          ↓
análise do perito grafoscopista
          ↓
 ├─ 7 QUESTIONED_COMPATIBLE
 └─ 7 QUESTIONED_IMPOSTOR
          ↓
calibração forense
          ↓
comparações RC×QUESTIONED e RO×QUESTIONED
```

Diferença entre protocolo forense e científico

No `FORENSIC_CASE`, a classificação humana pode participar da formação das populações de calibração conforme o pipeline documentado. No `SCIENTIFIC_EXTERNAL`, as QUESTIONED permanecem fora da estimação do threshold; os non-matches vêm de impostores externos e as QUESTIONED são usadas posteriormente na inferência/comparação.

5.3 Calibração estatística

A calibração constrói duas populações:

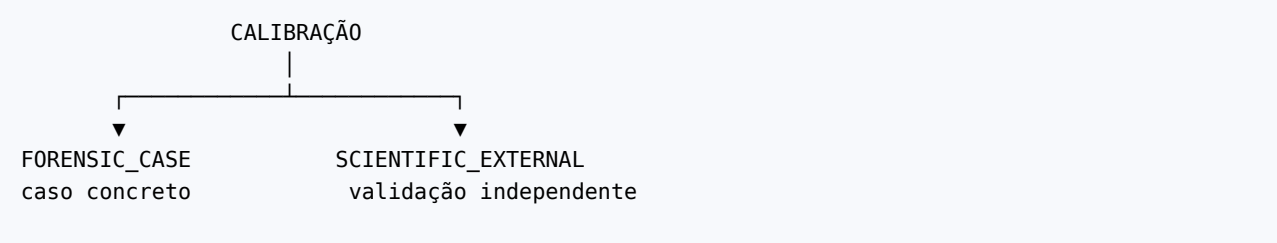
```
MATCH
NON-MATCH
```

e calcula:

- FAR;

- FRR;
- EER;
- τ_EER;
- limiar calibrado;
- τ operacional.

5.4 Dupla calibração adaptativa



5.5 Pipeline detalhado FORENSIC_CASE — RC

Entrada:

```
n referências RC
questionadas classificadas pelo grafoscopista
```

Genuine:

```
C(n,2)
```

Com cinco RC:

```
C(5,2)=10
```

Non-match:

```
QUESTIONED_IMPOSTOR × RC
```

Com sete questionadas impostoras:

```
7×5=35
```

Regra crítica: `QUESTIONED_COMPATIBLE` não é promovida à população genuína. Material questionado continua sendo material questionado.

5.6 Pipeline detalhado FORENSIC_CASE — RO

Genuine:

```
C(n_RO,2)
```

Com cinco RO:

```
10
```

Non-match:

```
todas as questionadas × RO
```

Com 14 questionadas e cinco RO:

```
14×5=70
```

Esse comportamento faz parte do protocolo forense RO estabelecido.

5.7 Pipeline detalhado SCIENTIFIC_EXTERNAL [16]

O **caso científico possui os mesmos três conjuntos documentais do caso forense**:

```
RC
RO
QUESTIONED
```

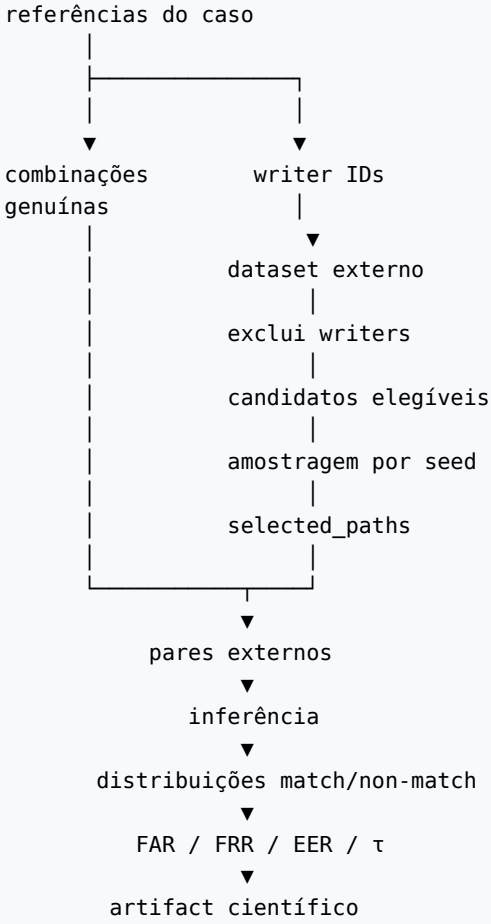
A diferença não está na existência desses conjuntos no caso, mas **na função metodológica de cada um durante a calibração**.

- **RC**: pode constituir o conjunto de referências do cenário científico RC;
- **RO**: pode constituir o conjunto de referências do cenário científico RO;
- **QUESTIONED**: permanece cadastrada no caso e é usada posteriormente na inferência/avaliação, **mas não participa da estimação do threshold científico**.

Assim, o fluxo completo do protocolo científico é:

```
CASO
 ├─ RC
 ├─ RO
 └─ QUESTIONED
       │
       ├──────────────────────────────────────┐
       │                                      │
       ▼                                      ▼
CALIBRAÇÃO CIENTÍFICA                  INFERÊNCIA FORENSE
usa RC ou RO + impostores externos     usa RC/RO + QUESTIONED
QUESTIONED excluída do threshold       aplica threshold calibrado
```

Essa distinção é essencial: **a assinatura questionada existe no caso científico, assim como no caso forense, porém é mantida fora da população usada para estimar o limiar científico**, preservando independência entre calibração e avaliação [16].



As questionadas **não entram** na estimação do threshold científico. Essa separação entre calibração e avaliação forense é um princípio metodológico central do protocolo científico associado ao SFAAO [16].

5.7.1 Cenário científico RC

```
RC do caso
 ↓
combinações RC×RC → população genuína
 ↓
RC×impostores externos → população non-match
 ↓
FAR / FRR / EER / threshold
 ↓
QUESTIONED permanece reservada para inferência
```

Com cinco RC:

```
C(5,2) = 10 pares genuínos
```

5.7.2 Cenário científico RO

```
RO do caso
 ↓
combinações RO×RO → população genuína
 ↓
RO×impostores externos → população non-match
 ↓
FAR / FRR / EER / threshold
 ↓
QUESTIONED permanece reservada para inferência
```

Com cinco RO:

```
C(5,2) = 10 pares genuínos
```

5.7.3 Papel das QUESTIONED no protocolo científico

As QUESTIONED:

```
existem no caso
são exibidas e gerenciadas normalmente
são usadas na etapa de comparação/inferência
não são usadas para estimar τ_EER científico
não entram na seleção de impostores externos
não alteram a população de calibração
```

Portanto:

```
FORENSIC_CASE       → RC + RO + QUESTIONED
SCIENTIFIC_EXTERNAL → RC + RO + QUESTIONED
```

Nos dois casos, os três conjuntos documentais pertencem ao caso. A diferença está **em como as QUESTIONED participam — ou não — da formação das populações de calibração**.

Com cinco referências e 2.000 impostores:

```
genuine = 10
non-match = 2.000×5 = 10.000
```

Com cinco referências e 100 impostores:

```
genuine = 10
non-match = 100×5 = 500
```

5.8 Proveniência científica [16]

A seleção externa pode registrar:

```
dataset_name
dataset_profile
dataset_root
dataset_version
profile_name
profile_version
selection_method
seed
excluded_writers
eligible_count
requested_count
selected_count
selection_hash_sha256
selected_paths
```

O hash SHA-256 permite identificar a seleção exata utilizada. Esse registro complementa a exigência científica de tornar a população externa identificável e reproduzível [16].

5.9 FAR, FRR e EER

As três métricas dependem da população de **MATCH** (genuínos) e **NON-MATCH** (impostores), do threshold τ e da direção da métrica de comparação [5].

5.9.1 FAR — False Acceptance Rate [5]

Definição formal. FAR é a proporção de non-matches aceitos incorretamente:

$$FAR(\tau) = \frac{\#\{\text{non-matches aceitos em } \tau\}}{N_{nonmatch}}.$$

No SFAAO:

$$FAR_{cos}(\tau) = \frac{\#\{s_i^I \geq \tau\}}{N_I}, \qquad FAR_E(\tau) = \frac{\#\{d_i^I \leq \tau\}}{N_I}.$$

Assim, o erro de falsa aceitação usa $\geq$ para similaridade por cosseno e $\leq$ para distância Euclidiana.

5.9.2 FRR — False Rejection Rate [5]

Definição formal. FRR é a proporção de matches rejeitados incorretamente:

$$FRR(\tau) = \frac{\#\{\text{matches rejeitados em } \tau\}}{N_{match}}.$$

No SFAAO:

$$FRR_{cos}(\tau) = \frac{\#\{s_i^G < \tau\}}{N_G}, \qquad FRR_E(\tau) = \frac{\#\{d_i^G > \tau\}}{N_G}.$$

5.9.3 EER — Equal Error Rate [5]

Definição formal. O EER é a taxa associada ao ponto em que FAR e FRR se igualam ou, em dados discretos, ao threshold que minimiza sua diferença:

$$\tau_{EER} = \arg\min_{\tau} \left| FAR(\tau) - FRR(\tau) \right|.$$

O EER deve ser interpretado considerando conjunto de dados, protocolo, modelo, métrica e amostragem usados. Um EER igual a zero em uma calibração significa separação perfeita **naquela população avaliada**; não implica taxa de erro universal igual a zero.

5.10 Thresholds, FAR-alvo e dataset shift [15], [16]

O SFAAO distingue quatro conceitos que não devem ser confundidos:

- τ_{modelo} — threshold proveniente da avaliação do modelo no contexto de treinamento/validação; permanece como informação de proveniência do modelo e não determina, por si só, a decisão do caso;
- $\tau_{EER,cal}$ — threshold estimado no ponto em que FAR e FRR da população de calibração se aproximam;
- τ_{FAR} — threshold selecionado pela calibração sob a restrição de FAR-alvo empírico;
- $\tau_{operacional}$ — threshold efetivamente aplicado na decisão técnica.

No SFAAO 2.4.0, a interface oficial de calibração fixa `CalibrationWindow.TARGET_FAR = 0.01`. Portanto, o FAR-alvo operacional utilizado pela GUI é

$$\alpha = 0.01 = 1\%.$$

Esse parâmetro pertence à **calibração**, e não ao treinamento. Ele é transferido para `CalibrationConfig` como `target_far`; a interface oficial não expõe um controle para que o usuário altere esse valor.

Para similaridade por cosseno, a restrição é avaliada por

$$\widehat{FAR}_{cos}(\tau) = \frac{\#\{s_i^I \geq \tau\}}{N_I} \leq 0.01.$$

Para distância Euclidiana,

$$\widehat{FAR}_{E}(\tau) = \frac{\#\{d_i^I \leq \tau\}}{N_I} \leq 0.01.$$

A calibração procura, entre os thresholds avaliados, um limiar τ_{FAR} que satisfaça essa restrição empírica. Esse processo **não força o EER a ser menor ou igual a 1%**. O EER é calculado independentemente por

$$\tau_{EER,cal} = \arg\min_{\tau} \left| \widehat{FAR}(\tau) - \widehat{FRR}(\tau) \right|.$$

O threshold original de treinamento não substitui a calibração específica do caso. A motivação é o **dataset shift**: distribuições de scores no treinamento podem diferir das distribuições encontradas no caso operacional [15], [16].

5.10.1 Similaridade por cosseno: maior valor é mais genuíno

Como a aceitação ocorre para valores altos, uma política operacional mais conservadora desloca o threshold para cima. Quando τ_{FAR} e $\tau_{EER,cal}$ estão disponíveis:

$$\boxed{\tau_{operacional}^{cos} = \max\left(\tau_{EER,cal}, \tau_{FAR}\right)}.$$

A política operacional usa três zonas. Para similaridade por cosseno:

$$\begin{cases} \text{COMPATÍVEL}, & s \geq \tau_{operacional}^{cos}, \\ \text{INCONCLUSIVO}, & \tau_{EER,cal} \leq s < \tau_{operacional}^{cos}, \\ \text{INCOMPATÍVEL}, & s < \tau_{EER,cal}. \end{cases}$$

5.10.2 Distância Euclidiana: menor valor é mais genuíno

Na distância Euclidiana, a orientação é oposta: valores menores indicam maior proximidade. Portanto, uma política operacional mais conservadora desloca o threshold para baixo. Quando ambos os limiares estão disponíveis:

$$\boxed{\tau^{E}_{operacional} = \min\left(\tau_{EER,cal}, \tau_{FAR}\right)}.$$

A política operacional usa três zonas. Para distância Euclidiana:

$$\begin{cases} \text{COMPATÍVEL}, & d_E \leq \tau^{E}_{operacional}, \\ \text{INCONCLUSIVO}, & \tau^{E}_{operacional} < d_E \leq \tau_{EER,cal}, \\ \text{INCOMPATÍVEL}, & d_E > \tau_{EER,cal}. \end{cases}$$

Interpretação estatística do FAR-alvo

O requisito $\widehat{FAR} \leq 1\%$ é uma restrição **empírica** observada na população de calibração. Ele não constitui garantia matemática de que a FAR verdadeira em toda população futura permanecerá menor ou igual a 1%. A interpretação depende da população impostora, do tamanho amostral, da dependência entre comparações, do modelo, da métrica e das condições operacionais.

Regra de segurança semântica

Nunca se deve reutilizar automaticamente a regra "score maior é melhor". O SFAAO registra a métrica e sua orientação: cosseno usa `higher_is_genuine=True`; Euclidiana usa `higher_is_genuine=False`. A decisão, os gráficos, FAR/FRR e a persistência de resultados devem respeitar essa semântica.

5.11 Política operacional de três zonas e intervalo entre limiares

No SFAAO 2.4.0, o intervalo entre τ_{EER} e $\tau_{operacional}$ é parte explícita da decisão computacional. A política deixa de ser binária e passa a distinguir **COMPATÍVEL**, **INCONCLUSIVO** e **INCOMPATÍVEL**.

Para similaridade por cosseno, quando $\tau^{cos}_{operacional} \geq \tau_{EER,cal}$:

$$\begin{cases} \text{COMPATÍVEL}, & s \geq \tau^{cos}_{operacional}, \\ \text{INCONCLUSIVO}, & \tau_{EER,cal} \leq s < \tau^{cos}_{operacional}, \\ \text{INCOMPATÍVEL}, & s < \tau_{EER,cal}. \end{cases}$$

Para distância Euclidiana, quando $\tau^{E}_{operacional} \leq \tau_{EER,cal}$:

$$\begin{cases} \text{COMPATÍVEL}, & d_E \leq \tau^{E}_{operacional}, \\ \text{INCONCLUSIVO}, & \tau^{E}_{operacional} < d_E \leq \tau_{EER,cal}, \\ \text{INCOMPATÍVEL}, & d_E > \tau_{EER,cal}. \end{cases}$$

Quando $\tau_{operacional} = \tau_{EER}$ não existe zona intermediária. Em LORO, essa regra é aplicada individualmente a cada fold A(N-1); os folds não são novamente promediados nem submetidos a votação majoritária.

5.11.1 Domínio estatístico alinhado: A1, A(N-1) e A(N)

A versão operacional atual do SFAAO distingue explicitamente três níveis de score. O score **A1** é o resultado pairwise entre uma referência e uma assinatura questionada. Em calibração e inferência LORO, A1 permanece como componente auditável e proveniência, mas não é a unidade estatística final de decisão.

Com N referências, o domínio decisório é **A(N-1)**. A estratégia `LEAVE_ONE_REFERENCE_OUT` exclui uma referência por fold e calcula a média dos scores contra as $N-1$ referências restantes. Assim, com cinco referências, a calibração e a inferência utilizam **A4** e produzem cinco folds.

A média contra todas as N referências, **A(N)**, permanece disponível como estatística descritiva. Ela não recebe o threshold LORO e não substitui os folds A(N-1).

A configuração operacional correspondente é:

```
decision_statistic = LORO_MEAN
fold_strategy      = LEAVE_ONE_REFERENCE_OUT
aggregation_size   = reference_count - 1
fold_count         = reference_count
```

Portanto, quando $N = 5$:

```
A1 = score pairwise / proveniencia
A4 = unidade decisoria LORO
A5 = estatistica descritiva
folds = 5
```

5.11.2 Thresholds e política operacional de três zonas

O SFAAO diferencia três conceitos de limiar:

- τ_{EER}: threshold correspondente ao ponto de EER da calibração corrente;
- $\tau_{forensic}$: threshold associado ao FAR-alvo configurado;
- $\tau_{operacional}$: threshold final efetivamente empregado pela política operacional.

Para métricas de similaridade, como Cosine Similarity:

$$\tau_{operacional} = \max(\tau_{forensic}, \tau_{EER}).$$

A regra é:

```
score >= tau_operacional
    -> COMPATIBLE

tau_EER <= score < tau_operacional
    -> INCONCLUSIVE

score < tau_EER
    -> INCOMPATIBLE
```

Para métricas de distância, como Euclidean Distance:

$$\tau_{operacional} = \min(\tau_{forensic}, \tau_{EER}).$$

A regra é:

```
score <= tau_operacional
    -> COMPATIBLE

tau_operacional < score <= tau_EER
    -> INCONCLUSIVE

score > tau_EER
    -> INCOMPATIBLE
```

Quando $\tau_{operacional} = \tau_{EER}$, não existe zona inconclusiva intermediária.

No protocolo LORO, a regra acima é aplicada **individualmente a cada fold A(N-1)**.

5.11.3 Consolidação conservadora dos folds

Os folds LORO não são novamente promediados para gerar uma segunda decisão e não existe votação majoritária. A consolidação é por unanimidade:

- todos `COMPATIBLE`: `STABLE_COMPATIBLE`;
- todos `INCOMPATIBLE`: `STABLE_INCOMPATIBLE`;
- todos `INCONCLUSIVE`: `STABLE_INCONCLUSIVE`;
- qualquer mistura: `REFERENCE_SENSITIVE`, com decisão consolidada `INCONCLUSIVE`.

Esse desenho permite detectar sensibilidade ao conjunto de referências e evita ocultá-la por média ou maioria.

5.11.4 Alinhamento entre calibração e inferência

O schema v6 registra separadamente o domínio da calibração e o domínio da inferência. O campo `alignment_status` pode assumir:

```
ALIGNED
MISMATCH
LEGACY_MISMATCH
```

Novas execuções operacionais LORO devem ser `ALIGNED`. `MISMATCH` indica desalinhamento estatístico. Registros históricos em que um threshold A1 foi aplicado a uma estatística agregada permanecem identificados como `LEGACY_MISMATCH` e não são reinterpretados silenciosamente.

5.11.5 Semântica dos protocolos RC, RO e científico

No protocolo `SCIENTIFIC_EXTERNAL`, as assinaturas QUESTIONED do caso não participam da estimação do threshold; os non-matches provêm de impostores externos e a avaliação das QUESTIONED ocorre posteriormente.

No protocolo `FORENSIC_CASE` com RC, apenas QUESTIONED previamente classificadas pelo perito como `QUESTIONED_IMPOSTOR` podem compor a população non-match. QUESTIONED compatíveis não são promovidas a genuine e QUESTIONED não classificadas não entram silenciosamente nessa população.

No contexto RO, as QUESTIONED são intencionalmente tratadas como non-match frente às referências oficiais, conforme a definição operacional desse protocolo.

5.11.6 Persistência dos folds e decisão auditável

No schema v6, `questioned_results` registra a estatística descritiva, o domínio decisório, as contagens de folds, o estado de estabilidade e a decisão consolidada. Cada fold é persistido em `questioned_decision_folds`, incluindo referência excluída, A(N-1), τ_{EER}, $\tau_{operacional}$ e decisão técnica.

A persistência do resultado da QUESTIONED e de seus folds é atômica: ou o resultado e todos os folds são gravados, ou a transação é revertida. Não deve existir resultado LORO operacional com conjunto parcial de folds.

5.12 Pipeline detalhado de comparação

```
01. abrir caso
02. escolher modelo ativo
03. resolver calibração selecionada
04. escolher RC ou RO
05. carregar referências
06. carregar questionadas
07. para cada questionada:
08.     para cada referência:
09.         preprocessar
10.         inferir embeddings
11.         calcular score
12.         persistir score
13. preservar A1 pairwise como proveniencia
14. construir folds LORO A(N-1)
15. validar alignment_status = ALIGNED
16. aplicar tau_EER / tau_operacional em cada fold
17. persistir questioned_decision_folds
18. consolidar STABLE_* ou REFERENCE_SENSITIVE
19. persistir ComparisonRun + questioned_results
20. manter A(N) apenas como estatistica descritiva
```

Com cinco referências e 14 questionadas:

```
70 scores individuais
14 agregados
```

5.13 Decisão técnica versus classificação pericial [16]

```
CLASSIFICAÇÃO DO GRAFOSCOPISTA
           ≠
DECISÃO COMPUTACIONAL
```

Uma assinatura classificada pericialmente como compatível pode receber decisão computacional incompatível. O sistema não copia a classificação humana. No protocolo científico, as questionadas permanecem fora da calibração e são reservadas à avaliação [16].

Capítulo 6

Rastreabilidade, auditoria e segurança

6.1 Rastreabilidade

Cada resultado deve permitir reconstruir:

```
usuário
 → caso
 → assinatura
 → modelo
 → configuração
 → calibração
 → threshold
 → ComparisonRun
 → scores
 → agregado
 → laudo
```

6.2 Hierarquia dos dados

```
Usuário/Perito
   ↓
Caso
   ↓
Assinaturas
   ↓
Modelo
   ↓
Calibração
   ↓
Comparação
   ↓
Resultados
   ↓
Relatório
```

A calibração pertence simultaneamente ao **caso** e ao **modelo**. Ela nunca é global.

6.3 Modelo como artefato científico

Um modelo não é apenas um arquivo de pesos.

Seu pacote deve incluir, quando aplicável:

```
model weights
model_config.json
training_config.json
```

```
history
evaluation metrics
EER
threshold
plots
metadata
```

Cada modelo deve poder ser tratado como experimento científico independente.

6.4 Auditoria e integridade

Alterações relevantes de classificação devem registrar:

```
old value
new value
actor
timestamp
```

Exemplo:

```
QUESTIONED_COMPATIBLE
→ QUESTIONED_IMPOSTOR
→ QUESTIONED_COMPATIBLE
```

A integridade evita remoção silenciosa de dados já utilizados.

6.5 Regras de exclusão

- calibração selecionada ou utilizada em comparação: bloquear;
- assinatura utilizada por resultados/comparações: bloquear;
- caso com evidência histórica: preservar;
- exclusão física somente quando segura;
- artefatos em storage só devem ser removidos depois do sucesso da operação no banco.

6.5.1 Política de acesso a dados

A especificação arquitetural determina independência entre usuários e estabelece que um usuário não deve obter automaticamente informações pertencentes a outro. A implementação concreta dessa política deve respeitar a configuração institucional e as regras de autorização definidas para o deployment.

Essa regra é distinta da separação de **roles**:

```
ADMIN    → governa a plataforma
EXAMINER → executa operações periciais autorizadas
```

A ocultação de funções administrativas para EXAMINER foi homologada no release 2.4.0. Políticas adicionais de segregação de casos por usuário ou unidade organizacional devem ser validadas de acordo com a implantação institucional.

6.6 Segurança

6.6.1 Autenticação

Senhas não são armazenadas em texto puro. O SFAAO utiliza **PBKDF2-HMAC-SHA256**, mecanismo de derivação de chave que aplica muitas iterações criptográficas à senha antes de persistir seu verificador. Um **salt aleatório** é um valor diferente associado a cada senha para dificultar tabelas pré-calculadas e ataques por hashes repetidos. `compare_digest` realiza comparação resistente a diferenças simples de tempo de execução. Esses mecanismos reduzem o risco de exposição direta das credenciais caso o banco seja acessado indevidamente.

Há também controle de tentativas de login e bloqueio temporário para dificultar ataques repetitivos de adivinhação de senha.

6.6.2 ADMIN

Governa:

- usuários;
- datasets;
- modelos;
- treinamento;
- ativação;
- operações administrativas.

6.6.3 EXAMINER

Opera:

- casos;
- assinaturas;
- calibração;
- comparação;
- laudos.

Não deve visualizar menus administrativos.

Princípio:

```
O PERITO USA.
O ADMINISTRADOR GOVERNA.
```

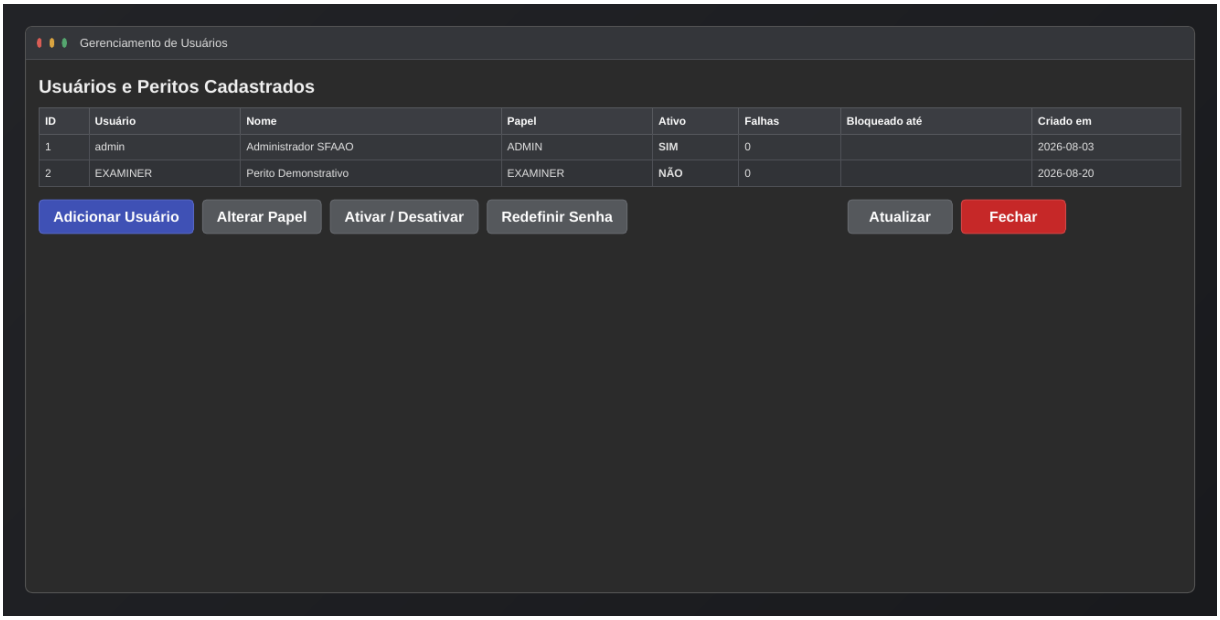


Figura 6.1: Gestão de usuários e peritos, exibindo cadastro, papéis, estado de ativação e ações administrativas.

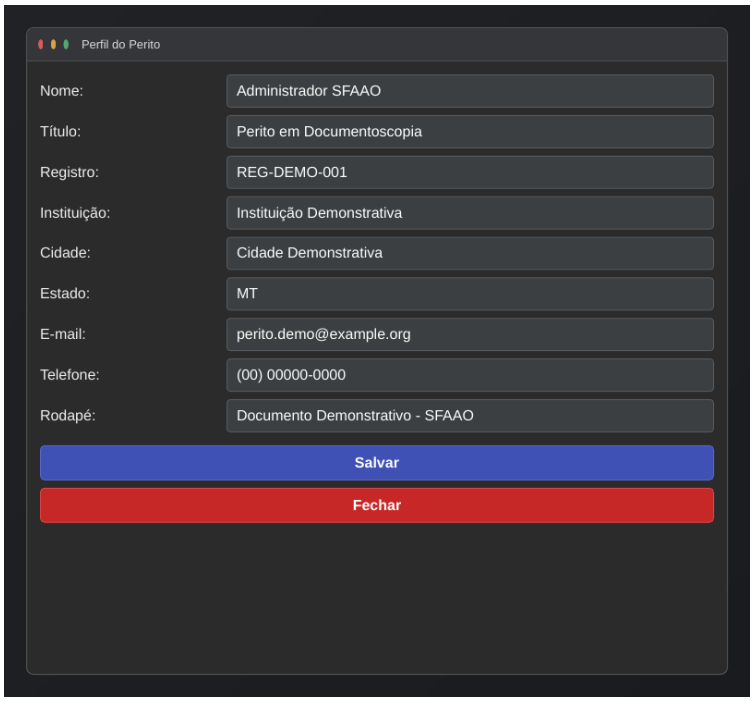


Figura 6.2: Cadastro do perfil profissional, com dados de identificação e contato do perito.

6.6.4 Bootstrap de segurança e integridade

Antes de qualquer tela de autenticação ser apresentada, o `SecurityBootstrap` executa uma cadeia de verificações de segurança. A sequência consolidada do release final compreende proteção do storage, validação criptográfica da licença, vínculo com a identidade da máquina, validação do estado temporal protegido, proteção contra retrocesso do relógio e verificação do manifesto de integridade por SHA-256. Uma falha crítica interrompe a inicialização antes da GUI.

O bootstrap de segurança ocorre exclusivamente em segundo plano, sem uma tela ou janela própria. O usuário interage com o sistema somente após a conclusão das verificações internas.

6.7 Primeiro administrador

Instalação limpa:

```
0 usuários
 ↓
FirstAdminWindow
 ↓
criação de um ADMIN
 ↓
login convencional
```

Não existem credenciais padrão embutidas.

6.8 Segurança do storage

O **storage** é a área persistente que guarda banco de dados, casos, imagens, modelos, calibrações, resultados, relatórios, licença e estado de segurança. Como esses itens podem ter valor pericial, sua proteção é parte da arquitetura do SFAAO.

Em sistemas POSIX, a política utiliza permissões restritivas, como `umask 077`, diretórios 700 e arquivos 600 quando aplicável.

Na arquitetura Windows 2.4.0, a persistência deve ser mantida em:

```
C:
ProgramData
SFAAO
```

A instalação em `C:`
`Program Files`
`SFAAO` utiliza junctions para direcionar os diretórios mutáveis ao `ProgramData`:

```
storage          → C:
ProgramData
SFAAO
storage
models/default   → C:
ProgramData
SFAAO
models
default
models/custom    → C:
ProgramData
SFAAO
models
custom
security/state   → C:
ProgramData
SFAAO
security
state
```

Uma **junction** é um vínculo do filesystem do Windows que faz um caminho apontar para outro diretório. Ela permite manter o executável em Program Files e os dados persistentes em ProgramData sem alterar os caminhos esperados pela aplicação.

No ciclo Windows 2.4.0, o instalador aplicou proteção por ACL aos dados persistentes. A política é evitar acesso amplo por `Users` ou `Everyone` e conceder somente as permissões necessárias às contas autorizadas. Qualquer instalação em novo equipamento requer confirmação local das ACLs efetivas.

O banco SQLite, arquivos WAL/SHM e demais artefatos persistentes permanecem sujeitos à mesma política de proteção. A continuidade dos dados após atualização/reinstalação foi verificada no ambiente Windows 11 homologado; recomenda-se novo teste de persistência em cada ambiente institucional.

Capítulo 7

Laudos e fluxos operacionais

7.1 Pipeline detalhado de geração de laudo

No release final, a geração de laudo combina montagem técnico-científica, versionamento, integridade do artefato e governança transacional de **quota**. Quota é o limite quantitativo de uso definido pela licença para determinado recurso. Neste fluxo, a reserva e o consumo são registrados de forma transacional para evitar inconsistências. O fluxo consolidado é:

```
01. caso e protocolo selecionados
02. validar feature/formato permitido pela licença
03. construir contexto pericial atual
04. calcular fingerprint e chave lógica SHA-256 do contexto
05. procurar laudo lógico reutilizável
06. BEGIN IMMEDIATE
07. se novo laudo: validar quota + reservar versão + registrar ledger
08. se formato complementar: reutilizar report_uid/versão sem nova quota
09. ReportPipeline carrega caso, perito, modelo, calibrações e comparações
10. gerar DOCX ou PDF
11. reconstruir contexto e verificar ausência de mudança concorrente
12. versionar artefato físico
13. calcular SHA-256 do arquivo
14. vincular caminho/hash ao registro do laudo
15. em falha sem artefato válido: marcar FAILED + liberar quota de forma idempotente
```

A reserva de um novo laudo e o lançamento da quota ocorrem na mesma transação. A geração do formato complementar do mesmo laudo lógico — por exemplo, PDF após DOCX, ou DOCX após PDF — reutiliza o mesmo `report_uid` e a mesma versão, sem novo consumo. Quando um novo laudo falha antes de possuir artefato válido associado, o registro permanece com estado `FAILED` para rastreabilidade e a reserva de quota correspondente é liberada de forma idempotente. A geração do formato complementar do mesmo laudo lógico reutiliza o mesmo `report_uid` e não cria novo consumo.

O laudo pode conter identificação, demanda, metodologia, modelo, dataset, backbone, loss, métrica, embedding, dropout, métricas originais, calibrações RC/RO, FAR/FRR/EER, limiares, UIDs, resultados consolidados, resumo executivo, achados automáticos, síntese computacional e anexos. A síntese computacional não substitui a conclusão do perito.

Proveniência estatística do laudo. O pipeline atual de laudos consome resultados persistidos e valida o alinhamento estatístico antes da materialização documental. A metadata de proveniência estatística utiliza `schema_version 3`. O fingerprint lógico inclui, entre outros elementos, domínio de calibração e inferência, tamanho de agregação, estratégia de folds, número de referências, número de folds, τ_{EER}, $\tau_{operacional}$, resultados A(N), folds A(N-1), contagens, `stability_status` e decisão consolidada. Alterações nesses elementos decisórios modificam o contexto lógico do laudo.

7.2 Fluxo operacional completo

```
INSTALAÇÃO
 ↓
PRIMEIRO ADMIN
 ↓
LOGIN
 ↓
CADASTRO DE USUÁRIOS
 ↓
CADASTRO/VALIDAÇÃO DE DATASET
 ↓
TREINAMENTO OU IMPORTAÇÃO DE MODELO
 ↓
ATIVAÇÃO DO MODELO
 ↓
CRIAÇÃO DO CASO
 ↓
CADASTRO DO PERITO
 ↓
UPLOAD RC
 ↓
UPLOAD RO
 ↓
UPLOAD QUESTIONADAS
 ↓
CLASSIFICAÇÃO PERICIAL
 ↓
CALIBRAÇÃO FORENSE OU CIENTÍFICA
 ↓
COMPARAÇÃO RC
 ↓
COMPARAÇÃO RO
 ↓
REVISÃO DOS RESULTADOS
 ↓
GERAÇÃO DOCX/PDF
 ↓
ARQUIVAMENTO
```

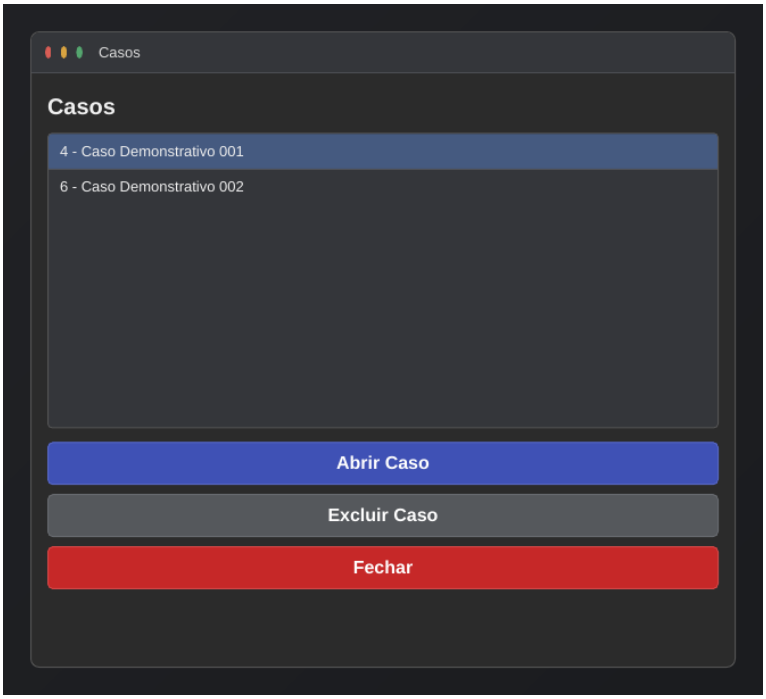


Figura 7.1: Lista de casos, com opções de abertura, exclusão e fechamento da janela.

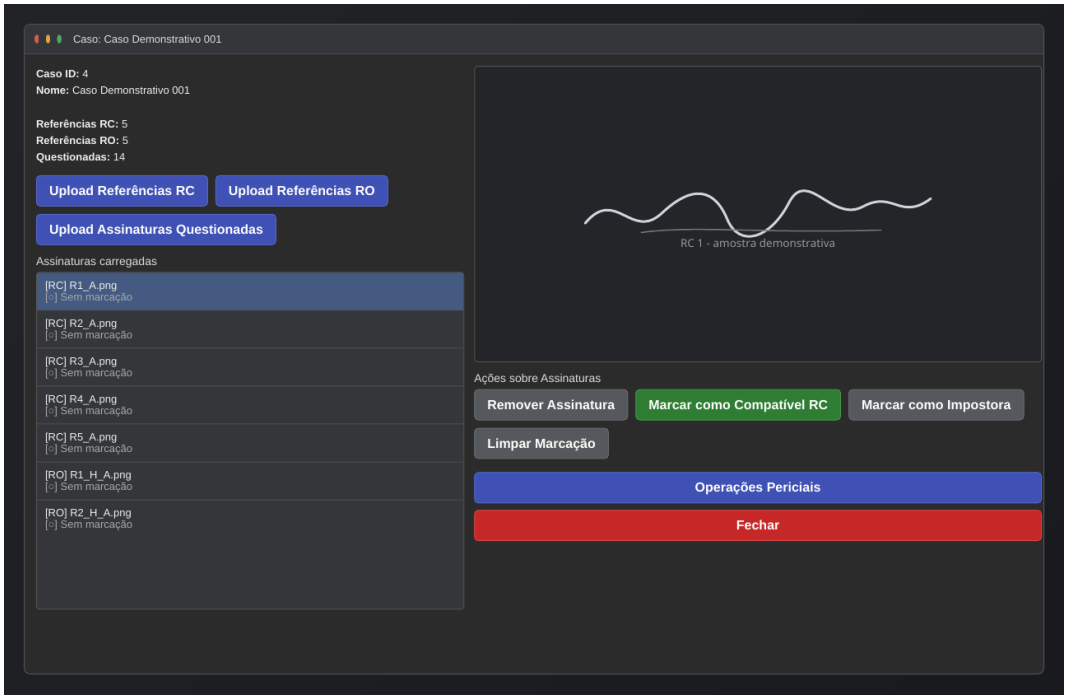


Figura 7.2: Janela de assinaturas do caso: referências, amostras questionadas, visualização e operações sobre arquivos.



Figura 7.3: Menu de operações do caso, com acesso aos dados e à geração de laudos, calibração, comparação e resultados em lote/CSV.

7.2.1 Pipeline de inicialização da aplicação

```
app.py
 ↓
SecurityBootstrap
 ↓
proteção do storage / permissões seguras
 ↓
validação da licença e criação do LicenseContext
 ↓
MachineFingerprint V2
 ↓
SecureTimeState + TimeGuard
 ↓
IntegrityChecker + manifesto SHA-256
 ↓
inicialização/migração do schema SQLite
 ↓
contagem de usuários
 ├─ 0 usuários → FirstAdminWindow
 └─ ≥1 usuário → LoginWindow
 ↓
autenticação + resolução ADMIN/EXAMINER
 ↓
Main Window com permissões correspondentes
```

A GUI somente é iniciada quando as validações críticas de segurança são aprovadas. O fingerprint V2 evita dependência de hostname ou endereço MAC e utiliza identificadores estáveis do sistema operacional, combinados com plataforma e arquitetura, antes da derivação SHA-256.

7.2.2 Pipeline de governança de modelo

```
ADMIN
 ↓
Gerenciador de Modelos
 ↓
importar pacote OU treinar novo modelo
 ↓
validar estrutura/configuração
 ↓
registrar no catálogo
 ↓
definir como ativo
 ↓
modelo disponível aos fluxos operacionais
```

A ativação não altera resultados históricos. Comparações antigas permanecem vinculadas ao modelo e à calibração originalmente utilizados.

7.2.3 Pipeline de governança de dataset

```
ADMIN
 ↓
Gerenciador de Datasets
 ↓
adicionar dataset
 ↓
selecionar profile
 ↓
validar classificação e estrutura
 ↓
persistir catálogo
 ↓
dataset habilitado para treinamento/calibração científica
```

A remoção de um dataset do catálogo não equivale à redistribuição nem à exclusão física de uma base de terceiros.

7.2.4 Licenciamento, perfis e cotas

Uma **licença** define quais recursos e limites a instalação pode utilizar. Uma **quota** é o limite quantitativo de determinado recurso. O release final possui três perfis operacionais: `TRIAL`, `ACADEMIC` e `FULL`. O `LicenseContext` é a estrutura interna que reúne as permissões válidas após a licença ser verificada. A política **fail-closed** significa que, se um limite obrigatório estiver ausente ou inválido, o sistema bloqueia a operação em vez de presumir permissão. O valor interno `None`, quando previsto pelo perfil válido, representa recurso ilimitado.

Recurso	TRIAL	ACADEMIC	FULL
Usuários ativos	1	10	ilimitado
Peritos EXAMINER ativos	1	5	ilimitado
Casos	2	50	ilimitado
Comparações individuais	ilimitado	ilimitado	ilimitado

Recurso	TRIAL	ACADEMIC	FULL
Execuções em lote	3	100	ilimitado
Laudos	1	20	ilimitado
Exportações CSV	3	50	ilimitado
Treinamentos	3	100	ilimitado
Calibrações	2	50	ilimitado
Datasets disponíveis	2	10	ilimitado
Modelos cadastrados	3	20	ilimitado

Quanto aos protocolos, TRIAL permite científico e forense; ACADEMIC permite o protocolo científico e bloqueia o protocolo forense por política; FULL permite ambos. O recurso `audit_trail` é habilitado apenas em FULL. Treinamento, calibração, comparação, análise em lote e geração DOCX/PDF permanecem habilitados nos três perfis, sujeitos às cotas aplicáveis.

7.2.4.1 Semântica das cotas

As cotas são classificadas em três grupos. Um **ledger de uso** é um registro transacional dos consumos já realizados, semelhante a um livro de lançamentos. Recursos cumulativos — `training_runs`, `calibrations`, `batch_executions`, `reports` e `csv_exports` — são contabilizados por esse ledger. Recursos de estado atual — `users`, `examiners`, `datasets` e `models` — são medidos diretamente no banco. Casos usam regra híbrida, adotando o maior valor entre ledger e estado atual, evitando que a simples exclusão reduza retroativamente o consumo autorizado.

A política operacional do SFAAO 2.4.0 utiliza **reserva transacional com rollback idempotente** para recursos cumulativos. **Rollback** significa desfazer o consumo reservado quando a operação falha sem produzir o resultado válido. **Idempotente** significa que repetir a liberação da mesma reserva não provoca consumo negativo nem efeito duplicado. A quota é autorizada antes da operação, mas uma falha operacional que não produz o artefato/resultado válido correspondente libera a reserva aplicável sem apagar o registro de falha necessário à rastreabilidade.

No release homologado, há liberação idempotente de uso para treinamento, calibração, execução em lote e exportação CSV quando a operação falha. Para relatórios, um novo laudo que falha antes de possuir artefato válido permanece `FAILED` e libera a quota; a criação do formato complementar DOCX/PDF do mesmo laudo lógico não consome novamente. Cancelamentos ocorridos antes da autorização, como a escolha de destino de CSV, também não geram consumo.

Esse desenho evita cobrança de quota por operação que não produziu o resultado esperado, sem perder a trilha de auditoria da tentativa.

A homologação Windows 11 relatou dois testes de treinamento iniciado e posteriormente cancelado, nos quais não houve consumo de quota de `training_runs`. Essa evidência cobre os dois cenários exercitados; a recuperação de quota em outros caminhos de falha exige testes específicos e não deve ser considerada universalmente demonstrada. A geração de laudos DOCX foi homologada funcionalmente no Windows 11, no Linux Mint e no Red Hat, segundo o responsável técnico. Isso não dispensa ensaios específicos de rotas transacionais não descritas no protocolo de homologação.

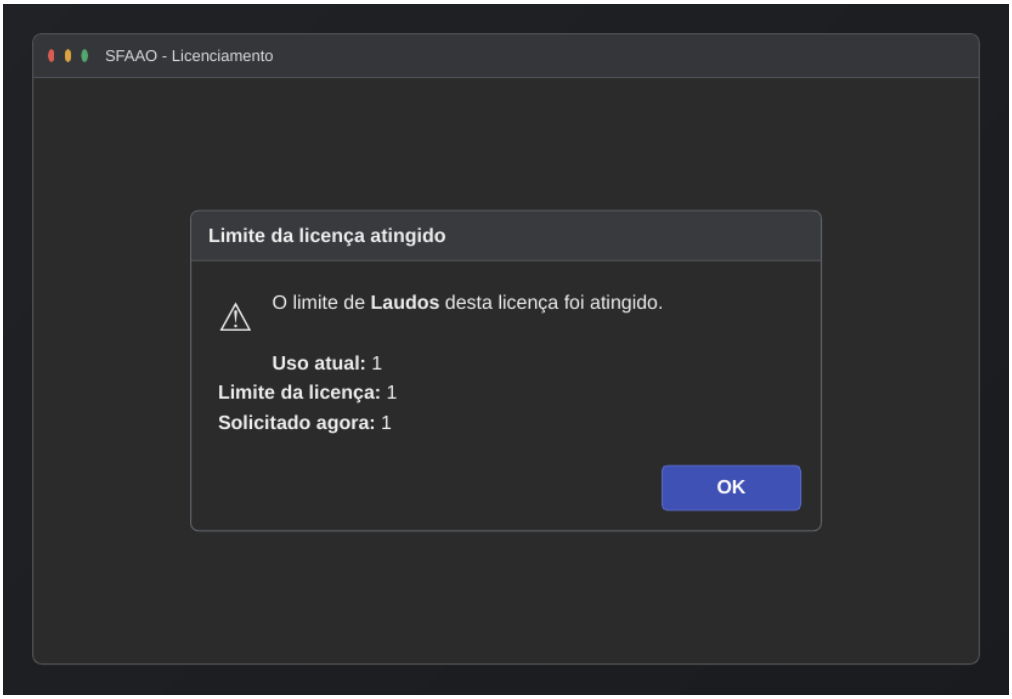


Figura 7.4: Alerta de limite de licença, com recurso afetado, utilização e limite permitido.

7.2.5 Segurança de inicialização e continuidade

A licença, a identidade da máquina, o estado temporal e a integridade do código são verificados antes da operação. O manifesto protegido registra hashes SHA-256 dos arquivos monitorados; arquivos modificados ou ausentes interrompem a inicialização.

Na versão 2.4.0, o payload Linux protegido tem verificações internas de integridade e os quatro arquivos principais do empacotamento instalado foram confrontados com o manifesto `SHA256SUMS`, com resultado `OK`. O manifesto operacional de integridade instalado no Linux corresponde ao SHA-256 eb12e2bce3baf8ee73f426b162c67d0ef39548c7b062c8e4f19a8a2124233f72, verificado com `sha256sum -c SHA256SUMS` em `/opt/sfaao`. No Windows, o instalador 2.4.0 e seu manifesto protegido foram validados no respectivo ciclo de homologação; não se deve reutilizar o hash do Linux como se identificasse o manifesto Windows.

Essa proteção é independente dos manifests científicos de calibração, que documentam a integridade dos artefatos experimentais e seus próprios hashes.

Capítulo 8

Requisitos, instalação e utilização

8.1 Dependências de software, bibliotecas, serviços e requisitos

O SFAAO é uma aplicação desktop predominantemente local. Gestão de casos, persistência, inferência com modelos já disponíveis, calibração com dados locais, comparação e geração de relatórios não dependem de uma API comercial de IA ou de um serviço cloud obrigatório.

8.1.1 Plataforma Python e runtime

Runtime é o ambiente de execução necessário para que a aplicação funcione. O instalador Windows 2.4.0 homologado inclui runtime Python privado, portanto a execução normal **não depende de uma instalação global do Python**. As versões exatas devem ser identificadas no runtime distribuído, e não inferidas de uma especificação preliminar.

Referência de versões utilizada durante o desenvolvimento (verificar no pacote instalado antes de tratá-las como versões definitivas):

```
Python 3.10.11
PyTorch 2.12.0+cpu
torchvision 0.27.0+cpu
CUDA = False
```

A distribuição Windows 2.4.0 entregue é **CPU-only** e utiliza um runtime privado homologado. **CUDA** é a plataforma de computação em GPU da NVIDIA; o instalador Windows CPU não exige GPU NVIDIA.

8.1.2 Bibliotecas principais

Dependência	Função no SFAAO
PyQt5	Interface gráfica desktop.
PyTorch	Redes neurais, treinamento e inferência.
torchvision	Backbones, pesos pretrained e transformações.
NumPy	Cálculo numérico.
OpenCV	Processamento de imagens quando utilizado pelo pipeline.
Matplotlib	Gráficos científicos.
SQLite / sqlite3	Persistência transacional local.
python-docx	Geração de documentos DOCX.
ReportLab	Geração/apoio a PDF quando utilizado pelo pipeline.
cryptography	Operações criptográficas de licença e segurança.
Pillow	Manipulação de imagens.
scikit-learn	Funções auxiliares de avaliação quando utilizadas.

O runtime Python privado acompanha o instalador Windows 2.4.0; a lista exata de dependências deve ser conferida no manifesto específico de cada build homologado.

8.1.3 Internet e pesos pretrained

A operação normal do SFAAO não exige conexão contínua com Internet. Quando `pretrained=true`, o `torchvision` pode precisar baixar o checkpoint do backbone na primeira utilização se esse peso ainda não existir no cache local. Esse download não é uma inferência remota: depois de obtido, o modelo executa localmente.

```
backbone pretrained solicitado
 ↓
checkpoint existe localmente?
 ├─ SIM → carregar localmente
 └─ NÃO → download inicial → cache local
```

Datasets externos devem ser obtidos pelo usuário/instituição diretamente de suas fontes legítimas e de acordo com as respectivas licenças.

8.1.4 Requisitos do sistema operacional

8.1.4.1 Linux

A versão Linux utiliza permissões POSIX restritivas e requer ambiente compatível com as bibliotecas distribuídas. A interface foi validada corretamente no ambiente Linux utilizado durante o desenvolvimento.

8.1.4.2 Windows CPU — requisito do release 2.4.0

A distribuição Windows 2.4.0 CPU foi preparada para Windows 11 x64 e inclui runtime Python privado; os arquivos de programa e os dados persistentes são separados entre Program Files e ProgramData.

8.1.5 Hardware

O SFAAO pode operar integralmente em CPU. Treinamento de modelos profundos pode ser mais rápido em ambientes com GPU compatível; a distribuição Windows CPU descrita neste manual não depende de GPU NVIDIA.

O consumo de memória e tempo depende principalmente de backbone, batch size, quantidade de pares/triplets/imagens, tamanho do dataset, número de épocas e preprocessamento.

Entre os backbones ResNet homologados, a ResNet50 possui maior custo computacional que ResNet18 e ResNet34. A documentação oficial do TorchVision registra 25.557.032 parâmetros e 4,09 GFLOPS para a ResNet50 de referência [17]. Assim, mantendo hardware, resolução, batch size e demais condições comparáveis, **qualquer modo de treinamento do SFAAO que utilize ResNet50 — PAIR, TRIPLET ou CLASSIFICATION — tende a demandar mais tempo de processamento**.

Esse comportamento não define um tempo absoluto: a duração depende de CPU/GPU, quantidade de imagens/pares/triplets, batch size, número de épocas, `freeze_backbone`, I/O, workers e demais parâmetros. O ciclo Windows 2.4.0 incluiu testes funcionais de treinamento; a duração e as métricas dependem da configuração e precisam ser registradas por execução.

8.1.6 Instalação Windows CPU

A distribuição Windows 11 x64 CPU 2.4.0 possui instalador Inno Setup homologado em ambiente de testes, com inicializador `SFAAO.exe` e runtime Python privado. O aplicativo é instalado em `C:\Program Files\SFAAO`, e os dados persistentes em `C:\ProgramData\SFAAO`. As junções do instalador preservam `storage`, `models/default`, `models/custom` e `security/state` durante reinstalação/atualização, conforme o comportamento observado no ciclo de testes [18], [19].

O instalador distribuído é `SFAAO_Setup_2.4.0_CPU_FINAL_PRO_LAUNCHER_RC1.exe`; seu SHA-256 consta na seção de distribuição. A preservação de casos, modelos e configurações existentes e a abertura da interface foram verificadas no Windows 11. Os componentes Python distribuídos são protegidos com PyArmor Pro; o emissor administrativo e a chave privada de licenciamento **não** integram o cliente.

Para ativar, utilizar os utilitários públicos `sfaao-license-request.cmd` e `sfaao-license-install.cmd`; enviar a solicitação ao emissor autorizado, receber `license.sfaao` e instalá-la no armazenamento persistente do cliente. Não distribuir chaves privadas, arquivos administrativos do fabricante ou credenciais em anexos do laudo. A assinatura Authenticode não é declarada como concluída neste manual, por ausência de evidência específica.

8.1.7 Instalação Linux 2.4.0 CPU

A release final possui dois pacotes Linux independentes.

8.1.7.1 Debian, Ubuntu e Linux Mint

A documentação Debian permite instalar arquivos locais `.deb` com `apt install`, resolvendo dependências disponíveis nos repositórios configurados [20]. Para o SFAAO:

```
sudo apt install ./sfaao_2.4.0_amd64.deb
sfaao-license-request
sfaao-license-install /caminho/license.sfaao
sfaao
```

8.1.7.2 Rocky Linux 9 / RHEL-compatible

A documentação Red Hat orienta a instalação de um RPM local com `dnf install <path_to_RPM_file>` [21]. Para o SFAAO:

```
sudo dnf install ./sfaao-2.4.0-1.x86_64.rpm
sfaao-license-request
sfaao-license-install /caminho/license.sfaao
sfaao
```

No Linux, o inicializador `/usr/bin/sfaao` cria, quando ausente, o atalho `~/SFAAO_Data` para `~/.local/share/SFAAO/runtime/data`, com as subpastas `datasets`, `images` e `models`. No contêiner, elas aparecem respectivamente como `/data/datasets`, `/data/images` e `/data/models`. O estado operacional persistente permanece separado do payload protegido.

O script interno `/opt/sfaao/run-sfaao.sh` também admite `SFAAO_ROOT` para ambientes isolados, mas **não cria automaticamente o atalho visível** `~/SFAAO_Data`; este é criado pelo inicializador `/usr/bin/sfaao`. Não direcionar um atalho de produção para o armazenamento temporário de homologação.

No Rocky Linux 9.8 homologado, SELinux permaneceu **Enforcing**. O launcher detecta SELinux e aplica rotulagem privada `:Z` aos volumes controlados pelo SFAAO; durante a instalação da licença, o arquivo recebido é montado somente leitura com `:ro,Z` quando SELinux está ativo. A documentação do Podman descreve `Z` como relabeling privado e `ro` como montagem somente leitura [22].

8.1.8 Proteção do código distribuído — PyArmor Pro

O payload Linux 2.4.0 utiliza **PyArmor Pro 9.2.6**, com runtime `pyarmor_runtime_015424` [23]. A licença PyArmor informa capacidade para BCC e RFT, porém a capacidade licenciada não é tratada como evidência de ativação desses modos no payload: o estado efetivamente distribuído deve ser comprovado pelo script/log do build correspondente. Qualquer alteração desses modos exige novo build e nova

homologação. A proteção aumenta a resistência à inspeção e à reutilização direta do código Python distribuído, sem ser apresentada como garantia absoluta contra engenharia reversa.

PyArmor é uma camada de proteção do código e não substitui o licenciamento operacional próprio do SFAAO, a verificação de integridade, a autenticação, a autorização por papéis ou a rastreabilidade. O regfile/regcode privado do PyArmor, a chave privada de licenciamento SFAAO e o emissor administrativo não pertencem ao cliente e jamais devem ser distribuídos.

Quando `pretrained=true`, o TorchVision pode baixar pesos na primeira utilização se o checkpoint ainda não estiver em cache [17]. O payload Linux 2.4.0 inclui o certificado público necessário ao caminho TLS/CA previsto pela aplicação. Em qualquer teste de treinamento, deve-se registrar se houve download inicial, reutilização de cache ou execução inteiramente offline; esse registro é específico do ambiente e do checkpoint utilizado.

8.2 Uso: adicionar dataset

```
ADMIN
 → Gerenciar Datasets
 → Adicionar Dataset
 → nome
 → profile
 → versão
 → diretório
 → auditoria 0–100%
 → estatísticas
 → cadastro
```

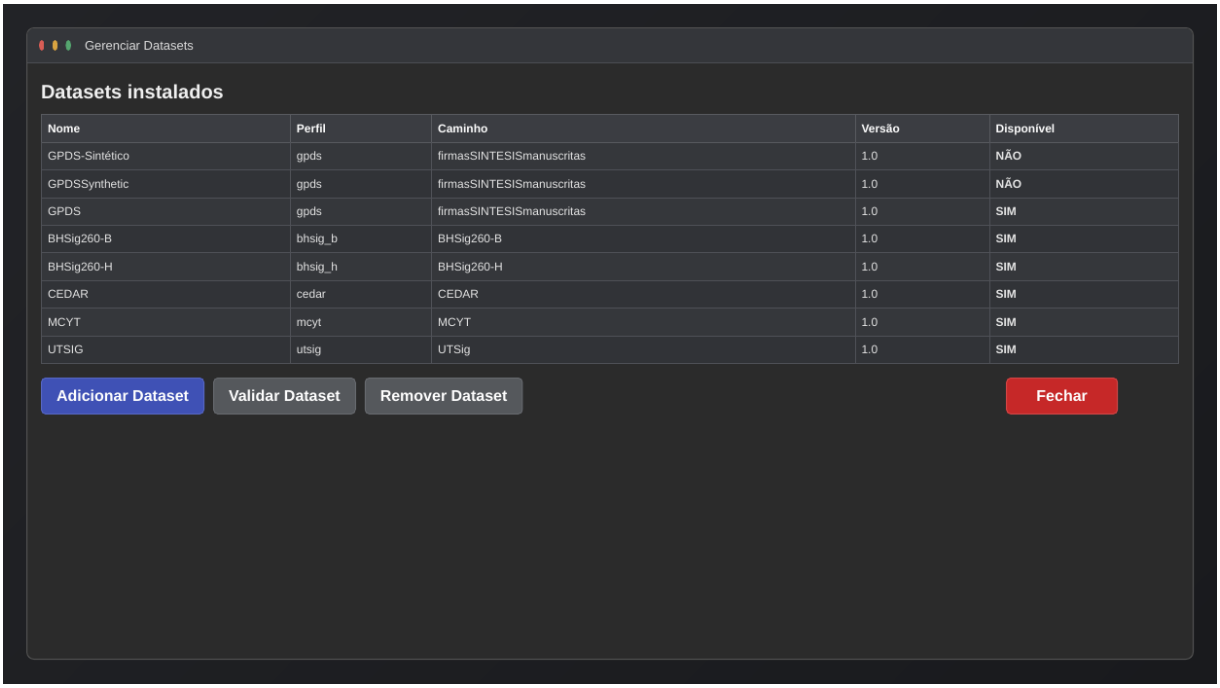


Figura 8.1: Gerenciador de datasets instalados, com catálogo, disponibilidade e ações de inclusão, validação e remoção.

8.3 Uso: modelos

O modelo pode ser:

- treinado pelo SFAAO;
- importado como pacote compatível.

O ADMIN seleciona o modelo operacional ativo. Modelos históricos permanecem vinculados aos resultados já produzidos, mesmo quando outro pacote passa a ser o ativo.

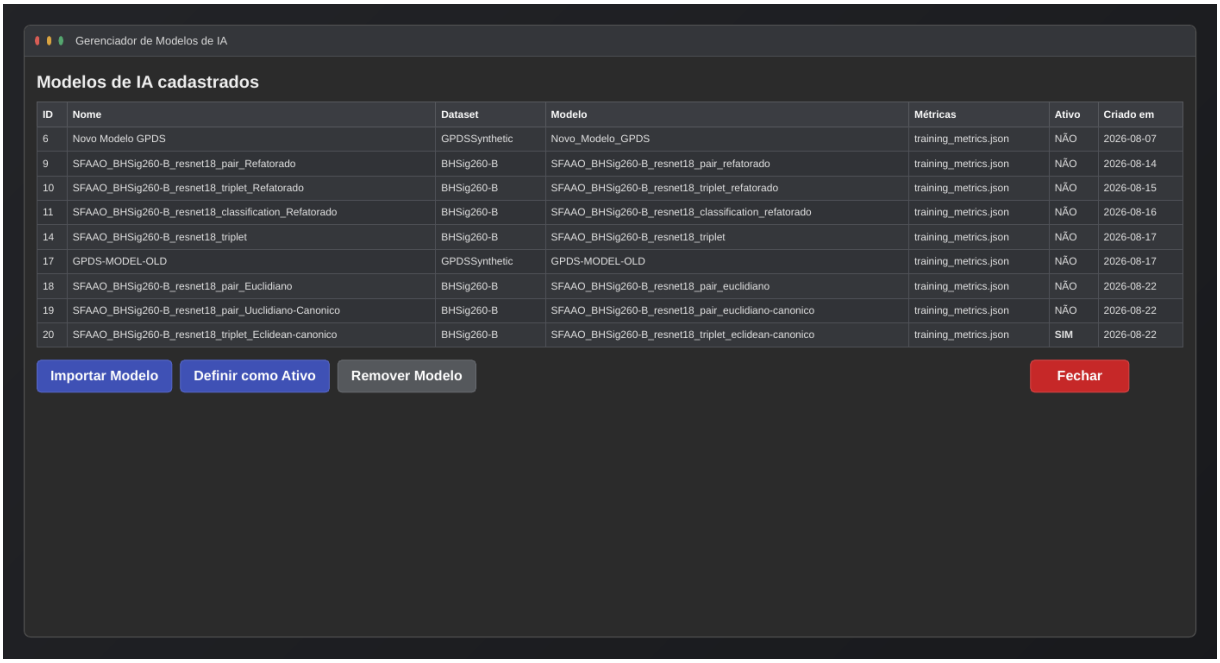


Figura 8.2: Gerenciador de modelos de IA cadastrados, com tabela de modelos e ações de importação, obtenção e remoção.

8.4 Uso: treinamento

O ADMIN configura:

- dataset;
- backbone;
- loss;
- métrica;
- embedding;
- dropout;
- pretrained;
- épocas;
- batch;
- learning rate;
- optimizer;
- scheduler;
- sampling;
- early stopping.

O pacote produzido é cadastrado no Model Manager. O framework final oferece PAIR, TRIPLET e CLASSIFICATION; ResNet18/34/50, MobileNetV3, EfficientNet-B0 e ConvNeXt-Tiny; losses Contrastive, Cosine Contrastive, Triplet e ArcFace conforme o modo; distâncias Cosine e Euclidean; Adam/AdamW; e schedulers none, StepLR, cosine e plateau.

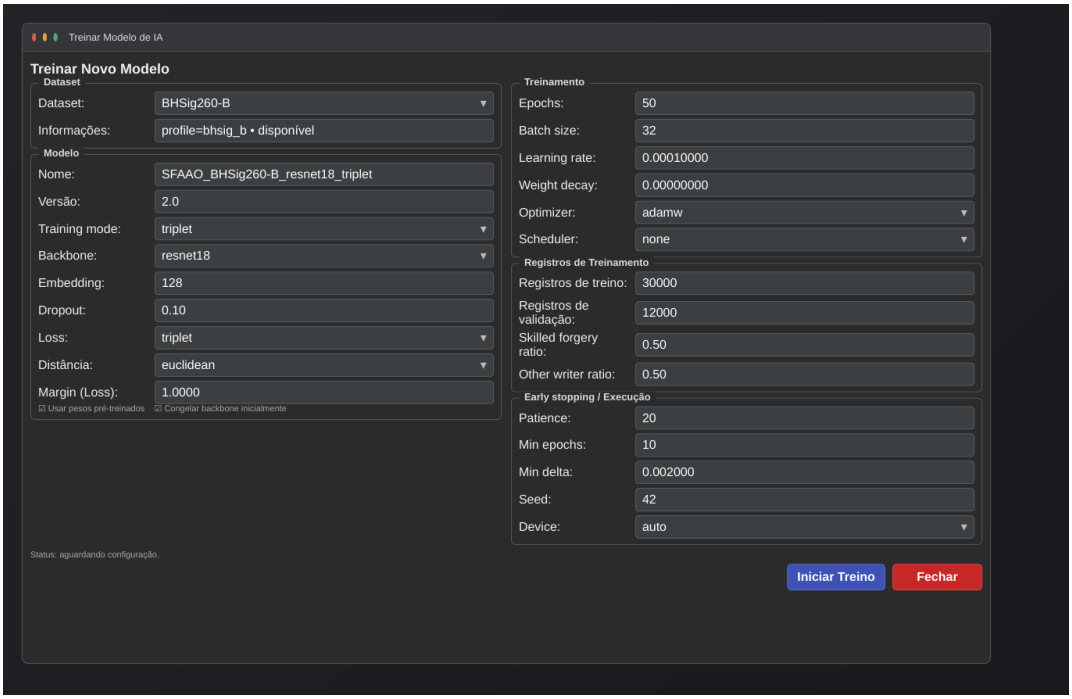


Figura 8.3: Formulário de treinamento, com seleção de dataset, arquitetura, modo, parâmetros de otimização e configuração da execução.

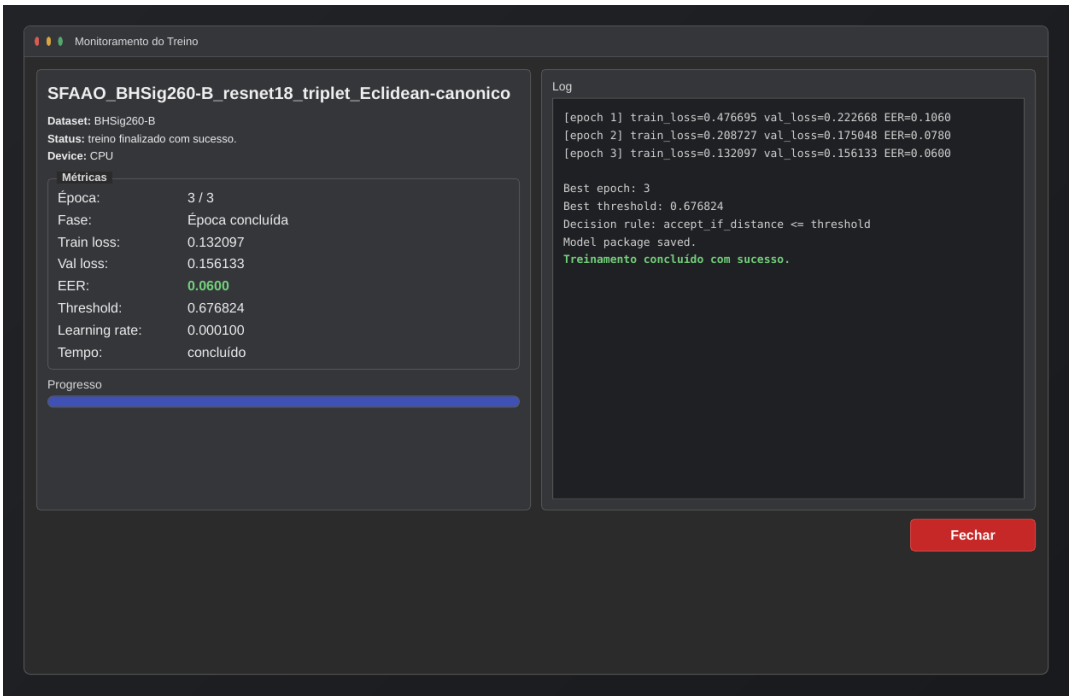


Figura 8.4: Monitor de treinamento, com evolução da execução, métricas e registro de mensagens.

8.5 Uso: calibração

Antes da comparação:

```
caso
 → modelo
 → RC/RO
```

```
→ protocolo
→ parâmetros
→ executar
→ revisar métricas
→ selecionar calibração
```

A abertura da janela de calibração no release final utiliza diálogo de progresso indeterminado e criação da janela na thread principal do Qt, reduzindo a percepção de congelamento sem deslocar widgets para threads secundárias.

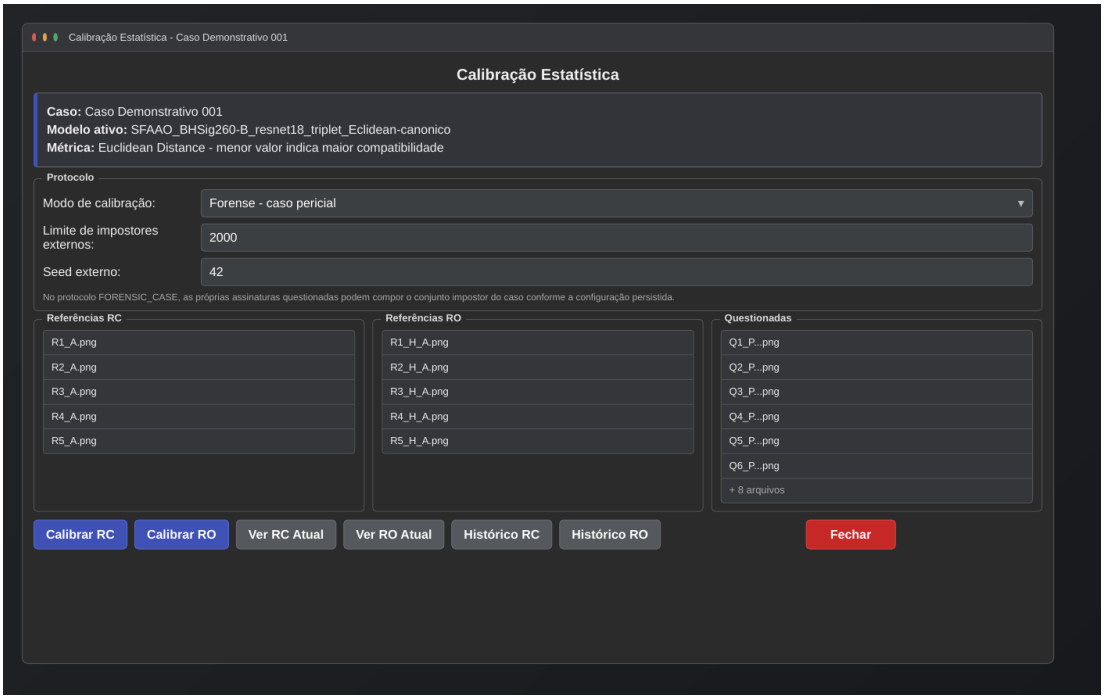


Figura 8.5: Janela de calibração estatística no protocolo forense, com modelo ativo, métrica, parâmetros e conjuntos RC, RO e questionadas.

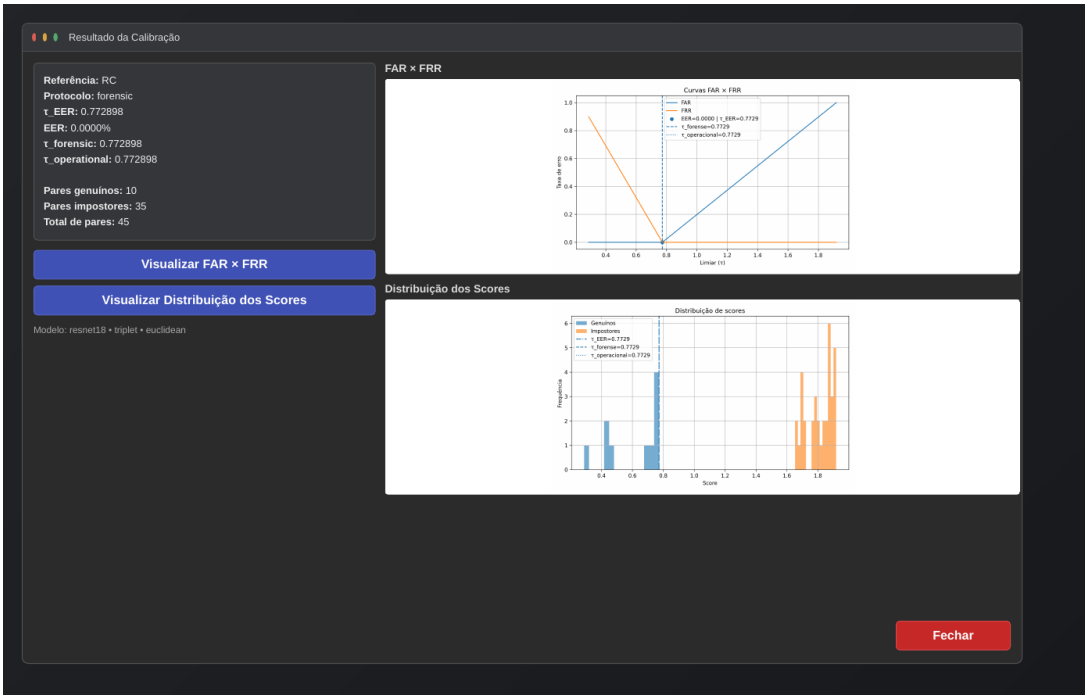


Figura 8.6: Resultados de calibração para referência RC: EER, limiares, contagens de pares, curvas FAR/FRR e distribuição dos scores.

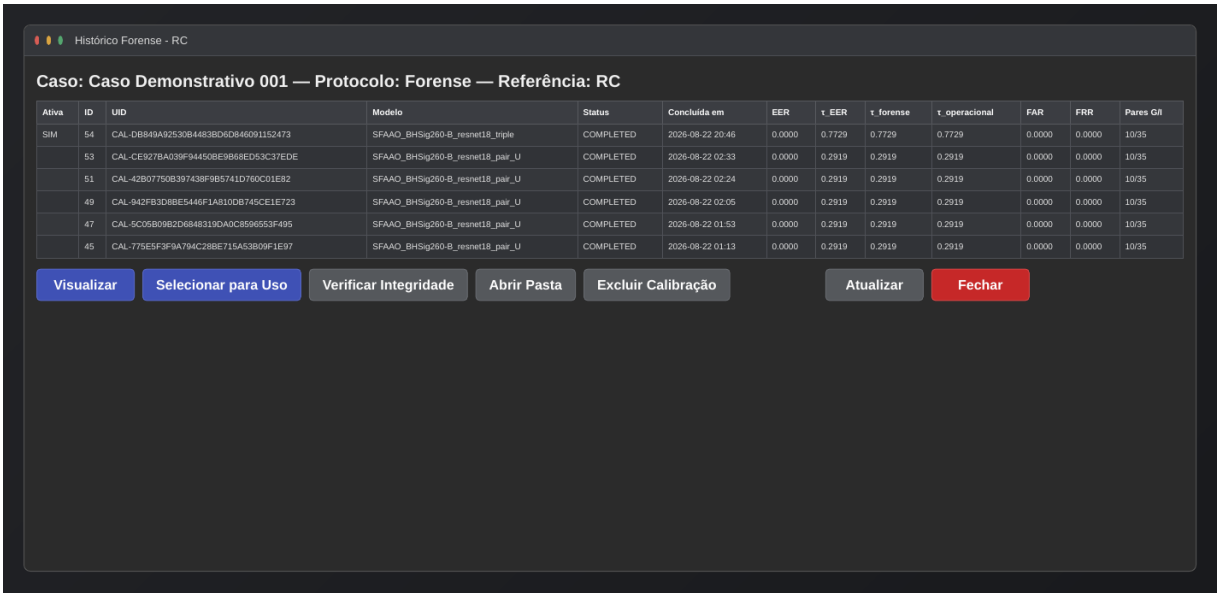


Figura 8.7: Histórico de calibrações do caso, com registros para auditoria, seleção de resultados e ações de visualização.

8.6 Uso: comparação

No release 2.4.0, a abertura de **Comparação de Assinaturas** apresenta uma janela de escolha com **quatro opções**: **(1) Comparação Individual (LORO/A4)**, **(2) Inspecionar Score Pairwise**, **(3) Comparar em Lote** e **(4) Atualizar Contexto**. Esta organização separa a decisão operacional, a inspeção do score elementar, o processamento em lote e a atualização do contexto selecionado.

8.6.1 Comparação Individual (LORO/A4)

Selecionar o caso, o modelo operacional e a calibração válida; escolher as referências **RC** ou **RO** e a assinatura **QUESTIONED**. A execução utiliza o contexto ativo e exige compatibilidade entre a métrica,

o domínio estatístico da calibração e a inferência. No protocolo alinhado LORO, cada fold $A(N-1)$ é a unidade decisória; o score elementar **A1** constitui proveniência, e $A(N)$ pode ser exibido apenas como estatística descritiva, sem receber indevidamente o limiar LORO.

8.6.2 Inspecionar Score Pairwise

Permite examinar um score calculado entre duas assinaturas para fins de diagnóstico e rastreabilidade. **O score A1 isolado não equivale automaticamente ao resultado operacional LORO/A4**, nem substitui a consolidação dos folds e a política de decisão definida na calibração.

8.6.3 Comparar em Lote

Executa as comparações elegíveis dentro do contexto do caso e apresenta a síntese dos resultados persistidos, com acesso ao detalhe dos folds quando aplicável. A exportação CSV permanece condicionada às permissões/cotas da licença. Verificar registros, referências, domínio decisório, estabilidade e limiares antes de utilizar o resultado em um laudo.

8.6.4 Atualizar Contexto

Atualiza as informações do caso/modelo/calibração usadas pela janela de comparação. Utilizar após alterar o caso ativo, as referências, o modelo ou a calibração. A atualização não deve ser interpretada como execução de nova comparação ou recalibração.

8.6.5 Regra de decisão e interpretação

Para **Euclidean Distance**, menor distância representa maior compatibilidade e a regra de aceitação utiliza $d \leq \tau$. Para **Cosine Similarity**, maior score representa maior compatibilidade e a regra utiliza $s \geq \tau$. Aplicar exclusivamente o threshold compatível com o protocolo e a estatística decisória selecionados. Os resultados computacionais não classificam autonomamente a autenticidade das assinaturas QUESTIONED.

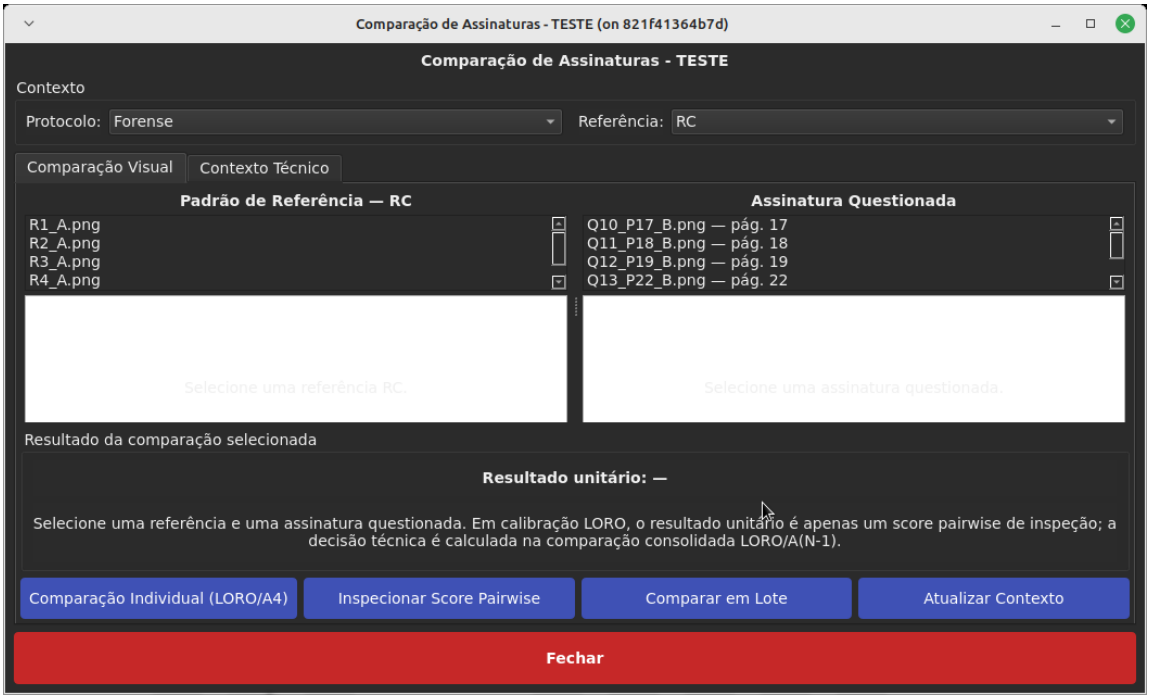


Figura 8.8: Comparação de Assinaturas 2.4.0: contexto ativo, referências RC/RO, assinaturas questionadas e quatro ações disponíveis.

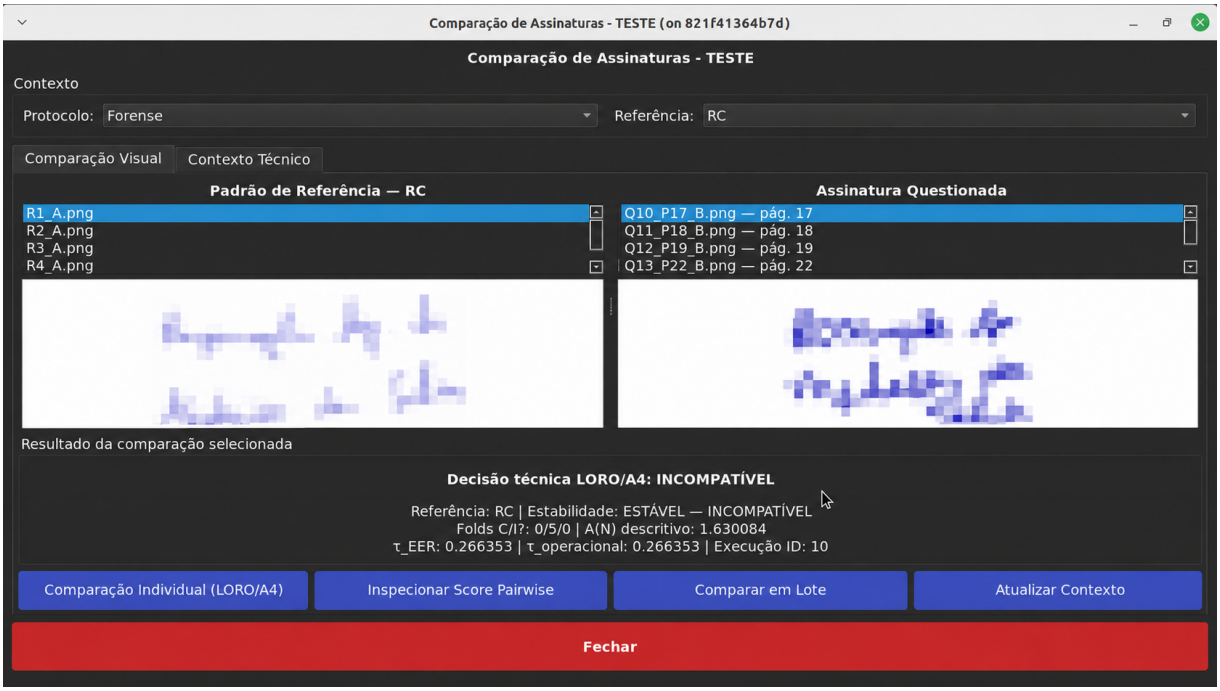


Figura 8.9: Exemplo de comparação individual LORO/A4 com referência RC: amostras selecionadas, estabilidade e decisão técnica apresentada pela interface.

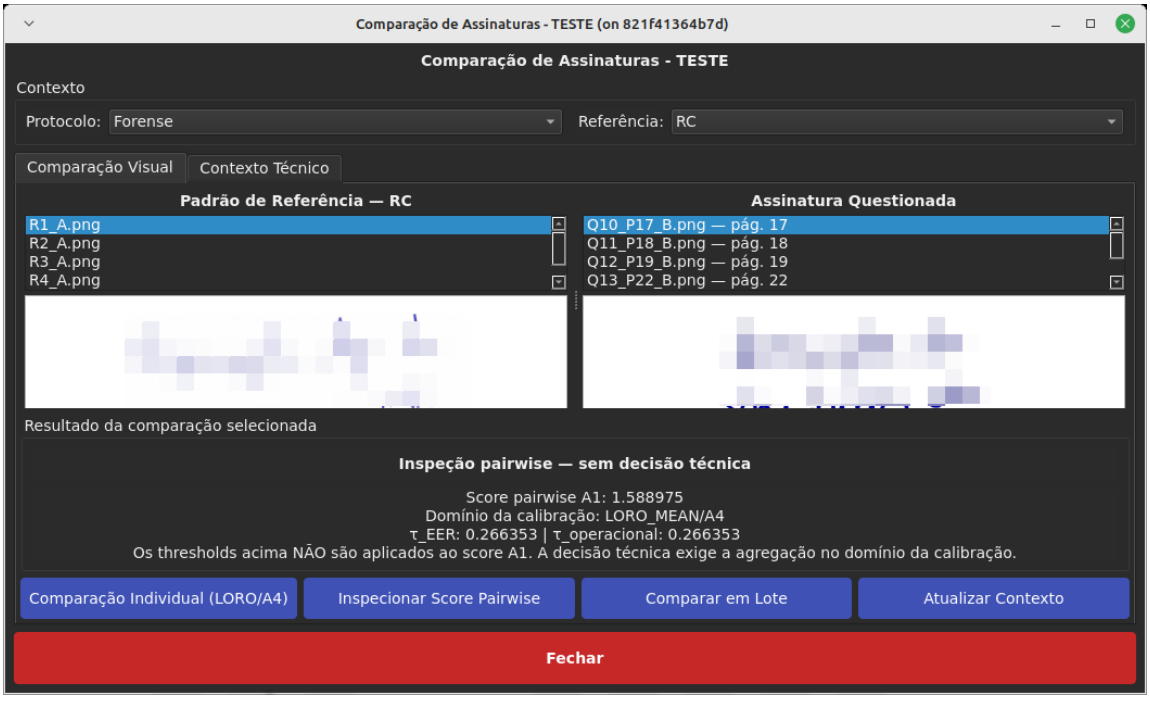


Figura 8.10: Inspeção de score pairwise A1: avaliação pontual sem decisão técnica; os limiares de calibração não são aplicados diretamente a esse score.

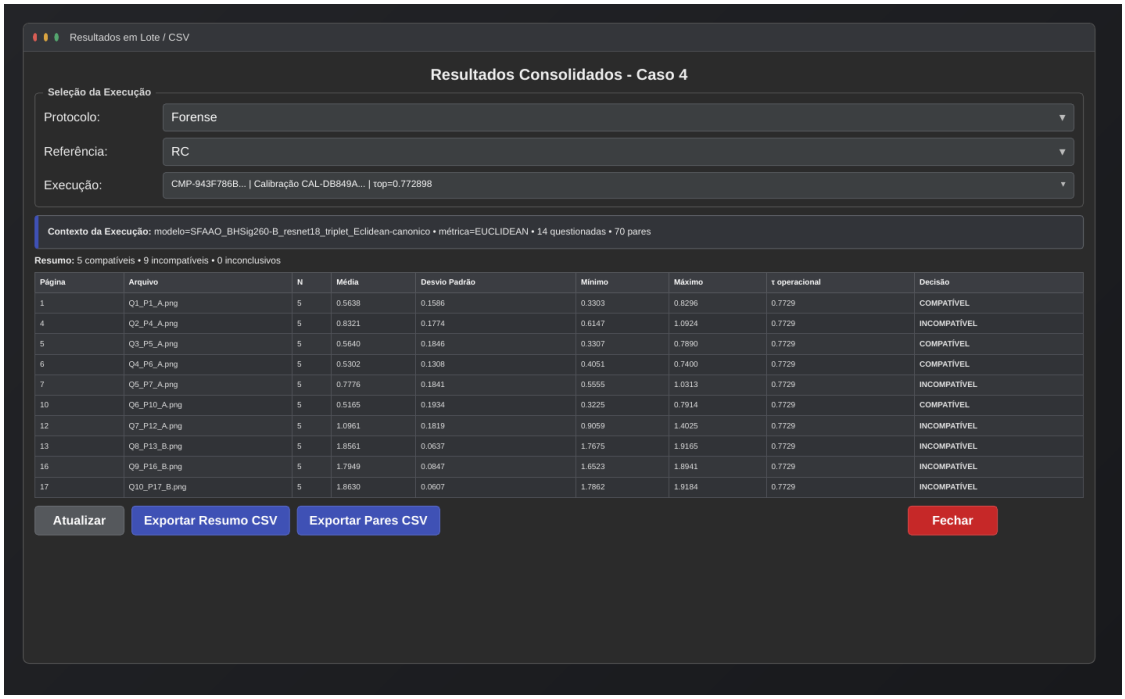


Figura 8.11: Tabela de resultados consolidados da comparação em lote, com filtros e opções de exportação CSV.

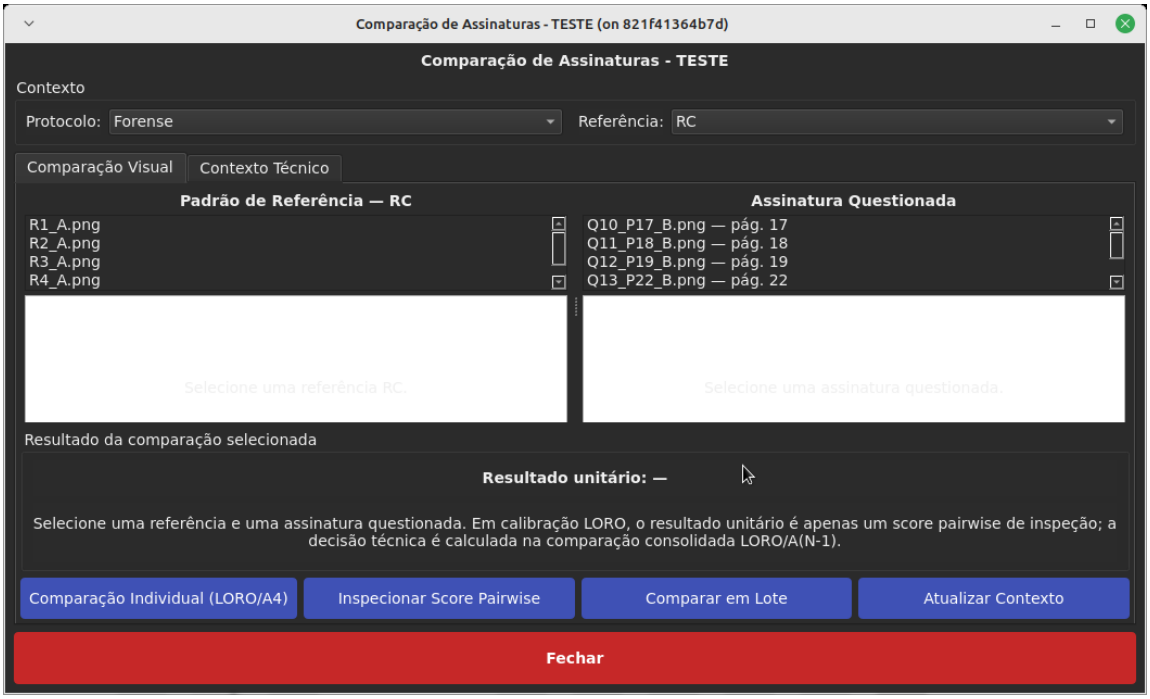


Figura 8.12: Janela após Atualizar Contexto: seleção e resultado unitário limpos, sem iniciar automaticamente uma nova comparação.

8.7 Uso: laudo

O relatório deve ser emitido após revisão do caso, do modelo, da calibração, da comparação e dos dados profissionais do perito. A conclusão grafoscópica é de responsabilidade humana e deve ser revisada antes da entrega.

Geração DOCX homologada nas três plataformas do SFAAO 2.4.0: a emissão de laudos DOCX foi validada funcionalmente no Windows 11, no Linux Mint e no Red Hat, segundo confirmação do

responsável técnico. No Linux Mint, o ambiente isolado executou a imagem incorporada aos pacotes de 19/10/2026, com hash da imagem TAR verificado. No Windows, a funcionalidade foi exercitada no ciclo do instalador 2.4.0 com launcher dedicado. No Red Hat, a homologação funcional incluiu o pacote RPM e a geração DOCX. Esses resultados não substituem a revisão de formatação, conteúdo técnico, integridade das evidências e assinatura do laudo pelo perito responsável.

Quando permitido pela operação, DOCX e PDF representam formatos complementares do mesmo laudo lógico, preservando a identificação e a governança de cotas descritas na seção de relatórios. A emissão de um segundo formato não é uma nova conclusão pericial. Registrar o identificador do caso, modelo, execução, calibração e versão do laudo.

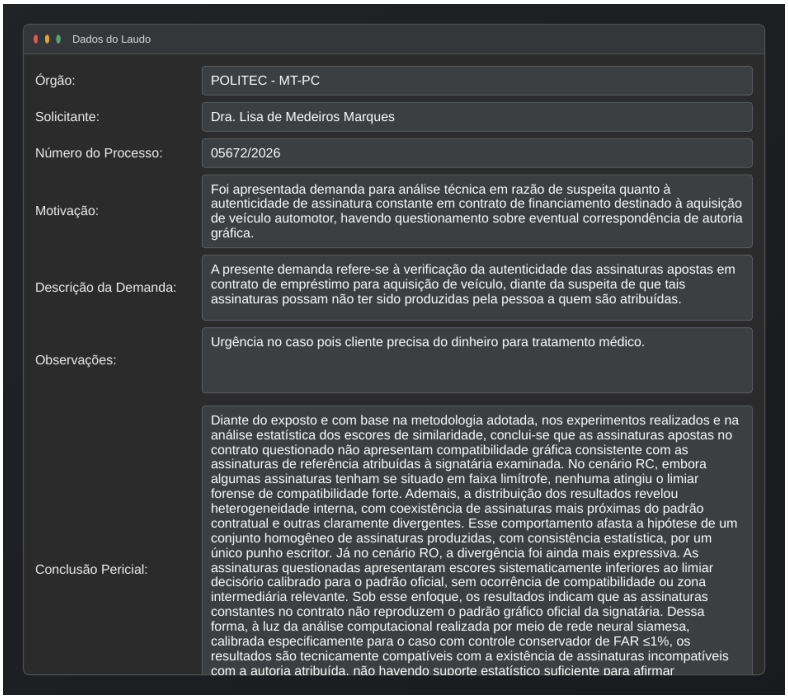


Figura 8.13: Formulário de informações descritivas do laudo pericial, incluindo identificação, fundamentação e conclusão.

A figura a seguir apresenta o arquivo de laudo no formato **DOCX**, aberto no LibreOffice Writer. O formato editável permite a revisão humana do texto e da apresentação antes da emissão do documento definitivo. A imagem posterior mostra separadamente a visualização do laudo em PDF.

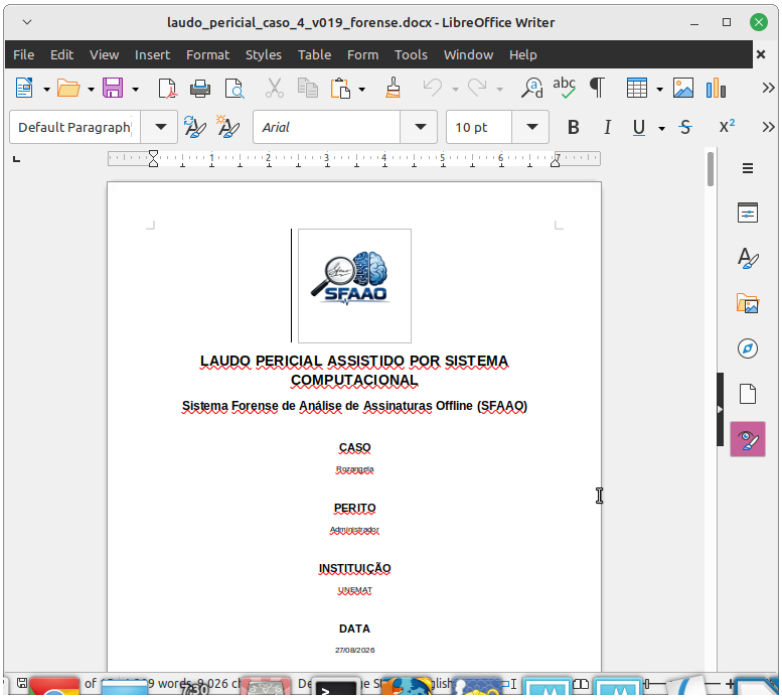


Figura 8.14: Laudo pericial assistido por sistema computacional, no formato DOCX, aberto no LibreOffice Writer. A barra de título identifica o arquivo com extensão `.docx`; a página exibida contém a identificação do caso, do perito e da instituição.

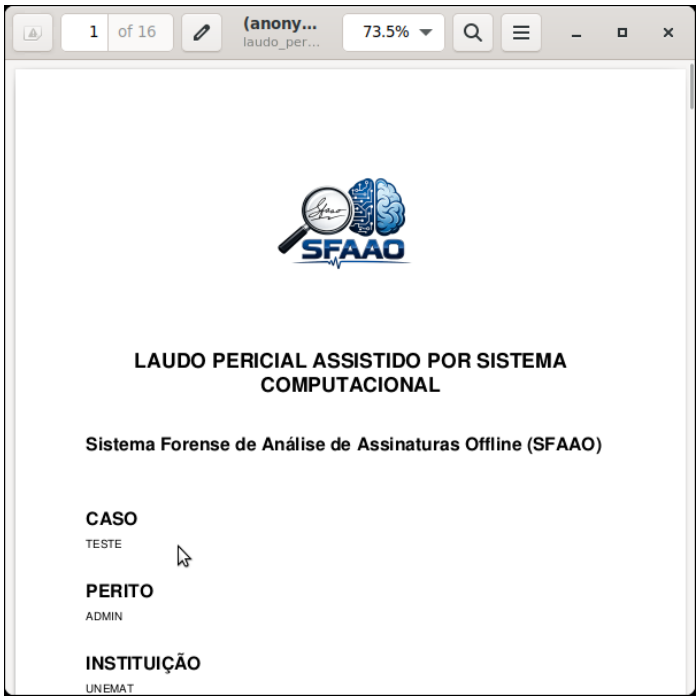


Figura 8.15: Primeira página de um laudo pericial SFAAO 2.4.0 aberta em um visualizador de PDF, com identificação do caso, do perito e da instituição. A figura representa o laudo em PDF, não o documento DOCX.

Capítulo 9

Governança, distribuição e homologação

9.1 Política de datasets e responsabilidade de aquisição

9.1.1 Datasets não fazem parte do produto comercial

O SFAAO é distribuído **sem bases de dados de assinaturas de terceiros**.

A aquisição/licença do software SFAAO **não inclui**:

```
GPDS
CEDAR
BHSig260
UTSig
MCYT
ou qualquer outro dataset de terceiros
```

O produto comercial contém o software, os mecanismos de integração e, quando aplicável, **profiles/adapters** capazes de interpretar determinadas estruturas de bases.

O conteúdo das bases permanece propriedade e responsabilidade de seus respectivos titulares.

9.1.2 Responsabilidade do usuário ou instituição

Para treinar um modelo ou utilizar uma base externa em calibração científica, o usuário/instituição deverá:

```
1. identificar o dataset desejado
2. consultar os termos de licença
3. obter autorização quando necessária
4. realizar o download diretamente da fonte legítima
5. armazenar o dataset localmente
6. selecionar o profile correspondente no SFAAO
7. validar a estrutura da base
8. utilizar a base apenas dentro dos direitos concedidos
```

O SFAAO **não contorna mecanismos de acesso, autenticação ou licenciamento de datasets**.

9.1.3 Suporte não significa redistribuição

Quando o manual informa que o SFAAO “suporta GPDS”, “suporta CEDAR” ou “suporta BHSig260”, isso significa:

> a arquitetura possui ou pode possuir um perfil capaz de interpretar a estrutura da base legitimamente obtida pelo usuário.

Não significa que:

- a base está embutida no instalador;

- a licença do SFAAO concede direito sobre a base;
- o dataset é vendido junto com o software;
- o SFAAO redistribui imagens desses datasets.

9.1.4 Situação das licenças

As condições de uso variam entre datasets e podem mudar. Portanto, antes de uso comercial ou institucional, os termos vigentes na fonte oficial devem ser conferidos.

Como política conservadora de distribuição do SFAAO 2.4.0:

```
NENHUM DATASET DE TERCEIRO É REDISTRIBUÍDO
```

Essa política independe de o profile estar presente na instalação.

9.1.5 Dataset para treinamento

Treinamento:

```
dataset legitimamente obtido
 ↓
armazenamento local pela instituição
 ↓
cadastro no SFAAO
 ↓
validação
 ↓
geração de pares/triplets
 ↓
treinamento
```

9.1.6 Dataset para SCIENTIFIC_EXTERNAL

Calibração científica:

```
caso
 ├─ RC
 ├─ RO
 └─ QUESTIONED
       │
       │  QUESTIONED reservada à inferência
       │
       ▼
dataset externo legitimamente obtido
 ↓
seleção de impostores externos
 ↓
calibração RC ou RO
```

O dataset externo não substitui RC/RO e não incorpora as questionadas à população de calibração.

9.1.7 Bases próprias da instituição

Uma instituição pode utilizar:

- dataset próprio;
- base sintética própria;
- acervo institucional;
- dataset de terceiro autorizado.

Para bases novas, deve ser criado ou configurado um profile/adaptador compatível, preservando classificação, writer IDs e rastreabilidade.

9.1.8 Privacidade, cadeia de custódia e direitos

Antes de importar bases próprias, a instituição é responsável por verificar:

- base legal para tratamento dos dados;
- direitos de uso;
- finalidade;
- política de retenção;
- anonimização/pseudonimização quando aplicável;
- cadeia de custódia;
- proteção de dados pessoais.

O SFAAO fornece mecanismos técnicos de análise e armazenamento; a conformidade jurídica da base importada depende do contexto institucional.

9.2 Política de modelos

O release comercial inicial é distribuído:

```
SEM MODELOS TREINADOS
```

A instituição treina seu próprio modelo ou importa um modelo cujo uso esteja autorizado.

Modelos usados em homologação não entram no instalador.

9.3 Distribuição

A distribuição SFAAO 2.4.0 CPU é organizada em `SFAAO_DISTRIBUTION_2.4.0`. O conteúdo comercial corresponde à plataforma e não inclui datasets de terceiros, modelos de homologação, casos reais nem ferramentas privadas do fabricante.

- **DEB (amd64):** `01_DEB/sfaao_2.4.0_amd64.deb`. SHA-256: `5e7c730ed0cdd59426d896f31dc8a277ef897fba104aa38511cec0e3abc522a5`.
- **RPM (x86_64):** `02_RPM/sfaao-2.4.0-1.x86_64.rpm`. SHA-256: `d5ead5e0fe03e2fb10fef602afc93200ff568398cda20745dc237ebfb8de4d4e`.
- **Windows CPU:** `03_WINDOWS_EXE/SFAAO_Setup_2.4.0_CPU_FINAL_PRO_LAUNCHER_RC1.exe`. SHA-256: `5de43c5dba6559e55442e4c25c6ba13303c95561aef09d2630059b160f78c66d`.

O manifesto único é `06_CHECKSUMS/SHA256SUMS`. Antes da instalação, verificar seu conteúdo a partir da raiz da distribuição com `sha256sum -c 06_CHECKSUMS/SHA256SUMS` em Linux, ou calcular e confrontar os hashes com ferramenta equivalente no Windows. As cópias finais DEB/RPM/EXE retornaram **OK** na conferência em 27/10/2026. O hash interno da imagem OCI usada nos dois pacotes Linux é `e2d04f319fb57ff712af26deb5efe666bb2763d26495f287e53bd9c029dba362`.

A integridade do RPM e de seu payload foi verificada e **a instalação e a operação funcional na plataforma Red Hat foram homologadas**, conforme confirmação do responsável técnico, incluindo geração DOCX. Essa validação se aplica ao ambiente efetivamente ensaiado e não deve ser extrapolada automaticamente a todas as versões de Fedora, Rocky ou AlmaLinux.

9.3.1 O que o comprador recebe

O comprador/instituição recebe o pacote oficial da plataforma, runtime, launchers, ferramentas cliente de solicitação/instalação de licença, profiles/adapters autorizados, mecanismos de treinamento, inferência, calibração, comparação, relatórios, segurança, integridade, licenciamento e documentação pública necessária à operação autorizada.

9.3.2 O que o comprador não recebe

O cliente não recebe datasets de terceiros, imagens biométricas de terceiros, modelos de homologação, casos reais, bancos de desenvolvimento, caches privados, chave privada, emissor de licenças ou regfile privado do PyArmor. A aquisição do SFAAO não transfere automaticamente direitos sobre datasets de terceiros.

9.3.3 Fluxo após aquisição

```
obter pacote oficial + verificar SHA-256
 -> instalar SFAAO
 -> executar sfaao-license-request
 -> enviar SFAAO_LICENSE_REQUEST.json ao emissor autorizado
 -> receber license.sfaao
 -> executar sfaao-license-install
 -> criar primeiro ADMIN
 -> obter dataset legitimamente
 -> cadastrar/validar dataset
 -> treinar modelo próprio ou importar modelo autorizado
 -> operar casos e protocolos permitidos pela licença
```

9.4 Testes e homologação

Em outubro de 2026, os pacotes DEB, RPM e Windows CPU 2.4.0 foram reunidos na distribuição oficial e seus hashes SHA-256 foram conferidos. Os pacotes Linux, produzidos em 19/10/2026 após a correção DOCX, contêm manifestos correspondentes e a mesma imagem OCI, cujo conteúdo foi verificado diretamente dentro de cada instalador.

9.4.1 Linux Mint, DEB e contêiner Podman

A instalação sob `/opt/sfaao` passou na verificação de seus quatro arquivos pelo `SHA256SUMS`: imagem OCI, script de instalação, script de execução e manifesto de integridade. O launcher utiliza Podman preferencialmente, expõe a GUI por X11 e mantém o estado persistente sob `~/.local/share/SFAAO`. O atalho `~/SFAAO_Data` aponta ao diretório de dados principal e contém `datasets`, `images` e `models`.

Foi utilizado ambiente isolado `SFAAO_HOMOLOG_DOCX_2.4.0`, com dados separados, licença de teste e imagem equivalente à dos instaladores. O responsável técnico confirmou o funcionamento da aplicação e **a geração bem-sucedida de laudo DOCX**. A homologação geral do release foi também confirmada nas plataformas Windows 11 e Red Hat; o registro desta sessão isolada não substitui o detalhamento dos ensaios científicos de cada plataforma.

9.4.2 Windows 11 x64 CPU

O instalador 2.4.0 com launcher dedicado foi compilado e testado no Windows 11, incluindo integridade do payload protegido, instalação sobre versão anterior com preservação de dados e operação funcional. **A geração de laudos DOCX também foi homologada funcionalmente no Windows 11**, conforme confirmação do operador. Os módulos protegidos do Windows foram auditados. O SHA-256 do EXE da distribuição corresponde ao artefato homologado.

9.4.3 Red Hat (RPM): instalação e operação homologadas

O pacote RPM 2.4.0 passou nas verificações estruturais, de digest, de manifesto e de hash da imagem incorporada. O responsável técnico confirmou ainda a **homologação da instalação e do funcionamento do SFAAO em Red Hat, incluindo a geração de laudos DOCX**. A versão específica do Red Hat e a ata detalhada dos testes não foram informadas nesta revisão documental; essa confirmação não constitui certificação automática de todas as distribuições RPM compatíveis.

9.4.4 Política de fechamento

A homologação é vinculada ao artefato e ao ambiente testado. Alteração de código, recursos, runtime, ferramentas cliente ou instalador exige análise de impacto, novos hashes e testes proporcionais. Dados biométricos de terceiros, chaves privadas e emissores de licença permanecem fora da distribuição.

9.5 Limitações

- qualidade da imagem influencia desempenho;
- generalização depende do dataset;
- domain shift pode alterar distribuições;
- IA é probabilística;
- thresholds precisam de contexto;
- calibração é específica do modelo e caso;
- quantidade/representatividade das referências é relevante;
- o SFAAO não substitui a análise grafoscópica.

Capítulo 10

Boas práticas e arquitetura técnica detalhada

10.1 Boas práticas

- preservar imagens originais;
- documentar origem das referências;
- utilizar caso correto;
- verificar modelo ativo;
- calibrar antes de comparar;
- não reutilizar calibração de outro modelo;
- não alterar evidências depois da comparação;
- preservar artifacts;
- revisar UIDs;
- revisar o laudo;
- utilizar backup consistente;
- para SQLite em WAL, preferir SQLite Backup API em vez de cópia bruta do banco aberto.

10.2 Extensibilidade

A plataforma pode receber:

- novos backbones;
- novas losses;
- novas métricas;
- novos optimizers;
- novos schedulers;
- novos profiles;
- novas estratégias de amostragem;
- novas calibrações;
- novos relatórios;
- futuras modalidades biométricas baseadas em embeddings.

10.3 Estrutura recomendada do repositório privado

```
README.md
CHANGELOG.md
CONTRIBUTING.md
SECURITY.md
LICENSE
docs/
  ARCHITECTURE.md
  INSTALLATION.md
  USER_MANUAL.md
  TUTORIAL.md
  DATASETS.md
  SECURITY.md
  TRACEABILITY.md
```

```
  GLOSSARY.md
app.py
ai/
controllers/
database/
reports/
security/
storage_services/
ui/
scripts/
tests/
requirements.txt
```

10.3.1 Mapa da implementação: módulos, arquivos e classes

Esta seção relaciona a arquitetura conceitual do SFAAO aos principais componentes da implementação 2.4.0 confirmados na base revisada durante a homologação. O objetivo não é substituir a documentação do código-fonte, mas permitir que mantenedores identifiquem rapidamente onde cada responsabilidade está localizada.

10.3.1.1 Inicialização, autenticação e administração

- **Entrada da aplicação** — `app.py` — inicialização do runtime
- **Primeiro administrador** — `controllers/first_admin_controller.py` — `FirstAdminController`
- **Primeiro administrador - GUI** — `ui/first_admin_window.py` — `FirstAdminWindow`
- **Login** — `ui/login_window.py` — `LoginWindow`
- **Usuários** — `database/repositories/user_repository.py` — `UserRepository`
- **Administração de usuários** — `controllers/admin_user_controller.py` — `AdminUserController`
- **Administração de usuários - GUI** — `ui/user_manager_window.py` — `UserManagerWindow`
- **Política criptográfica** — `security/auth.py` — funções de autenticação/hash
- **Permissões do storage** — `security/storage_permissions.py` — proteção POSIX/Windows
- **Janela principal** — `ui/main_window.py` — `MainWindow`

10.3.1.2 Casos, assinaturas e operação pericial

- **Operações do caso** — `ui/case_operations_window.py` — `CaseOperationsWindow`
- **Metadados do caso** — `ui/case_metadata_window.py` — `CaseMetadataWindow`
- **Perfil do perito** — `ui/expert_profile_window.py` — `ExpertProfileWindow`
- **Persistência de assinaturas** — `database/repositories/signature_repository.py` — `SignatureRepository`
- **Visualização de imagem** — `ui/image_viewer_window.py` — `ImageViewerWindow`
- **Auditoria de papel pericial** — `signature_role_audit` — tabela e operações associadas

10.3.1.3 Datasets

- **Referência de dataset** — `ai/datasets/dataset_reference.py` — `DatasetReference`
- **Perfil estrutural** — `ai/datasets/dataset_profile.py` — `DatasetProfile`
- **Acesso/validação** — `ai/datasets/dataset_provider.py` — `DatasetProvider`
- **Catálogo** — `ai/datasets/dataset_catalog.py` — `DatasetCatalog`
- **Factory** — `ai/datasets/dataset_factory.py` — factory de dataset
- **Loader** — `ai/datasets/dataset_loader.py` — loaders de amostras
- **Dataset de triplets** — `ai/datasets/signature_triplet_dataset.py` — dataset TRIPLET
- **Dataset de classificação** — `ai/datasets/signature_classification_dataset.py` — dataset CLASSIFICATION
- **Controller** — `controllers/dataset_controller.py` — `DatasetController`

- **GUI** — `ui/dataset_manager_window.py` — `DatasetManagerWindow`

Os profiles pertencentes a `ai/datasets/profiles/` descrevem estruturas de bases suportadas sem redistribuir as próprias bases.

10.3.1.4 Configurações

- **Modelo** — `ai/config/model_config.py` — `ModelConfig`
- **Treinamento** — `ai/config/training_config.py` — `TrainingConfig`
- **Calibração** — `ai/config/calibration_config.py` — `CalibrationConfig`

Essas configurações compõem a identidade reprodutível do experimento e não devem ser tratadas como simples preferências visuais.

10.3.1.5 Treinamento

- **Pipeline principal** — `ai/training/training_pipeline.py` — `TrainingPipeline`
- **Pipeline base** — `ai/training/base_training_pipeline.py` — abstração de treinamento
- **Capacidades/modos** — `ai/training/training_capabilities.py` — `TrainingCapabilities`
- **Pré-processamento/utilidades** — `ai/training/utils.py` — funções auxiliares
- **Interface de treinamento** — `ui/training_window.py` — `TrainingWindow`
- **Monitoramento** — `ui/training_monitor_window.py` — `TrainingMonitorWindow`

O treinamento utiliza factories para selecionar backbone, loss, optimizer, scheduler, distância e modelo conforme a configuração.

10.3.1.6 Backbones e construção do modelo

O diretório `ai/backbones/` concentra extratores de características. Entre os componentes confirmados/operacionais no SFAAO 2.4.0 estão:

```
ResNet18
ResNet34
ResNet50
EfficientNet-B0
ConvNeXt-Tiny
MobileNetV3
```

Arquivos como `ai/backbones/resnet18.py`, `resnet34.py` e `resnet50.py` utilizam utilidades comuns de construção do extrator. As demais famílias são resolvidas pela infraestrutura modular correspondente do release.

A criação deve ocorrer por factories/configurações, preservando o princípio de que a GUI não instancia arquiteturas científicas diretamente.

10.3.1.7 Modelos e repositório

- **Repositório de modelo** — `ai/models/model_repository.py` — `ModelRepository`
- **Controle de modelos** — `controllers/model_controller.py` — `ModelController`
- **Gerenciador visual** — `ui/model_manager_window.py` — `ModelManagerWindow`

O pacote do modelo deve preservar pesos e metadados suficientes para reconstruir sua configuração.

10.3.1.8 Calibração

- **Base da calibração**
  `ai/calibration/base_calibration_pipeline.py`
  `BaseCalibrationPipeline`

- **Forense**
  `ai/calibration/forensic_calibration_pipeline.py`
  `ForensicCalibrationPipeline`
- **Científica**
  `ai/calibration/scientific_calibration_pipeline.py`
  `ScientificCalibrationPipeline`
- **Construção de pares** — `ai/calibration/pair_builder.py` — `PairBuilder`
- **Estatísticas** — `ai/calibration/calibration_statistics.py` — cálculos FAR/FRR/EER
- **Resultado** — `ai/calibration/calibration_result.py` — `CalibrationResult`
- **Execução** — `ai/calibration/calibration_execution.py` — `CalibrationExecution`
- **Artifact** — `ai/calibration/calibration_artifact_builder.py` — `CalibrationArtifactBuilder`
- **Repositório de artifact** — `ai/calibration/calibration_artifact_repository.py` — repositório de artifacts
- **Repositório no banco** — `database/repositories/calibration_repository.py` — `CalibrationRepository`
- **GUI** — `ui/calibration_window.py` — `CalibrationWindow`
- **Histórico** — `ui/calibration_history_window.py` — `CalibrationHistoryWindow`
- **Resultados** — `ui/calibration_results_window.py` — `CalibrationResultsWindow`

10.3.1.9 Inferência e comparação

- **Inferência** — `ai/inference/inference_pipeline.py` — `InferencePipeline`
- **Comparação** — `ai/comparison/comparison_pipeline.py` — `ComparisonPipeline`
- **GUI de comparação** — `ui/compare_window.py` — `CompareWindow`
- **Resultados em lote** — `ui/batch_results_window.py` — `BatchResultsWindow`
- **Persistência** — `database/repositories/comparison_repository.py` — repositório de comparações

10.3.1.10 Relatórios

- **Orquestração** — `controllers/report_controller.py` — `ReportController`
- **Consolidação** — `reports/report_pipeline.py` — pipeline de relatório
- **DOCX** — `reports/docx_report_generator.py` — gerador DOCX
- **PDF** — `reports/pdf_report_generator.py` — gerador PDF

Os relatórios consomem dados já persistidos; não devem recalcular silenciosamente a ciência do caso com parâmetros diferentes daqueles associados à execução registrada.

10.3.1.11 Regra para manutenção

O mapa acima representa o release 2.4.0 homologado. Em futuras versões, uma mudança de nome ou localização de classe deve atualizar este manual e o changelog, sem alterar silenciosamente a semântica científica da plataforma.

10.3.2 Modelo de dados e schema SQLite

O SFAAO utiliza SQLite para metadados, rastreabilidade, auditoria e relacionamentos. Imagens, pesos e artefatos científicos permanecem no filesystem e são referenciados pelo banco.

O schema possui versionamento próprio; no ciclo do release 2.4.0 há estruturas identificadas internamente por versão de schema, como o schema v6. O número de versão do **schema** não deve ser confundido com a versão do **protocolo científico** ou com a versão comercial do SFAAO.

10.3.2.1 Tabelas centrais

Tabela	Função
schema_versions	versionamento/migrações do schema
users	credenciais, roles e estado do usuário
expert_profiles	dados profissionais do perito
cases	unidade central da perícia
case_metadata	informações descritivas e conclusão humana
signatures	RC, RO e QUESTIONED
signature_role_audit	histórico de mudanças de classificação pericial
datasets	catálogo local de datasets legitimamente obtidos
trained_models	catálogo de modelos registrados
calibrations	execução/configuração de calibração caso+modelo
calibration_samples	amostras explicitamente registradas para calibração quando aplicável
calibration_results	métricas e artifacts da calibração
comparison_runs	execução identificável de comparação
similarity_results	scores individuais referência-questionada
questioned_results	agregação e decisão por questionada
reports	versões de relatórios emitidos

10.3.2.2 Relações principais

```
USERS
  ├── EXPERT_PROFILES
  └── CASES
        ├── CASE_METADATA
        ├── SIGNATURES
        │     └── SIGNATURE_ROLE_AUDIT
        ├── CALIBRATIONS ──── TRAINED_MODELS
        │     ├── CALIBRATION_SAMPLES
        │     └── CALIBRATION_RESULTS
        ├── COMPARISON_RUNS ── TRAINED_MODELS
        │     ├── SIMILARITY_RESULTS
        │     └── QUESTIONED_RESULTS
        └── REPORTS
```

10.3.2.3 users

Campos funcionais incluem:

```
id
username
password_hash
full_name
email
role
is_active
failed_login_attempts
locked_until
created_at
updated_at
```

Os dois últimos campos de segurança de login (`failed_login_attempts` e `locked_until`) fazem parte do hardening do release e devem existir em instalações novas ou ser acrescentados por migração defensiva.

10.3.2.4 cases

O caso possui `case_uid`, usuário/perito associado, nome, descrição, status, diretório de storage e timestamps. O `case_uid` fornece identidade estável independente do ID inteiro local.

10.3.2.5 signatures

A tabela diferencia:

```
RC
RO
QUESTIONED
```

Também preserva `forensic_role` para a classificação humana das questionadas quando aplicável.

10.3.2.6 calibrations

A calibração registra, entre outros:

```
calibration_uid
case_id
model_id
created_by
calibration_mode
reference_type
distance_name
status
artifact_dir
configuration_path
parameters_json
is_selected
timestamps
```

Os modos centrais documentados neste manual são:

```
FORENSIC_CASE
SCIENTIFIC_EXTERNAL
```

Qualquer valor legado/adicional presente no schema não deve ser utilizado como protocolo operacional sem documentação e validação correspondentes.

10.3.2.7 calibration_results

Pode armazenar:

```
eer
eer_threshold
forensic_threshold
target_far
far
frr
```

```
genuine_count
impostor_count
genuine_mean/genuine_std
impostor_mean/impostor_std
bootstrap confidence interval
paths de métricas e gráficos
```

O nome interno legado `forensic_threshold` pode existir no armazenamento mesmo quando a apresentação científica utiliza a expressão mais geral **Limiar calibrado**. A persistência histórica não deve ser renomeada de forma destrutiva apenas por terminologia de interface.

10.3.2.8 `comparison_runs`

Cada execução liga explicitamente:

```
case_id
model_id
calibration_id
created_by
reference_type
distance_name
threshold
status
artifacts/parâmetros
```

Essa estrutura impede que um resultado seja interpretado sem conhecer o modelo e a calibração que o originaram.

10.3.2.9 Integridade referencial

O runtime habilita foreign keys. Registros científicos utilizados por comparações ou relatórios possuem proteções contra exclusões que destruiriam a rastreabilidade.

Exemplos:

```
calibração usada → não remover livremente
assinatura usada → não remover livremente
caso com evidência histórica → proteger
```

10.3.2.10 SQLite em WAL

A aplicação utiliza configurações adequadas a um desktop transacional, incluindo WAL, `synchronous` e `busy_timeout` conforme o runtime homologado. Para backup de banco aberto, deve-se utilizar mecanismo consistente como a SQLite Backup API.

10.3.3 Propriedade intelectual e identificação do software

O SFAAO possui Registro de Programa de Computador expedido pelo Instituto Nacional da Propriedade Industrial (INPI):

```
Processo: BR512026005360-9
Título: SFAAO: Sistema Forense de Análise de Assinaturas Offline
        Baseado em Deep Learning e Redes Neurais Siamesas
```

```
        com Calibração Estatística Pericial
Data de criação: 27/03/2026
Titular: Universidade do Estado de Mato Grosso - UNEMAT
Autor: Diógenes Antonio Marques José
Linguagem registrada: Python; outros
Expedição do certificado: 14/07/2026
```

A existência do registro do software não deve ser confundida com eventual proteção patentária de método. Informações de patente e segredo técnico devem seguir as orientações da UNEMAT/AGINOV e do procedimento de propriedade intelectual correspondente.

10.3.4 Declaração de uso de Inteligência Artificial Generativa

O desenvolvimento do **SFAAO - Software Forense para Análise de Assinaturas Offline** contou com o apoio exclusivo do **GPT, por meio do ChatGPT, desenvolvido pela OpenAI**, como ferramenta de Inteligência Artificial Generativa. Nenhum outro sistema de IA generativa foi utilizado no processo de desenvolvimento descrito neste manual.

O GPT foi empregado como ferramenta de assistência ao longo do desenvolvimento, contribuindo para atividades como análise e revisão de código-fonte, identificação de inconsistências, discussão e refinamento de arquitetura de software, elaboração e revisão de algoritmos, estruturação de pipelines de treinamento, inferência e calibração, preparação e revisão de testes, análise de consistência, documentação técnica, organização metodológica e apoio à redação científica.

A interação com o GPT ocorreu de forma iterativa e supervisionada. Sugestões produzidas pelo modelo foram avaliadas, testadas, corrigidas e validadas antes de sua incorporação ao SFAAO. Portanto, o uso do GPT é declarado como **assistência computacional ao desenvolvimento**, e não como substituição da autoria humana, da programação, da validação experimental, da responsabilidade científica ou da atuação profissional do perito.

Todas as decisões relativas à concepção do SFAAO, definição de requisitos, metodologia científica, protocolos de calibração, arquitetura, implementação, testes, validação, interpretação de resultados e utilização pericial permanecem sob responsabilidade humana. O GPT não realiza perícia grafoscópica, não determina autoria documental e não assume responsabilidade por conclusões periciais produzidas com auxílio da plataforma.

O GPT e o ChatGPT são tecnologias desenvolvidas pela **OpenAI**. Registra-se também reconhecimento histórico a **Sam Altman, cofundador da OpenAI**, pela sua participação na trajetória da organização e no desenvolvimento do ecossistema contemporâneo de IA generativa. Essa menção possui caráter de reconhecimento e transparência e **não implica autoria, endosso, certificação, parceria institucional ou responsabilidade da OpenAI ou de Sam Altman sobre o SFAAO**.

10.3.4.1 Créditos de desenvolvimento

```
Concepção, desenvolvimento, metodologia e responsabilidade científica:
Diógenes Antonio Marques José

Assistência de Inteligência Artificial Generativa:
GPT / ChatGPT - OpenAI

Reconhecimento institucional e histórico:
OpenAI; Sam Altman
```

A declaração acima integra a política de transparência do projeto e deve ser preservada nas versões oficiais deste manual.

Capítulo 11

Glossário, listas de verificação e encerramento

11.1 Glossário

ADMIN — papel de governança.

EXAMINER — papel operacional pericial.

Backbone — extrator de características.

Embedding — vetor representativo.

Pair — par de amostras.

Triplet — anchor, positive, negative.

RC — assinaturas colhidas diretamente do subscritor reproduzindo o padrão que ele utiliza em contratos; são referências conhecidas e não provêm do contrato questionado.

RO — assinaturas colhidas diretamente do subscritor reproduzindo o padrão que ele utiliza em documentos oficiais; são usadas como referências oficiais para calibração e comparação.

QUESTIONED — qualquer assinatura existente no contrato sob exame; todas permanecem questionadas mesmo depois de receber classificação pericial humana.

QUESTIONED_COMPATIBLE — assinatura QUESTIONED classificada pelo perito grafoscopista como compatível com o padrão do subscritor; continua sendo QUESTIONED.

QUESTIONED_IMPOSTOR — classificação pericial humana utilizada como non-match no protocolo RC.

FORENSIC_CASE — calibração vinculada ao caso.

SCIENTIFIC_EXTERNAL — calibração independente com impostores externos; o caso continua contendo RC, RO e QUESTIONED, porém as QUESTIONED são excluídas da estimação do threshold e usadas somente na etapa posterior de avaliação/inferência.

FAR — False Acceptance Rate; proporção de non-matches aceitos incorretamente. A desigualdade depende da métrica: cosseno usa score alto; Euclidiana usa distância baixa.

FRR — False Rejection Rate; proporção de matches rejeitados incorretamente.

EER — Equal Error Rate; taxa associada ao ponto em que FAR e FRR se aproximam no protocolo avaliado.

τ_EER — threshold correspondente ao EER estimado.

τ_operacional — threshold efetivamente aplicado; usa orientação dependente da métrica, conforme as Seções 39 e 40.

Cosine Similarity — similaridade angular; maior valor indica maior compatibilidade no SFAAO.

Euclidean Distance — distância geométrica; menor valor indica maior compatibilidade no SFAAO.

DatasetProfile — descrição estrutural de uma base.

DatasetProvider — acesso padronizado ao dataset.

PairBuilder — construtor de populações de pares.

ComparisonRun — execução identificável de comparação.

Calibration UID — identificador de calibração.

Selection hash — SHA-256 da seleção externa.

Proveniência — origem verificável dos dados.

Rastreabilidade — reconstrução da cadeia que produziu um resultado.

Reprodutibilidade — repetição do experimento sob as mesmas condições.

WAL — Write-Ahead Logging do SQLite.

11.1.1 Glossário de componentes de treinamento

As definições formais, finalidade, momento de uso e aplicação concreta dos termos abaixo estão consolidadas na Seção 23.1. Este glossário é uma referência rápida, não substitui o referencial conceitual.

PAIR — modo de treinamento baseado em pares de imagens rotulados como semelhantes ou diferentes.

TRIPLET — modo de treinamento baseado em anchor, positive e negative.

CLASSIFICATION — modo supervisionado baseado em classes/identidades fornecidas pelo dataset.

Optimizer — algoritmo responsável por atualizar os parâmetros treináveis da rede.

Adam — otimizador adaptativo.

AdamW — variante do Adam com weight decay desacoplado.

Scheduler — mecanismo que altera o learning rate durante o treinamento.

StepLR — scheduler baseado em reduções periódicas.

Cosine scheduler — scheduler com curva cossenoidal.

Plateau scheduler — reduz learning rate quando a métrica monitorada estabiliza.

Pretrained — inicialização do backbone com pesos previamente aprendidos.

Freeze backbone — congelamento dos parâmetros do extrator de características.

Batch size — quantidade de amostras processadas por atualização do otimizador.

Learning rate — intensidade de atualização dos parâmetros.

Weight decay — regularização aplicada durante otimização.

Positive ratio — proporção planejada de pares positivos.

Forgery negative ratio — proporção planejada de negativos construídos a partir de falsificações.

Best state dict — conjunto de pesos correspondente ao melhor estado identificado durante treinamento.

Score distribution — distribuição estatística de scores genuínos e impostores.

11.2 Checklist de operação

```
[ ] usuário correto
[ ] caso correto
[ ] perito correto
[ ] modelo correto
[ ] modelo ativo
[ ] RC conferidas
[ ] RO conferidas
[ ] questionadas conferidas
[ ] papéis periciais revisados
```

```
[ ] calibração selecionada
[ ] threshold conferido
[ ] comparação RC executada
[ ] comparação RO executada
[ ] resultados revisados
[ ] laudo gerado
[ ] anexos conferidos
```

11.3 Checklist de release

✓ Imagens internas DEB/RPM, manifestos e hashes: consistentes.

✓ Pacotes finais DEB e RPM copiados para a distribuição, sem reconstrução.

✓ Instalador Windows 2.4.0 validado e copiado para a distribuição.

✓ `06_CHECKSUMS/SHA256SUMS`: os três instaladores verificaram `OK`.

✓ SFAAO 2.4.0: homologação funcional confirmada em Windows 11, Linux Mint (DEB/Podman) e Red Hat (RPM), incluindo geração de laudos DOCX.

✓ Instalação e execução do RPM em Red Hat: homologadas conforme confirmação do responsável técnico; versão exata e ata de testes devem ser arquivadas no dossiê de release.

✓ As capturas operacionais atualizadas foram incorporadas e revisadas; a documentação visual utiliza exclusivamente a interface homologada de comparação.

☐ Revisão editorial e conferência final dos materiais do cliente: pendente.

☐ Metadados definitivos do artigo aceito (DOI/volume/páginas): incluir após publicação.

11.3.1 Estado de homologação do Manual e do Release 2.4.0

O release dispõe de instaladores DEB, RPM e Windows EXE com hashes validados. O responsável técnico confirmou a homologação funcional nos três ambientes — Windows 11, Linux Mint e Red Hat — incluindo a geração DOCX e a instalação do pacote RPM na plataforma Red Hat. Esta revisão documental incorpora o aceite do artigo científico, ainda sem afirmar publicação formal. As 23 capturas operacionais atualizadas integram esta edição do manual; a imagem legada da comparação foi arquivada separadamente.

11.4 Conclusão

O SFAAO 2.4.0 integra:

```
GESTÃO DE CASOS
+
DEEP LEARNING
+
REDES SIAMESAS
+
EMBEDDINGS
+
DATASET PROFILES
+
PAIR / TRIPLET / CLASSIFICATION
+
TREINAMENTO CONFIGURÁVEL
```

```
+
AVALIAÇÃO
+
INFERÊNCIA
+
DUPLA CALIBRAÇÃO
+
COMPARAÇÃO
+
RASTREABILIDADE
+
AUDITORIA
+
SEGURANÇA
+
DOCUMENTAÇÃO PERICIAL
```

Seu objetivo é transformar uma comparação biométrica em um processo computacional estruturado, rastreável e tecnicamente auditável.

Cada resultado deve ser interpretado juntamente com o modelo, os dados, a calibração, o threshold, a execução e o contexto do caso. A interpretação final e a conclusão grafoscópica permanecem sob responsabilidade do profissional habilitado.

Capítulo 12

Referências bibliográficas e citação

12.1 Referências científicas e técnicas

As referências abaixo fundamentam os principais conceitos científicos utilizados neste manual: Deep Learning, redes neurais siamesas, contrastive loss, arquiteturas ResNet, métricas biométricas, verificação automática de assinaturas, datasets de assinaturas e dataset shift.

> **Situação editorial da referência [16]:** conforme informação atualizada fornecida pelos autores, o artigo do SFAAO foi **aceito para publicação no *IEEE Access*** em 2026. Ainda não foram fornecidos DOI, volume, número, páginas nem data de publicação. Assim, a referência deve indicar **artigo aceito para publicação**; não se deve afirmar que já foi publicado.

[1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015. doi: 10.1038/nature14539.

[2] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah, "Signature verification using a Siamese time delay neural network," in *Advances in Neural Information Processing Systems (NIPS)*, 1994, pp. 737–744. doi: 10.5555/2987189.2987282.

[3] R. Hadsell, S. Chopra, and Y. LeCun, "Dimensionality Reduction by Learning an Invariant Mapping," in *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, New York, NY, USA, 2006, pp. 1735–1742. doi: 10.1109/CVPR.2006.100.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778. doi: 10.1109/CVPR.2016.90.

[5] A. K. Jain, P. Flynn, and A. A. Ross, *Handbook of Biometrics*. Springer, 2008. doi: 10.1007/978-0-387-71041-9.

[6] L. Prechelt, "Early stopping — but when?" in *Neural Networks: Tricks of the Trade*. Springer, 1998, pp. 55–69. doi: 10.1007/3-540-49430-8_3.

[7] R. Plamondon and G. Lorette, "Automatic signature verification and writer identification — The state of the art," *Pattern Recognition*, vol. 22, no. 2, pp. 107–131, 1989. doi: 10.1016/0031-3203(89)90059-9.

[8] M. A. Ferrer, M. Diaz-Cabrera, and A. Morales, "Static signature synthesis: A neuromotor inspired approach for biometrics," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 37, no. 3, pp. 667–680, Mar. 2015. doi: 10.1109/TPAMI.2014.2343981.

[9] University of Las Palmas de Gran Canaria, "GPDS: Grupo de Procesado Digital de Señales," 2025. Online resource. Accessed Feb. 25, 2026.

[10] S. Jain, M. Khanna, and A. Singh, "Comparison among different CNN architectures for signature forgery detection using Siamese neural network," in *Proceedings of the International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, 2021, pp. 481–486. doi: 10.1109/ICCCIS51004.2021.9397114.

[11] M. Aleem et al., "Comparative analysis of deep learning models: CNN, MobileNetV2, and ResNet50 for offline signature verification," *Letters in High Energy Physics*, vol. 2024, pp. 1686–1694, 2024.

[12] Y. A. C. Santos, L. C. Rêgo, and R. Ospina, “Online handwritten signature verification via network analysis,” *Physica A: Statistical Mechanics and its Applications*, vol. 600, p. 127582, 2022. doi: 10.1016/j.physa.2022.127582.

[13] L. Zhang, Y. Guo, Y. Ding, and H. Sato, “1-D CNN-based online signature verification with federated learning,” *arXiv preprint* arXiv:2406.06597, 2024.

[14] X. Zhao, L. Zheng, P. Yuan, and Y. Zheng, “Fusing deep and hand-crafted features by deep canonically correlated contractive autoencoder for offline signature verification,” *Pattern Recognition*, vol. 168, p. 111834, 2025. doi: 10.1016/j.patcog.2025.111834.

[15] J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla, and F. Herrera, “A unifying view on dataset shift in classification,” *Pattern Recognition*, vol. 45, no. 1, pp. 521–530, 2012. doi: 10.1016/j.patcog.2011.06.019.

[16] D. A. M. José, L. de Medeiros Marques, A. Bueno, and R. B. Z. da Silva, “Independent External-Impostor Calibration for Mitigating Dataset Shift in Forensic Offline Signature Verification,” *IEEE Access*, aceito para publicação, 2026. ID do manuscrito Access-2026-35145. DOI e dados bibliográficos finais pendentes.

[17] PyTorch/TorchVision, “resnet50 — Torchvision documentation.” Official documentation. Available: https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet50.html. Accessed Sep. 8, 2026.

[18] Microsoft, “ProgramData,” Windows Hardware / Shell Setup documentation. Available: https://learn.microsoft.com/windows-hardware/customize/desktop/unattend/microsoft-windows-shell-setup-folderlocations-programdata. Accessed Sep. 8, 2026.

[19] Microsoft, “mklink,” Windows Commands. Available: https://learn.microsoft.com/windows-server/administration/windows-commands/mklink. Accessed Sep. 8, 2026.

[20] Debian Project, “Debian Reference — Chapter 2: Debian package management.” Available: https://www.debian.org/doc/manuals/debian-reference/ch02.en. Accessed Sep. 8, 2026.

[21] Red Hat, “Managing software with the DNF tool — Installing RHEL 9 content.” Available: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/managing_software_with_the_dnf_tool/assembly_installing-rhel-9-content_managing-software-with-the-dnf-tool. Accessed Sep. 8, 2026.

[22] Podman, “podman-run — volume options and SELinux labeling.” Available: https://docs.podman.io/en/latest/markdown/podman-run.1.html. Accessed Sep. 8, 2026.

[23] PyArmor Project, “PyArmor 9.2 Documentation.” Available: https://pyarmor.readthedocs.io/en/stable/. Accessed Sep. 8, 2026.

12.2 Relação entre os conceitos do manual e as referências

Para facilitar a leitura técnica, as principais relações são:

Tema do SFAAO	Referências recomendadas
Deep Learning	[1]
Redes neurais siamesas	[2]
Contrastive Loss / metric learning	[3]
ResNet	[4]
FAR, FRR, EER e biometria	[5]
Early stopping	[6]
Fundamentos de verificação automática de assinaturas	[7]

Tema do SFAAO	Referências recomendadas
GPDS e síntese de assinaturas	[8], [9]
Siamese CNN para assinatura offline	[10]
Comparação de CNN, MobileNet e ResNet	[11]
Métodos alternativos de verificação de assinatura	[12]–[14]
Dataset shift	[15]
Calibração independente com impostores externos, separação entre treinamento/calibração/inferência, estudo forense RC/RO e metodologia diretamente relacionada ao SFAAO	[16]

12.2.1 Citações recomendadas ao longo do manual

Ao converter este manual em documentação acadêmica ou técnica formal, recomenda-se inserir citações nas seguintes seções:

- **Deep Learning e embeddings:** [1];
- **Redes siamesas:** [2];
- **Contrastive Loss:** [3];
- **ResNet18/34/50:** [4];
- **FAR, FRR e EER:** [5];
- **Early stopping:** [6];
- **Offline signature verification:** [7], [10], [11], [14];
- **GPDS / assinatura sintética:** [8], [9];
- **Dataset shift:** [15];
- **Independent External-Impostor Calibration:** [16];
- **Separação entre training, calibration e forensic inference:** [16];
- **Exclusão das questionadas da calibração científica:** [16];
- **calibração com 5 referências, 10 genuine pairs e população externa de impostores:** [16].

12.3 Como citar o SFAAO

Enquanto o software não possuir publicação bibliográfica própria definitiva, há duas formas complementares de referência.

12.3.1 Referência ao software

```
JOSÉ, Diógenes Antonio Marques. SFAAO — Sistema Forense de Análise de
Assinaturas Offline. Versão 2.4.0. Universidade do Estado de Mato Grosso
(UNEMAT), 2026. Software.
```

12.3.2 Referência científica à metodologia

> JOSÉ, D. A. M.; MARQUES, L. M.; BUENO, A.; SILVA, R. B. Z. *Independent External-Impostor Calibration for Mitigating Dataset Shift in Forensic Offline Signature Verification*. IEEE Access, **aceito para publicação**, 2026. Manuscrito Access-2026-35145. DOI e dados bibliográficos finais pendentes.

Após a publicação formal, atualizar a referência com DOI, volume, páginas e demais metadados fornecidos pelo periódico, sem alterar a descrição dos resultados científicos ou suas condições experimentais.